Politechnika Warszawska

WYDZIAŁ ELEKTRYCZNY



Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana

w specjalności Inżynieria Oprogramowania

Analiza Wydajności i Bezpieczeństwa Protokołu MCP w Integracji Systemów Agentowych z Wielkimi Zbiorami Danych

Karol Sygiet

numer albumu 340801

promotor

dr inż. Włodzimierz Dąbrowski, prof. uczelni

WARSZAWA 2026

Analiza Wydajności i Bezpieczeństwa Protokołu MCP w Integracji Systemów Agentowych z Wielkimi Zbiorami Danych

Streszczenie

Praca dotyczy wydajności i bezpieczeństwa komponentów bramy pośredniczącej w standardzie Model Context Protocol (MCP), integrującej systemy agentowe oparte na modelach językowych z relacyjną bazą danych o złożonym schemacie. Zaprojektowano i zaimplementowano w języku Python 3.12 bramę zawierającą: selekcję schematu łączącą wyszukiwanie leksykalne i wektorowe (RRF) z autorską ekspansją grafową o tabele mostkujące, wielokolumnową paginację kursorową keyset, transfer danych binarnych poza ramką JSON-RPC (Out-of-Band) z analitycznym modelem progu opłacalności oraz walidator zapytań oparty na analizie drzew AST wykonywany w puli procesów. Na zbiorze 500 zapytań Text-to-SQL w schemacie o skali 1000 tabel ekspansja grafowa istotnie zwiększyła czułość selekcji tabel (Table-Recall@5 55,3% wobec 47,0% dla RRF@8 i 53,2% dla BM25@8 przy równym budżecie tabel), redukując objętość DDL w prompcie o ponad 99% względem pełnego zrzutu schematu (263 113 tokenów). Paginacja keyset zachowuje stałe opóźnienie na relacji 1 mln wierszy (0,43 ms przy offsecie 950 000 wobec 429 ms dla OFFSET/LIMIT), transfer przez pamięć współdzieloną (SHM) na jednym węźle jest szybszy od kodowania Base64 dla ładunków powyżej 12,9 KiB, zgodnie z modelem progu, a wieloprocesowa walidacja AST obniża medianę opóźnienia pętli zdarzeń przy 100 klientach z 975 do 0,40 ms. Walidator zablokował wszystkie 24 badane wektory SQL Injection bez fałszywych alarmów. Wyniki eksperymentalne potwierdziły nadrzędną hipotezę badawczą w odniesieniu do zaimplementowanych komponentów bramy MCP.

Słowa kluczowe: Model Context Protocol, systemy agentowe, bazy danych, duże modele językowe, paginacja kursorowa, pgvector, bezpieczeństwo AI, Text-to-SQL

Performance and Security Analysis of the Model Context Protocol in Integrating Agent Systems with Big Data

Abstract

This thesis investigates the performance and security of the components of a Model Context Protocol (MCP) gateway integrating LLM-based agent systems with a relational database having a complex schema. A gateway was designed and implemented in Python 3.12 featuring: schema selection combining lexical and vector retrieval (RRF) with foreign-key bridge-table expansion, multi-column keyset cursor pagination, out-of-band binary data transfer outside the JSON-RPC frame with an analytical break-even model, and an AST-based query validator executed in a dedicated process pool. On a benchmark of 500 Text-to-SQL queries over a 1000-table schema, graph expansion significantly increased table retrieval recall (Table-Recall@5 of 55.3% versus 47.0% for RRF@8 and 53.2% for BM25@8 at an equal table budget), reducing prompt DDL volume by over 99% compared to the full database dump (263,113 tokens). Keyset pagination maintains constant latency on a 1-million-row table (0.43 ms at offset 950,000 versus 429 ms for OFFSET/LIMIT), shared-memory transfer (SHM) on a single node outperforms Base64 encoding for payloads above 12.9 KiB consistent with the analytical model, and multi-process AST validation reduces the median event-loop lag under 100 concurrent clients from 975 to 0.40 ms. The validator successfully blocked all 24 tested SQL injection attack vectors without false positives. The experimental findings confirm the overarching research hypothesis for the investigated MCP gateway architecture.

Keywords: Model Context Protocol, agentic systems, databases, large language models, cursor pagination, pgvector, AI security, Text-to-SQL

Spis treści

1 Wstęp **11**

1.1 Tło i motywacja . . . 11
1.2 Problem badawczy . . . 11
1.3 Cel pracy . . . 12
1.4 Zakres pracy . . . 12
1.5 Pytania badawcze . . . 13
1.6 Hipotezy badawcze . . . 14
1.7 Metodyka badań . . . 14
1.8 Wykorzystane narzędzia i środowisko . . . 15
1.9 Osiągnięte rezultaty . . . 16
1.10 Główne trudności . . . 16
1.11 Możliwości wykorzystania wyników . . . 17
1.12 Wykorzystanie narzędzi i metod sztucznej inteligencji . . . 17
1.13 Układ pracy . . . 17

2 Teoretyczne podstawy integracji modeli językowych z danymi **19**

2.1 Ewolucja paradygmatów integracji: od odizolowanych LLM do Tool Use . . . 19
2.1.1 Od pamięci parametrycznej do paradygmatu Tool Use . . . 19
2.1.2 Redukcja przestrzeni relacyjnej i ograniczenia metod promptingowych . . . 20
2.2 Geneza standaryzacji interfejsów komunikacyjnych w AI . . . 21
2.2.1 Fragmentacja ekosystemu i redukcja złożoności integracyjnej . . . 21
2.3 Formalna specyfikacja Model Context Protocol (MCP) . . . 21
2.3.1 Role architektoniczne w protokole . . . 21
2.3.2 Cykl życia sesji, transport i prymitywy semantyczne . . . 22
2.4 Specyfika systemów wielkoskalowych w interakcji z agentami . . . 23
2.4.1 Fizyczne i logiczne bariery mechanizmu uwagi . . . 23
2.4.2 Transakcyjność i modele spójności w komunikacji bezstanowej . . . 24
2.4.3 Stochastyczne transakcje agentowe i paradygmat SAGA . . . 24

3 Analiza porównawcza protokołów komunikacyjnych **25**

3.1 Protokół JSON-RPC 2.0 jako rdzeń Model Context Protocol . . . 25
3.1.1 Narzut serializacji tekstowej UTF-8 i tokenizacja danych Base64 . . . 25

3.2 Taksonomia architektur komunikacyjnych w systemach agentowych . . . 26
3.2.1 Charakterystyka porównywanych paradygmatów architektonicznych . . . 26
3.2.2 Model dwoistości stanowej w protokole MCP . . . 27
3.2.3 Wzorcowa tabela taksonomiczna . . . 27
3.3 Równowaga kompromisów: uzasadnienie wyboru JSON-RPC 2.0 . . . 28
3.3.1 Przyczyny odrzucenia architektur REST i gRPC . . . 28
3.3.2 Krytyczna polemika z paradygmatem Code-as-Action . . . 28
3.4 Relacja MCP do protokołów komunikacji wieloagentowej (ACP i A2A) . . . 29
3.5 Wektory ataków i deterministyczne bezpieczeństwo protokołów . . . 29
3.5.1 Autoryzacja i podatność Confused Deputy w środowiskach MCP . . . 29
3.5.2 Zagrożenie pośredniego wstrzykiwania instrukcji i perymetr obronny . . . 29

4 Projekt systemu integracji MCP z hurtownią danych 31
4.1 Architektura logiczna i model referencyjny bramy pośredniczącej . . . 31
4.1.1 Koncepcja trójwarstwowej architektury systemu . . . 31
4.1.2 Model separacji domen i bezpieczeństwa . . . 32
4.1.3 Model przepływu komunikatów synchronicznych i asynchronicznych . . . 32
4.2 Formalizacja mechanizmu adaptacyjnej paginacji kursorowej . . . 35
4.2.1 Formalizacja techniki Keyset Pagination dla kluczy kompozytowych . . . 35
4.2.2 Model bezstanowego kursora nieprzejrzystego (Opaque Cursor Token) . . . 36
4.2.3 Izolacja snapshotowa w paginacji bezstanowej . . . 37
4.2.4 Algorytm adaptacyjnej estymacji budżetu tokenów Welforda . . . 38
4.3 Koncepcja progresywnego odkrywania schematu (JIT Schema Discovery) . . . 39
4.3.1 Model grafowy schematu relacyjnego . . . 39
4.3.2 Problem utraty relacji asocjacyjnych w wyszukiwaniu leksykalno-semantycznym 40
4.3.3 Model fuzji rankingów Reciprocal Rank Fusion . . . 40
4.3.4 Algorytm ekspansji grafowej o tabele mostkujące . . . 40
4.4 Analityczny model transferu binarnego Out-of-Band (Crossover Model) . . . 41
4.4.1 Problem narzutu kodowania binarnego w JSON-RPC . . . 41
4.4.2 Separacja pasma sterującego od pasma danych . . . 41
4.4.3 Analityczny model punktu przecięcia (Break-even Latency Model) . . . 42
4.5 Architektura bezpieczeństwa i izolacji współbieżnej pętli zdarzeń . . . 43
4.5.1 Założenia perymetru bezpieczeństwa w systemach bazodanowych Agentic AI 43
4.5.2 Projekt wielopoziomowego perymetru ochronnego AST . . . 43
4.5.3 Model izolacji współbieżnej pętli zdarzeń . . . 44

5 Implementacja i środowisko badawcze 45
5.1 Architektura środowiska uruchomieniowego . . . 45
5.2 Implementacja autorskiego serwera MCP (Middleware Gateway) . . . 48
5.3 Silnik semantycznego odkrywania schematu . . . 49
5.4 Mechanizm adaptacyjnej paginacji kursorowej . . . 49
5.5 Transfer danych binarnych Out-of-Band i asynchroniczne strumieniowanie . . . 50

5.6 Moduł bezpieczeństwa zapytań (SQL Sanitizer) . . . 50
5.7 Klient agentowy oraz zautomatyzowana aparatura pomiarowa . . . 51

6 Wyniki badań i ewaluacja eksperymentalna 55
6.1 Metodyka pomiarowa i formalizacja aparatu ewaluacyjnego . . . 55
6.1.1 Formalizacja metryk oceny . . . 55
6.1.2 Środowisko pomiarowe i protokół eksperymentu . . . 57
6.1.3 Procedura wnioskowania statystycznego . . . 58
6.2 Semantyczna jakość odkrywania schematu relacyjnego (Scenariusz A) . . . 59
6.2.1 Zbiór zapytań i badane strategie . . . 59
6.2.2 Jakość selekcji schematu . . . 60
6.2.3 Dokładność generacji Text-to-SQL . . . 62
6.2.4 Dekompozycja błędów generacji zapytań . . . 65
6.2.5 Badanie ablacyjne parametru ekspansji grafowej $\mathrm{M}_{\mathrm{exp}}$ . . . 66
6.2.6 Czas do pierwszego tokenu w funkcji rozmiaru promptu . . . 67
6.3 Analiza progu opłacalności transferu binarnego Out-of-Band (Scenariusz B1) . . . 68
6.3.1 Warunki pomiaru . . . 68
6.3.2 Punkty przecięcia . . . 69
6.3.3 Reguła architektoniczna . . . 71
6.4 Paginacja, obciążenie współbieżne i bilans zasobów (Scenariusze B2 & C1) . . . 71
6.4.1 Skalowalność paginacji na wolumenie 1 000 000 rekordów . . . 71
6.4.2 Wpływ postaci predykatu kursorowego na plan wykonania . . . 72
6.4.3 Odporność na obciążenie wieloagentowe i profilowanie zasobów . . . 73
6.4.4 Dryf schematu w trakcie obsługi zapytań . . . 76
6.5 Odporność perymetru bezpieczeństwa (Scenariusz D) . . . 76
6.6 Formalna weryfikacja hipotez badawczych . . . 78
6.6.1 Statystyczna weryfikacja hipotez cząstkowych . . . 78
6.7 Konfrontacja wyników z literaturą . . . 79
6.7.1 Kontekst wskaźnika Execution Accuracy (DIN-SQL, DAIL-SQL, MAC-SQL) . 79
6.7.2 Granice deterministycznej inspekcji AST w świetle badań nad Prompt-to-SQL 79

7 Podsumowanie i wnioski 81
7.1 Synteza osiągniętych rezultatów inżynierskich i badawczych . . . 81
7.2 Weryfikacja hipotezy badawczej i odpowiedzi na pytania badawcze . . . 82
7.3 Macierz kompromisów architektonicznych . . . 84
7.4 Krytyczna ocena ograniczeń protokołu MCP i opracowanego rozwiązania . . . 84
7.5 Perspektywy rozwoju i kierunki dalszych prac badawczych . . . 88

Bibliografia 89

Wykaz skrótów i symboli 95

Spis rysunków **99**

Spis tabel **101**

Spis załączników **103**

Rozdział 1

Wstęp

1.1 Tło i motywacja

Duże modele językowe (ang. *Large Language Models*, LLM), oparte na architekturze Transformer [54], bardzo dobrze radzą sobie z tekstem, ale same w sobie są odcięte od otoczenia. Ich wiedza pochodzi z danych treningowych i z czasem się starzeje, a model, który jedynie odpowiada na pytania, nie może sam sprawdzić niczego w bazie danych ani wykonać operacji w systemie informatycznym.

Rozwiązaniem tego problemu stało się udostępnianie modelom narzędzi [42, 50]. Model pełni wtedy rolę agenta: planuje kolejne kroki, wywołuje zewnętrzne funkcje i na podstawie ich wyników decyduje, co zrobić dalej, np. w pętli ReAct (*Reasoning and Acting*) [60].

W latach 2023–2024 każda biblioteka i każdy dostawca modeli definiowali jednak własny sposób opisywania i wywoływania narzędzi (m.in. LangChain [9]). Połączenie M modeli z N narzędziami wymagało więc w praktyce $\mathrm{M} \times \mathrm{N}$ osobnych integracji. Pod koniec 2024 roku opublikowano otwartą specyfikację *Model Context Protocol* (MCP) [2], rozwijaną obecnie pod egidą Linux Foundation. MCP wyraźnie oddziela klienta (aplikację z modelem) od serwera udostępniającego narzędzia i dane, dzięki czemu wystarcza $\mathrm{M} + \mathrm{N}$ implementacji.

1.2 Problem badawczy

Przy łączeniu agentów z relacyjnymi bazami danych sama liczba wierszy nie jest głównym problemem, ponieważ miliony rekordów da się sprawnie przetworzyć na jednym serwerze. Trudność polega na tym, że schemat bazy bywa często większy niż to, co model jest w stanie przyjąć w kontekście. Schematy hurtowni danych liczą setki lub tysiące tabel powiązanych kluczami obcymi; schemat syntetyczny badany w tej pracy ma 1000 tabel i 1174 klucze obce. W takich warunkach pojawiają się cztery problemy:

1. **Rozmiar kontekstu.** Pełny opis schematu w postaci DDL zajmuje 263 113 tokenów. Nie mieści się to w oknie kontekstowym modeli uruchamianych lokalnie, a w modelach z większym oknem wyraźnie wydłuża czas odpowiedzi (podrozdziały 2.4.1 i 6.2.6).

2. **Pomijanie tabel pośrednich.** Wyszukiwanie oparte wyłącznie na podobieństwie wektorowym [29] nie bierze pod uwagę powiązań między tabelami. Może więc pominąć tabelę łączącą relację $\mathrm{N} : \mathrm{M}$, a model, który dostanie niepełny fragment schematu, zgaduje warunki złączeń.
3. **Koszt przesyłania danych binarnych.** Dane binarne w wiadomości JSON-RPC [7] trzeba zakodować w Base64, co zwiększa ich objętość o jedną trzecią i podnosi liczbę tokenów (podrozdział 3.1.1).
4. **Bezpieczeństwo.** Agent z dostępem do bazy może zostać zmanipulowany treścią zapisaną w samych danych (pośrednie wstrzykiwanie instrukcji [17, 41], problem zdezorientowanego zastępcy [18]). Zabezpieczenia oparte wyłącznie na instrukcjach w prompcie systemowym są zawodne [43], dlatego potrzebna jest deterministyczna kontrola zapytań, np. przez analizę ich drzewa składniowego (AST).

1.3 Cel pracy

Celem pracy jest analiza i ewaluacja wydajności, skalowalności oraz bezpieczeństwa integracji systemów agentowych opartych na dużych modelach językowych ze złożonymi relacyjnymi bazami danych za pośrednictwem protokołu Model Context Protocol.

Cel główny rozpisano na następujące cele szczegółowe:

1. analiza porównawcza protokołu MCP i formatu JSON-RPC 2.0 z innymi sposobami komunikacji (REST, gRPC) pod kątem kosztu przesyłania danych oraz zagrożeń bezpieczeństwa;
2. ocena, jak uwzględnienie grafu kluczy obcych przy doborze tabel wpływa na kompletność i objętość fragmentu schematu przekazywanego modelowi, a także na poprawność generowanych przez model zapytań SQL;
3. wyznaczenie, analitycznie oraz pomiarowo, rozmiaru danych, od którego przesyłanie ich poza wiadomością JSON-RPC jest szybsze niż kodowanie w Base64;
4. ocena wpływu przeniesienia walidacji zapytań SQL do puli procesów na responsywność serwera obsługującego wielu agentów jednocześnie;
5. ocena skuteczności walidacji zapytań oraz oczyszczania zwracanych rekordów w ochronie przed atakami typu SQL Injection i wstrzykiwaniem instrukcji;
6. porównanie skalowalności paginacji kursorowej i paginacji OFFSET/LIMIT przy przeglądaniu dużych zbiorów wyników.

1.4 Zakres pracy

Przedmiotem pracy jest integracja agentów opartych na modelach językowych z relacyjną bazą danych PostgreSQL o złożonym schemacie za pośrednictwem protokołu MCP. Obiektem badań jest

zaprojektowana i zaimplementowana na potrzeby pracy brama pośrednicząca, czyli serwer MCP, który udostępnia agentom dostęp do bazy.

Zakres pracy obejmuje:

- przegląd literatury oraz analizę porównawczą protokołów komunikacyjnych stosowanych w systemach agentowych;
- projekt i implementację bramy MCP współpracującej z PostgreSQL i Apache Kafka, w tym modułu doboru tabel łączącego wyszukiwanie leksykalne i wektorowe z rozszerzaniem wyniku o tabele pośrednie, paginacji kursorowej z kontrolą liczby tokenów, kanału przesyłania danych binarnych poza JSON-RPC oraz zapory zapytań SQL opartej na analizie drzewa składniowego i uruchamianej w puli procesów;
- przygotowanie środowiska badawczego: syntetycznej bazy danych (1000 tabel, 1174 klucze obce, tabela z 1 mln rekordów), zbioru 500 pytań testowych oraz zautomatyzowanej aparatury pomiarowej;
- eksperymentalną ocenę poszczególnych mechanizmów z użyciem dwóch modeli językowych oraz statystyczną weryfikację postawionych hipotez.

W pracy nie uwzględniono:

- operacji modyfikujących dane ani transakcji, ponieważ brama udostępnia wyłącznie odczyt;
- pomiarów pełnej komunikacji MCP przez warstwę transportową (`stdio`, HTTP), ponieważ mechanizmy bramy mierzono, wywołując je bezpośrednio;
- pomiarów w rzeczywistej sieci lokalnej i rozległej, dla których progi opłacalności wyznaczono z modelu analitycznego.

Praca nie obejmuje także dostrajania modeli językowych, ewaluacji modeli komercyjnych dostępnych przez API ani wdrożenia rozproszonego na wielu serwerach.

1.5 Pytania badawcze

Na podstawie opisanych problemów sformułowano trzy pytania badawcze:

PB1 *W jakim stopniu uzupełnienie wyszukiwania semantycznego o ekspansję grafową o tabele mostkujące (ang.* FK bridge expansion*) poprawia kompletność doboru tabel w złożonych schematach relacyjnych?*

PB2 *Przy jakiej wielkości przesyłanych danych narzut kodowania Base64 w protokole JSON-RPC uzasadnia przeniesienie transferu do osobnego kanału (Out-of-Band)?*

PB3 *W jakim stopniu przeniesienie walidacji składniowej SQL do procesów potomnych pozwala utrzymać wydajność serwera asynchronicznego pod dużym obciążeniem przy zachowaniu ochrony przed badanymi klasami ataków?*

1.6 Hipotezy badawcze

Hipoteza główna pracy brzmi:

> *Zastosowanie dedykowanej bramy pośredniczącej w standardzie MCP, integrującej semantyczno-grafową selekcję schematu, adaptacyjną paginację kursorową oraz wieloprocesową walidację zapytań, pozwala ograniczyć objętość kontekstu i koszt przekazania danych oraz chronić bazę danych przed badanymi klasami ataków bez degradacji responsywności pętli zdarzeń bramy pod obciążeniem.*

Do weryfikacji statystycznej rozbito ją na hipotezy szczegółowe:

H1: Uzupełnienie hybrydowego wyszukiwania schematu o ekspansję grafową o tabele mostkujące zapewnia statystycznie istotny wzrost czułości selekcji tabel (*Table-Recall*) w relacji do hybrydowego wyszukiwania RRF bez ekspansji oraz leksykalnego BM25 bez ekspansji przy równym budżecie tabel, tj. gdy warianty bazowe otrzymują maksymalną liczbę tabel zwracaną przez wariant autorski (Graph@5 wobec RRF@8 i BM25@8; test rangowanych znaków Wilcoxona dla prób zależnych z korektą Holma). Porównanie z gęstym k-NN ma charakter opisowy.

H1b: Analogiczny, statystycznie istotny wzrost dotyczy czułości selekcji kolumn (*Column-Recall*) w relacji do RRF@8 i BM25@8 (test rangowanych znaków Wilcoxona dla prób zależnych). Ponieważ w przyjętej metodyce kolumny wynikają automatycznie z wybranych tabel, H1b jest pochodną H1, a nie niezależną hipotezą.

H2: Istnieje wyznaczalny empirycznie próg wolumenu danych S^*, powyżej którego transfer binarny w kanale pozapasmowym cechuje się statystycznie istotnie niższym opóźnieniem niż standardowa serializacja JSON-RPC z kodowaniem Base64, a poniżej którego przewagę ma serializacja Base64 (test Manna-Whitneya dla rozmiaru ładunku poniżej i powyżej progu).

H3: Oddelegowanie walidacji drzew składniowych AST do wieloprocesowej puli wykonawczej zapewnia statystycznie istotnie niższe opóźnienie asynchronicznej pętli zdarzeń pod obciążeniem wieloagentowym niż walidacja w wątku pętli zdarzeń (test Manna-Whitneya dla $\mathrm{M} = 100$ i $\mathrm{M} = 500$ klientów).

1.7 Metodyka badań

Pracę prowadzono zgodnie z metodyką badań konstrukcyjnych (*Design Science Research*) [44]: najpierw zbudowano artefakt, czyli bramę MCP, a następnie oceniono go eksperymentalnie. Wszystkie pomiary wykonano w jednej, w pełni zautomatyzowanej rundzie na danych syntetycznych. Do wnioskowania użyto testów nieparametrycznych (Wilcoxona, McNemara, Manna-Whitneya) oraz przedziałów ufności wyznaczanych metodą bootstrap i metodą Cloppera-Pearsona.

1.8 Wykorzystane narzędzia i środowisko

Narzędzia

Podczas realizacji pracy wykorzystano:

- do budowy bramy: język Python 3.12, framework FastMCP [33] i SDK protokołu MCP [2], sterownik `asyncpg` [34] oraz biblioteki `aiokafka`, `pydantic` i `httpx`;
- jako warstwę danych: PostgreSQL z rozszerzeniem `pgvector` [22] (indeks HNSW [35]) oraz Apache Kafka [26];
- do wyboru tabel i kontroli zapytań: `sentence-transformers` z modelem `all-MiniLM-L6-v2` [49], `rank_bm25`, NetworkX oraz parser SQL `sqlglot` [36];
- jako modele językowe: `qwen2.5-coder:3b` i `qwen2.5-coder:7b` [48] uruchamiane w serwerze Ollama, a do liczenia tokenów bibliotekę `tiktoken` [40];
- do pomiarów i analiz: własną aparaturę pomiarową (podrozdział 5.7), bibliotekę `psutil`, a do obliczeń statystycznych i wykresów biblioteki SciPy, pandas, PyArrow, matplotlib i seaborn;
- do pracy nad kodem i tekstem: `pytest`, `uv`, Git, Docker z Docker Compose, Visual Studio Code oraz LaTeX z BibLaTeX.

Środowisko deweloperskie

Kod powstawał na komputerze z procesorem Intel Core i7-1165G7 (4 rdzenie, 8 wątków, 2,8 GHz) i 16 GB RAM, z systemem Ubuntu 26.04 LTS. Python 3.12 działał w środowisku wirtualnym `.venv` zarządzanym przez `uv`. Zależności opisano w pliku `pyproject.toml`, a ich dokładne wersje zapisano w `uv.lock`.

Bazę danych i brokera uruchamiano lokalnie w kontenerach Docker opisanych w pliku `testbed/docker-compose.yml` (tabela 5):

- `mcp_postgres` – obraz `pgvector/pgvector:0.7.4-pg16` z ustawieniami `max_connections=200`, `shared_buffers=1GB` i `work_mem=32MB` oraz trwałym wolumenem `pgdata`; przy pierwszym uruchomieniu skrypty z katalogu `testbed/init-scripts/` tworzą schemat 1000 tabel z 1174 kluczami obcymi, tabele z opisami i wektorami tabel oraz tabelę z 1 mln rekordów;
- `mcp_kafka` – obraz `apache/kafka:3.7.0` z jednym brokerem w trybie KRaft i tematem o trzech partycjach.

Poprawność kodu sprawdzano testami `pytest`, które obejmują m.in. zaporę SQL, wybór tabel, paginację, przesyłanie danych binarnych i samą aparaturę pomiarową. Przed właściwą rundą pomiarową uruchamiano skrócony przebieg wszystkich scenariuszy (`./run_benchmark.sh --quick`) oraz skrypt `scripts/preflight_check.py`, sprawdzający, czy usługi i dane są gotowe. Pełne eksperymenty z modelami językowymi prowadzono tylko w środowisku testowym.

Środowisko testowe

Rundę pomiarową wykonano na wynajętej instancji RunPod z kartą NVIDIA A100-SXM4-80GB (80 GiB), procesorem AMD EPYC 7742 64-Core Processor (16 vCPU, 250 GB RAM) i systemem Ubuntu 22.04 z CUDA. Skrypt `scripts/setup_runpod.sh` instaluje tam bez kontenerów PostgreSQL 16.4 z pgvector 0.7.4, Apache Kafka 3.7.0, Ollamę 0.35.0 i Pythona 3.12.3, a następnie wypełnia bazę tym samym generatorem co lokalnie. Pełne parametry podano w tabeli 6, a przebieg rundy opisano w podrozdziałach 5.7 i 6.1.2.

1.9 Osiągnięte rezultaty

Dzięki ekspansji grafowej do wyniku trafiało 55,3% potrzebnych tabel (Table-Recall), wobec 47,0% dla RRF@8 i 53,2% dla BM25@8 przy tej samej liczbie tabel. Przekazywany modelowi fragment schematu był przy tym ponad 99% mniejszy od pełnego opisu bazy. Nie przełożyło się to jednak na istotnie lepszą poprawność generowanych zapytań.

Przesyłanie danych przez pamięć współdzieloną okazało się na jednym serwerze szybsze od Base64 dla danych większych niż 12,9 KiB, zgodnie z modelem analitycznym. Paginacja kursorowa miała stały czas odpowiedzi (0,43 ms przy przesunięciu 950 000 wierszy, wobec 357 ms dla OFFSET/LIMIT). Przeniesienie walidacji do puli procesów zmniejszyło medianę opóźnienia pętli zdarzeń przy 100 klientach z 975 do 0,40 ms. Zapora zablokowała wszystkie 24 próby SQL Injection bez fałszywych alarmów, ale przepuściła zapytania `SELECT` wywołujące funkcje z efektami ubocznymi.

Hipotezy H1, H2 (dla jednego serwera) i H3 zostały potwierdzone, H1b częściowo, a hipotezę główną uznano za potwierdzoną w badanym zakresie.

1.10 Główne trudności

Najwięcej trudności sprawiły:

- **brak gotowych danych** – publiczne zbiory Text-to-SQL nie zawierają schematów z tysiącem tabel, więc trzeba było wydzielić własny generator schematu i przygotować własny zbiór pytań;
- **koszt i powtarzalność eksperymentów z modelami** – do uruchomienia modeli potrzebna była karta graficzna, a więc wynajęta instancja w chmurze; aby wyniki dało się powtórzyć, ustawiono temperaturę $\mathrm{T} = 0$ i stałe ziarna losowania, a każdą rundę opisano manifestem;
- **ograniczenia instancji chmurowej** – nie dało się na niej emulować opóźnień sieci (`tc netem`), dlatego progi dla sieci lokalnej i rozległej wyznaczono z modelu;
- **wpływ pomiaru na wynik** – odczyt zużycia pamięci w trakcie testu sam wydłużał najdłuższe opóźnienia, co trzeba było wydzielić w dodatkowej analizie.

1.11 Możliwości wykorzystania wyników

Wyniki mogą przydać się zespołom, które budują agentów z dostępem do firmowych baz danych. Próg S^* pomaga zdecydować, od jakiego rozmiaru danych warto przesyłać je poza JSON-RPC. Ekspansję grafową można dodać jako etap wyboru tabel w istniejących systemach Text-to-SQL, a paginację kursorową z kontrolą liczby tokenów zastosować w narzędziach MCP zwracających duże wyniki. Przeniesienie kosztownych obliczeń do puli procesów może znaleźć zastosowanie w serwerach opartych na `asyncio`, nie tylko w serwerach MCP. Z kolei wyniki testów bezpieczeństwa pokazują, że sama analiza AST nie wystarcza i trzeba ją łączyć z ograniczeniem uprawnień w bazie.

Generator schematu i aparatura pomiarowa pozwalają powtórzyć eksperymenty i porównać z nimi inne metody wyboru tabel.

1.12 Wykorzystanie narzędzi i metod sztucznej inteligencji

Podczas pisania pracy korzystano z narzędzi opartych na dużych modelach językowych do poprawek stylistycznych i redakcyjnych tekstu: ujednolicenia terminologii, poprawy składni i interpunkcji oraz skracania zbyt rozbudowanych zdań. Wszystkie proponowane zmiany autor sprawdził i ponosi pełną odpowiedzialność za treść pracy. Modele `qwen2.5-coder` opisane w rozdziale 6 były natomiast przedmiotem badań, a nie narzędziem redakcyjnym.

Przykładowe prompty użyte w trakcie tworzenia pracy udostępniono w repozytorium z kodem źródłowym pracy (załącznik 1):

`https://github.com/karol-pranko/mcp-thesis`

1.13 Układ pracy

Praca składa się z siedmiu rozdziałów:

- **Rozdział 1 – Wstęp:** tło i problem badawczy, cel i zakres pracy, pytania i hipotezy, metodyka oraz narzędzia i środowisko.
- **Rozdział 2 – Teoretyczne podstawy integracji modeli językowych z danymi:** sposoby łączenia modeli językowych z otoczeniem, specyfikacja MCP i ograniczenia okna kontekstowego.
- **Rozdział 3 – Analiza porównawcza protokołów komunikacyjnych:** porównanie JSON-RPC 2.0, REST, gRPC i MCP oraz przegląd zagrożeń bezpieczeństwa w systemach agentowych.
- **Rozdział 4 – Projekt systemu integracji MCP z hurtownią danych:** projekt bramy, czyli wybór tabel z ekspansją grafową, paginacja kursorowa, model progu S^* i ochrona pętli zdarzeń.
- **Rozdział 5 – Implementacja i środowisko badawcze:** implementacja modułów bramy, środowisko badawcze i aparatura pomiarowa.
- **Rozdział 6 – Wyniki badań i ewaluacja eksperymentalna:** wyniki eksperymentów i statystyczna weryfikacja hipotez.

- **Rozdział 7 – Podsumowanie i wnioski:** podsumowanie wyników, ograniczenia rozwiązania i kierunki dalszych prac.

Pracę uzupełniają załączniki z opisem repozytorium kodu źródłowego i wybranymi listingami.

Rozdział 2

Teoretyczne podstawy integracji modeli językowych z danymi

Generatywne modele językowe oparte na architekturze Transformer [54] generują tokeny autoregresyjnie, a wiedza zakodowana w ich wagach odzwierciedla jedynie stan korpusu treningowego; stochastyczne próbkowanie sprzyja ponadto halucynacjom [37], co w zastosowaniach wymagających determinizmu (relacyjne bazy danych, transakcje finansowe) czyni model odizolowany od zewnętrznych źródeł prawdy niewiarygodnym. Konieczność determinizmu wymusiła ewolucję paradygmatów integracji: od modyfikacji wag (*fine-tuning*), przez generowanie wspomagane wyszukiwaniem [29], po korzystanie z narzędzi [50], które doprowadziło do powstania standardu MCP [2].

2.1 Ewolucja paradygmatów integracji: od odizolowanych LLM do Tool Use

2.1.1 Od pamięci parametrycznej do paradygmatu Tool Use

Pasywna pamięć parametryczna modeli LLM (wagi sieci zapisane w procesie pre-treningu) odzwierciedla stan korpusu z chwili treningu i nie zapewnia deterministycznego dostępu do danych, co wyklucza bezpośrednie wnioskowanie na transakcyjnych danych korporacyjnych. Historyczne podejścia adaptacji – dostrajanie wag obarczone zjawiskiem katastrofalnego zapominania [23] oraz generowanie wspomagane wyszukiwaniem (RAG) [29] – zawodzą w kontakcie z relacyjnymi składami danych. RAG spłaszcza relacje logiczne bazy do postaci bezpostaciowego tekstu, a wyszukiwanie gęste po wektorach nie interpretuje predykatów selekcji ani złączeń algebraicznych. Szczegółowe zestawienie kompromisów architektonicznych tych paradygmatów w zestawieniu z paradygmatem korzystania z narzędzi [47] przedstawiono w tabeli 1.

Przełom przyniósł paradygmat korzystania z narzędzi: model może wywołać deterministyczną procedurę API, odebrać wynik i wykorzystać go w dalszym wnioskowaniu [50]. Wzorzec *ReAct* [60] porządkuje tę pętlę jako sekwencję decyzji, akcji i obserwacji. Rozdzielenie generowania od wykonania narzędzia stało się bezpośrednią przesłanką standaryzacji MCP.

Tabela 1. Syntetyczne porównanie paradygmatów integracji modeli językowych z zasobami danych

Paradygmat	**Determinizm danych**	**Świeżość informacji**	**Koszt aktualizacji bazy**	**Narzut na okno kontekstowe**	**Zdolność do efektów ubocznych**
Fine-tuning	Niski (aproksymacja probabilistyczna)	Bardzo niska (wymaga cyklu trenowania)	Bardzo wysoki (wymaga klastra GPU)	Brak narzutu ($\mathcal{O}(1)$)	Brak (model pasywny)
RAG (indeksy wektorowe)	Średni (wrażliwy na chunking i k-NN)	Umiarkowana (opóźnienia potoków ETL)	Umiarkowany (rekalkulacja wektorów)	Średni ($\mathcal{O}(\mathrm{k} \cdot \mathrm{L}_{\mathrm{chunk}})$)	Brak (tryb read-only)
Tool Use (np. bazy SQL)	Wysoki (determinizm silnika bazy danych)	Czas rzeczywisty (stan transakcyjny)	Minimalny (standardowy zapis w bazie)	Zmienny ($\mathcal{O}(\mathrm{L}_{\mathrm{schema}} + \mathrm{L}_{\mathrm{result}})$)	Pełna (kontrolowane zmiany stanu)

2.1.2 Redukcja przestrzeni relacyjnej i ograniczenia metod promptingowych

Gdy model integruje się z korporacyjną hurtownią danych o złożonym schemacie ($\mathrm{N_T} \geq 500$ tabel), wstrzyknięcie pełnego zrzutu DDL prowadzi do wyczerpania budżetu okna kontekstowego oraz spadku trafności generowanych zapytań SQL. W literaturze rozwinięto metody redukcji przestrzeni relacyjnej i łączenia schematu (ang. *schema linking*), w tym dekompozycję szkieletową zapytań RESDSQL [30], halucynacyjne przeszukiwanie struktur relacyjnych CRUSH4SQL [25], prompting kognitywny (DIN-SQL [46], MAC-SQL [55]), dobór demonstracji few-shot (DAIL-SQL [13]) oraz zewnętrzne filtry heurystyczne (CodeS [31], CHESS [51]). Wyzwania integracji modeli językowych z realistycznymi, rozbudowanymi bazami danych przedsiębiorstw formalizuje również benchmark Spider 2.0 [27].

Teoretyczna luka metod promptingowych i gęstego RAG: problem grafów o dużej średnicy. Metody promptingowe oraz gęste wyszukiwanie wektorowe (k-NN RAG) mogą zawodzić w obecności relacji o dużej odległości w grafie schematu ($\mathrm{dist}(\mathrm{u}, \mathrm{v}) \geq 2$). Gdy zapytanie analityczne wymaga powiązania dwóch encji skrajnych, a ścieżka w schemacie relacyjnym prowadzi przez tabele pośredniczące modelujące relacje $\mathrm{N} : \mathrm{M}$ (np. tabele asocjacyjne pozycji zamówień, partii magazynowych czy alokacji), nazwy tych tabel pośrednich często nie występują w pytaniu użytkownika.

W rezultacie wyszukiwanie oparte na podobieństwie do pytania może zwracać tabele krańcowe bez tabel mostkujących. Model LLM, pozbawiony definicji kluczy obcych łączących te relacje, może wtedy halucynować nieistniejące powiązania bezpośrednie albo generować iloczyn kartezjański. Ta obserwacja motywuje zaprojektowaną w niniejszej pracy ekspansję grafową o tabele mostkujące (ang. *FK bridge expansion*; w kodzie: `discover_schema_with_graph_pruning`), sformalizowaną w Rozdziale 4.

2.2 Geneza standaryzacji interfejsów komunikacyjnych w AI

2.2.1 Fragmentacja ekosystemu i redukcja złożoności integracyjnej

Rozwój modeli językowych i wczesnych interfejsów funkcyjnych doprowadził do powstania długu technologicznego kodu łączącego (ang. *glue code*). Brak otwartego standardu wymuszał dedykowane adaptery dla każdego środowiska, w tym osobne kontrakty błędów, autoryzacji i transmisji.

Analogicznie do protokołu *Language Server Protocol* [38], standaryzacja interfejsów w systemach agentowych redukuje liczbę wymaganych integracji między M środowiskami modeli a N źródłami danych i narzędzi z $\mathcal{O}(\mathrm{M} \times \mathrm{N})$ do $\mathcal{O}(\mathrm{M} + \mathrm{N})$. Analogia ta ma jednak wyłącznie charakter architektoniczny: standaryzacja obejmuje ramkę JSON-RPC 2.0 i metody odkrywania oraz wywołania narzędzi, nie eliminuje natomiast różnorodności kontraktów narzędziowych, które model musi interpretować w każdej sesji.

Praktyczną implementacją opisanego paradygmatu stał się otwarty standard *Model Context Protocol* [2], którego szczegółowa specyfikacja techniczna, prymitywy semantyczne oraz warstwy transportowe zostały poddane analizie w podrozdziale 2.3.

2.3 Formalna specyfikacja Model Context Protocol (MCP)

2.3.1 Role architektoniczne w protokole

Specyfikacja protokołu MCP [2] definiuje ścisły podział odpowiedzialności pomiędzy trzema głównymi węzłami logicznymi:

1. **Host (Aplikacja orkiestrująca):** Główny proces nadrzędny, w ramach którego realizowane jest zadanie użytkownika. Host zarządza interfejsem użytkownika, integruje model językowy, koordynuje procesy decyzyjne oraz sprawuje bezpośrednią kontrolę nad politykami bezpieczeństwa i zgodami użytkownika na wykonanie akcji. Przykłady Hostów to m.in. środowisko IDE (np. JetBrains, Visual Studio Code) lub dedykowany orkiestrator mikrousług;

2. **Klient MCP:** Biblioteczny moduł komunikacyjny osadzony wewnątrz architektury Hosta. Klient utrzymuje ścisłe połączenie 1:1 z danym serwerem MCP. Odpowiada za translację wewnętrznych struktur Hosta na komunikaty protokołu, zarządzanie warstwą transportową, serializację i deserializację pakietów JSON-RPC oraz obsługę błędów protokołu;

3. **Serwer MCP:** Niezależny, wyspecjalizowany komponent programistyczny (proces lokalny lub serwis sieciowy), hermetyzujący dostęp do określonej domeny danych lub zestawu operacji. Serwer publikuje swoje możliwości, udostępniając zasoby pasywne, aktywne narzędzia wykonawcze oraz szablony promptów.

Relacja między Hostem, Klientem i Serwerem ma charakter modularny. Pojedynczy Host może powołać do życia wiele niezależnych instancji Klientów MCP, z których każdy komunikuje się

z odrębnym Serwerem MCP (np. Serwer bazy PostgreSQL, Serwer brokera Kafka, Serwer systemu plików), tworząc spójny graf kontekstowy dostępny dla modelu językowego.

2.3.2 Cykl życia sesji, transport i prymitywy semantyczne

Opis dotyczy rewizji specyfikacji 2025-06-18 [39]; biblioteka `mcp` użyta w implementacji (wersja 2.2.0) obsługuje także nowsze rewizje, których nowe elementy nie są w pracy wykorzystywane. Zgodnie ze specyfikacją sesja Klient–Serwer przebiega w trzech fazach: inicjalizacji (obejmującej negocjację wersji i zdolności), operacji oraz zamknięcia, a logika komunikatów JSON-RPC jest oddzielona od medium transmisyjnego. Zestawienie przedstawiono w tabeli 2.

Tabela 2. Cykl życia sesji, warstwy transportowe i prymitywy semantyczne protokołu MCP (rewizja 2025-06-18)

Element	**Krótki opis**
Faza 1: inicjalizacja	Klient wysyła żądanie `initialize` z wersją protokołu i deklaracją zdolności (np. `roots`, `sampling`, `elicitation`); Serwer odpowiada wybraną wersją i własnymi zdolnościami (`tools`, `resources`, `prompts`, `logging`); Klient potwierdza powiadomieniem `notifications/initialized`. Brak wspólnej wersji kończy sesję.
Faza 2: operacja	Wymiana zgodna z wynegocjowanymi zdolnościami: listowanie prymitywów, `tools/call`, `resources/read`, powiadomienia oraz `ping`.
Faza 3: zamknięcie	Zakończenie połączenia przez warstwę transportową (dla `stdio`: zamknięcie strumienia wejściowego procesu serwera).
Transport `stdio`	Serwer jako proces potomny Hosta; komunikaty JSON-RPC w pojedynczych liniach przez `stdin`/`stdout`, `stderr` dla logów. Brak portów sieciowych.
Transport Streamable HTTP	Serwer jako niezależna usługa z jednym punktem końcowym: Klient wysyła komunikaty żądaniami HTTP POST, a Serwer odpowiada pojedynczą odpowiedzią JSON lub strumieniem Server-Sent Events [19]. Zastąpił on transport HTTP+SSE w rewizji 2025-03-26. Dla transportów HTTP specyfikacja definiuje opcjonalną autoryzację opartą na OAuth 2.1.
Zasoby (*Resources*)	Dane kontekstowe tylko do odczytu, identyfikowane przez URI (także szablonami parametryzowanymi).
Narzędzia (*Tools*)	Procedury o nazwie, opisie i parametrach opisanych w JSON Schema [59]; wynik zawiera listę bloków `content`, opcjonalnie ustrukturyzowany wynik `structuredContent` zgodny z deklarowanym `outputSchema`, a błąd wykonania narzędzia jest sygnalizowany polem `isError` w wyniku.
Prompty (*Prompts*)	Udostępniane przez Serwer szablony instrukcji i konwersacji.
Elicitacja (*Elicitation*)	Zdolność Klienta: Serwer może w trakcie operacji poprosić użytkownika o dodatkowe dane w ustrukturyzowanej postaci.

Implementacja referencyjna (Rozdział 5) uruchamia serwer FastMCP w transporcie `stdio` (`mcp.run(transport="stdio")`); transport Streamable HTTP nie był w pracy używany.

2.4 Specyfika systemów wielkoskalowych w interakcji z agentami

2.4.1 Fizyczne i logiczne bariery mechanizmu uwagi

Powszechne w przemyśle przekonanie, że ekspansja nominalnego okna kontekstowego modeli LLM (od 128k tokenów w rodzinie Llama 3.1 do miliona i więcej tokenów w architekturach komercyjnych) eliminuje konieczność selekcji schematów i paginacji danych, nie uwzględnia kosztów serwowania modeli ani wyników badań empirycznych nad mechanizmem samo-uwagi.

Koszt obliczeniowy samej operacji uwagi w fazie wstępnego przetwarzania promptu rośnie kwadratowo z długością sekwencji L, $\mathcal{O}(\mathrm{L}^2)$, natomiast koszt warstw liniowych (projekcje i sieci MLP) rośnie liniowo. Dla modeli rzędu kilku miliardów parametrów i kontekstów rzędu dziesiątek tysięcy tokenów składnik liniowy pozostaje dominujący; zgodnie z tym czas do pierwszego tokenu zmierzony w niniejszej pracy rósł w przybliżeniu liniowo w badanym zakresie do ok. 28 tys. tokenów (podrozdział 6.2.6). Ponadto utrzymanie długiego kontekstu wymaga alokacji bufora kluczy i wartości (*KV-Cache*), którego rozmiar w pamięci akceleratora skaluje się liniowo z L:

$$\mathrm{M}_{\mathrm{KV}} = 2 \cdot \mathrm{b}_{\mathrm{elem}} \cdot \mathrm{l}_{\mathrm{layers}} \cdot \mathrm{h}_{\mathrm{KV}} \cdot \mathrm{d}_{\mathrm{head}} \cdot \mathrm{L}, \tag{1}$$

gdzie $\mathrm{b}_{\mathrm{elem}} = 2$ B oznacza rozmiar słowa w precyzji FP16/BF16, mnożnik 2 wynika z przechowywania tensorów kluczy i wartości, $\mathrm{l}_{\mathrm{layers}}$ to liczba warstw, h_{KV} – liczba głów kluczy i wartości, a $\mathrm{d}_{\mathrm{head}}$ – wymiar pojedynczej głowy. W modelach z grupowaną uwagą zapytań (*Grouped-Query Attention*, GQA) [1] h_{KV} jest mniejsze od liczby głów zapytań. Dla modelu Qwen2.5-7B (28 warstw, 4 głowy KV, $\mathrm{d}_{\mathrm{head}} = 128$) [48] i sesji o długości $\mathrm{L} = 128\mathrm{k} = 131\,072$ tokenów wzór (1) daje $2 \cdot 2 \cdot 28 \cdot 4 \cdot 128 \cdot 131\,072$ B $= 7{,}0$ GiB na jedną sesję. Dla modelu tej samej skali z pełną uwagą wielogłowową (MHA), np. Llama 2 7B (32 warstwy, 32 głowy KV, $\mathrm{d}_{\mathrm{head}} = 128$) [53], ta sama długość kontekstu wymaga 64 GiB, czyli niemal całej pamięci jednego akceleratora klasy A100 80 GB.

Niezależnie od barier sprzętowych, badania nad dynamiką uwagi w długich sekwencjach wskazują na pogorszenie jakości wnioskowania:

- **Zapaść uwagi w środku sekwencji:** Liu i in. [32] wykazali, że zdolność modeli autoregresyjnych do ekstrakcji informacji ulega gwałtownej degradacji w centralnych segmentach kontekstu (zjawisko *Lost in the Middle*, charakterystyka w kształcie litery U), przez co definicje tabel umieszczone w środku obszernego zrzutu DDL są pomijane;
- **Wpływ długości wejścia:** Levy i in. [28] wykazali na zadaniach rozumowania (bez związku z SQL), że dopełnienie wejścia nieistotnym tekstem obniża trafność odpowiedzi modeli już przy długościach wejścia znacznie mniejszych od nominalnego okna kontekstowego. Można oczekiwać podobnego efektu przy wstrzykiwaniu obszernego, w większości nieistotnego zrzutu DDL, choć cytowane badanie nie dotyczyło generowania SQL.

Uwarunkowania te wskazują, że integracja z dużymi składami danych wymaga deterministycznych technik kompresji schematu oraz adaptacyjnej paginacji kursorowej bezpośrednio w warstwie pośredniczącej MCP.

2.4.2 Transakcyjność i modele spójności w komunikacji bezstanowej

Bezstanowe wywołania MCP i wielosekundowa inferencja czynią długie transakcje kosztownymi: przesuwają horyzont $\mathrm{x}_{\mathrm{min}}$ MVCC i zajmują połączenia z puli. Dlatego kolejne strony są domyślnie odczytywane w odrębnych migawkach, a spójność całego przeglądania zapewnia opcjonalna, krótkotrwała migawka opisana w Rozdziale 4.

2.4.3 Stochastyczne transakcje agentowe i paradygmat SAGA

2PC/XA wymagałoby utrzymywania blokad przez cały czas inferencji, a SAGA dodatkowo wymagałaby wiarygodnych akcji kompensujących [14, 16]. Ponieważ praca nie wykonuje modyfikacji danych, oba modele pozostają poza zakresem: bezpieczeństwo zapewnia niezmiennik czystego odczytu `SELECT`, a spójność przeglądania – migawka MVCC (Rozdział 4).

Rozdział 3

Analiza porównawcza protokołów komunikacyjnych

Integracja modeli językowych ze złożonymi składami danych wymaga zdefiniowania warstwy komunikacyjnej, a dobór protokołu wyznacza kompromis między przepustowością binarną (np. gRPC), narzutem serializacji tekstowej i re-autoryzacji (np. REST), bezpieczeństwem perymetru bazy danych oraz transparentnością śladu wnioskowania. Rozdział porównuje protokoły stosowane w systemach agentowych i uzasadnia wybór formatu JSON-RPC 2.0 w standardzie MCP [2].

3.1 Protokół JSON-RPC 2.0 jako rdzeń Model Context Protocol

Standard JSON-RPC 2.0 [21], oparty na formacie JSON (RFC 8259 [7]), jest agnostycznym transportowo protokołem zdalnego wywoływania procedur o trzech typach komunikatów: żądaniu (*Request*, z polami `jsonrpc`, `method`, `params`, `id`), odpowiedzi (*Response*, z obiektem `result` lub `error`) oraz powiadomieniu (*Notification*), czyli żądaniu bez pola `id`, na które odbiorca nie odpowiada. W MCP powiadomienia służą do sygnalizacji cyklu życia sesji i asynchronicznego Server Push.

3.1.1 Narzut serializacji tekstowej UTF-8 i tokenizacja danych Base64

Format JSON kodowany w UTF-8 zapewnia transparentność inspekcji, lecz w środowiskach relacyjnych rodzi istotny narzut transmisyjny wynikający z: (1) redundancji etykiet schematu poprzez powielanie tekstowych nazw kolumn w każdej krotce zbioru analitycznego; (2) ucieczki znaków wymagającej prefiksowania znaków specjalnych; oraz (3) kosztu tokenizacji danych binarnych zakodowanych w Base64. Brak typu binarnego w JSON wymusza rzutowanie danych binarnych na ciągi Base64 (narzut 33,3% bajtów). Ciągi Base64 mają wysoką entropię i nie zawierają częstych podsłów, więc tokenizatory podsłów (ang. *Byte-Pair Encoding*, BPE) [40] dzielą je na krótkie fragmenty. W analizie wtórnej (niezależnej od sprzętu) enkoder `cl100k_base` zakodował losowe dane binarne o rozmiarach od 1 KiB do 1 MiB po przekształceniu do Base64 na

0,94–0,97 tokenu na bajt danych binarnych (ok. 0,71 tokenu na znak tekstu Base64; skrypt `benchmarks/analysis_offline/o5_tokenizacja_base64.py`). Przekazanie 1 MiB danych binarnych w kontekście modelu kosztowałoby więc ok. 1 002 242 tokenów.

Przykład zwartej ramki wywołania narzędzia oraz odpowiedzi serwera MCP ze stronicowaniem kursorowym zamieszczono na listingu 1 w załączniku 2.

Most logiczny do badań własnych. Koszt tokenowy wskazuje, że dane binarne i masowe wyniki nie powinny trafiać do okna kontekstu modelu. Zaprojektowany w pracy transfer pozapasmowy (*Out-of-Band*, Rozdział 4) przekazuje je poza ramką JSON-RPC, a model otrzymuje jedynie deskryptor. Zmierzona korzyść tego mechanizmu dotyczy jednak hosta, a nie tokenów: krótszego czasu przekazania i mniejszego kosztu kodowania. Próg opłacalności transferu przez pamięć współdzieloną zmierzony na jednym węźle wynosi ok. 12,9 KiB (podrozdział 6.3).

3.2 Taksonomia architektur komunikacyjnych w systemach agentowych

3.2.1 Charakterystyka porównywanych paradygmatów architektonicznych

Ocena właściwości protokołu Model Context Protocol wymaga osadzenia go w kontekście wiodących standardów wymiany danych w systemach rozproszonych:

1. **Architektura REST (Fielding 2000 [12]):** zorientowana na reprezentację zasobów identyfikowanych za pomocą jednolitych adresów URI oraz manipulację stanem poprzez standardowe czasowniki HTTP; zakłada całkowity brak pamięci sesyjnej po stronie serwera pomiędzy kolejnymi żądaniami;

2. **Standard gRPC (Google [52]):** wysokowydajny, binarny framework RPC oparty na transportowym standardzie HTTP/2 [6] oraz mechanizmie serializacji Protocol Buffers [15], wykorzystujący statyczne deskryptory interfejsów (pliki `.proto`) z numerycznym znakowaniem pól w reżimie TLV;

3. **Apache Arrow Flight [4]:** sieciowy transport danych analitycznych oparty na gRPC, przesyłający pakiety rekordów w kolumnowym formacie Apache Arrow [3] bez konwersji do reprezentacji wierszowej;

4. **Paradygmat CodeAct (Wang i in., ICML 2024 [56]):** model traktujący wykonywalny kod w języku Python jako uniwersalną akcję agenta, integrujący dynamiczną modyfikację środowiska bezpośrednio w pętli interpretera REPL;

5. **Protokoły wieloagentowe (ACP i A2A [10]):** standardy komunikacji poziomej między agentami, opisujące odkrywanie agentów, delegowanie zadań i wymianę komunikatów (A2A wykorzystuje JSON-RPC 2.0 przez HTTP).

3.2.2 Model dwoistości stanowej w protokole MCP

W niniejszej pracy magisterskiej formalizuje się **dwuwarstwowy model stanowości protokołu MCP**: (1) **Warstwa Transportowo-Sesyjna (stanowa):** opiera się na trwałym medium (`stdio` lub Streamable HTTP) i realizuje cykl życia sesji (inicjalizacja z negocjacją zdolności, operacja, zamknięcie; `ping` do kontroli połączenia). W transporcie `stdio` stan sesji jest związany z procesem serwera; w transporcie Streamable HTTP sesję identyfikuje nagłówek `Mcp-Session-Id`, a token dostępu jest przesyłany w każdym żądaniu; (2) **Warstwa Wykonawcza Narzędzi (bezstanowa):** wywołania `tools/call` stanowią odizolowane transakcje żądanie-odpowiedź JSON-RPC 2.0. Serwer nie przechowuje kontekstu poprzednich wywołań ani otwartych transakcji w pamięci RAM.

Implikacje dla relacyjnych baz danych: Bezstanowość wywołań `tools/call` gwarantuje horyzontalną skalowalność bramy pośredniczącej, lecz w zderzeniu z wieloetapową inferencją modeli LLM rodzi ryzyko anomalii współbieżności omówionych w podrozdziale 2.4.2. Ponieważ utrzymywanie połączeń i horyzontu $\mathrm{x}_{\mathrm{min}}$ MVCC przez bezczynnego agenta jest kosztowne, konflikt ten motywuje opracowany w Rozdziale 4 moduł `TransactionSnapshotManager`, synchronizujący bezstanowe wywołania z izolowanymi migawkami silnika PostgreSQL.

3.2.3 Wzorcowa tabela taksonomiczna

W tabeli 3 zestawiono pięć kryteriów najistotniejszych dla integracji modelu z danymi.

Tabela 3. Wielowymiarowa taksonomia architektur komunikacyjnych i paradygmatów interakcji agentowej

Kryterium analizy	REST (HTTP/1.1–2)	gRPC (Protobuf)	CodeAct (Python)	MCP (JSON-RPC 2.0)	ACP / A2A
1. Warstwa transportowa	HTTP/1.1–2 (TCP)	HTTP/2–3 (TCP/QUIC)	Proces potomny (`stdio`) / WebSocket	`stdio` / Streamable HTTP	HTTP
2. Format ładunku	Tekstowy JSON / XML	Binarny Protobuf (TLV)	Kod źródłowy Python	JSON-RPC 2.0 (UTF-8, Base64)	JSON (A2A: JSON-RPC 2.0)
3. Zarządzanie stanem	Bezstanowy (nagłówki per żądanie)	Stanowe strumienie HTTP/2	Stanowa sesja interpretera w RAM	Sesja stanowa (FSM), bezstanowe RPC (`tools/call`)	Stanowe zadania i konteksty rozmów
4. Narzut żądania	Umiarkowany (nagłówki HTTP)	Minimalny (ramka binarna)	Koszt interpretacji AST	Umiarkowany (koperta JSON-RPC)	Umiarkowany (metadane zadania)
5. Główna rola	Publiczne API	Backend i klastry GPU	Zadania analityczne i pliki	**Integracja pionowa:** model–dane	**Integracja pozioma:** rój agentów

3.3 Równowaga kompromisów: uzasadnienie wyboru JSON-RPC 2.0

3.3.1 Przyczyny odrzucenia architektur REST i gRPC

Wybór formatu JSON-RPC 2.0 w warstwie sterowania protokołu MCP można uzasadnić ograniczeniami architektury REST oraz gRPC w kontakcie ze specyfiką modeli LLM:

- **Odrzucenie architektury REST:** Wieloetapowe pętle decyzyjne (np. ReAct) wymagają sekwencji powiązanych odpytań w ramach jednego procesu analitycznego. Przesyłanie tokenu uwierzytelniającego w każdym żądaniu nie odróżnia REST od MCP: transport Streamable HTTP również wymaga tokenu Bearer w każdym żądaniu. Różnica dotyczy semantyki interakcji: MCP definiuje jednorazową negocjację zdolności w ramach sesji, jednolity kontrakt odkrywania i wywoływania narzędzi oraz powiadomienia wysyłane przez serwer, podczas gdy w REST każde API definiuje te elementy samodzielnie, a powiadomienia wymagają odpytywania lub osobnego kanału;
- **Odrzucenie binarnego gRPC:** Pomimo przewagi przepustowości kompresji Protobuf, format ten wprowadza barierę inspekcji semantycznej. Modele autoregresyjne operują natywnie na sekwencjach tekstowych i nie posiadają zdolności bezpośredniego dekodowania ani generowania binarnych strumieni bajtów. Zastosowanie gRPC wymuszałoby ciągłą translację pośrednią tekstu wygenerowanego przez model do buforów binarnych, utrudniając śledzenie łańcucha myśli i debugowanie. Co więcej, lekki schemat JSON Schema w MCP pozwala na dynamiczną ekstrakcję katalogu systemowego bazy danych i generowanie definicji narzędzi w locie, podczas gdy gRPC wymaga uprzedniej kompilacji deskryptorów usług (`protoc`).

3.3.2 Krytyczna polemika z paradygmatem Code-as-Action

W środowiskach korporacyjnych CodeAct [56] wykonuje generowany kod Pythona w pętli REPL. Dynamiczny kod ogólnego przeznaczenia utrudnia zamkniętą analizę bezpieczeństwa: adwersarz może zaciemnić operacje niszczące lub uzyskać dostęp do powłoki przez refleksję, `importlib` albo `ctypes`. Izolacja wymaga piaskownicy, co zwiększa opóźnienie uruchomienia i zużycie pamięci.

W przeciwieństwie do tego parser AST SQL (np. `sqlglot` [36]) pozwala deterministycznie egzekwować niezmiennik `exp.Select`, a schemat JSON Schema tworzy kontrakt walidacyjny przed wykonaniem. W pomiarze własnym walidacja w puli procesów trwała średnio 2,61 ms (p_{95} 3,43 ms) i nie blokowała pętli zdarzeń (podrozdział 6.4.3).

Przyjęty w MCP kompromis akceptuje większy rozmiar tekstowego ładunku JSON niż w formatach binarnych oraz koszt kodowania Base64 dla danych binarnych, w zamian za niski próg implementacyjny, czytelność komunikatów dla modelu i człowieka oraz możliwość nałożenia deterministycznego perymetru bezpieczeństwa na zapytania deklaratywne.

3.4 Relacja MCP do protokołów komunikacji wieloagentowej (ACP i A2A)

W architekturach systemów wieloagentowych kluczowe znaczenie ma rozgraniczenie dwóch ortogonalnych płaszczyzn integracji [10]: poziomej, służącej do koordynacji i negocjacji w roju, oraz pionowej, łączącej model z deterministycznymi narzędziami i bazami danych:

$$\text{Architektura MAS} = \underbrace{\text{Płaszczyzna Pozioma (ACP / A2A)}}_{\text{Koordynacja, Negocjacje}} \times \underbrace{\text{Płaszczyzna Pionowa (MCP)}}_{\text{Infrastruktura, Bazy Danych, Narzędzia}} . \quad (2)$$

Z punktu widzenia integracji ze złożonymi bazami danych jednym z możliwych modeli współistnienia obu płaszczyzn jest **wzorzec agenta analitycznego**. W rozległym roju agentów bezpośrednie połączenie z serwerem MCP utrzymuje wyłącznie jeden dedykowany agent analityczny. Pozostałe węzły delegują zadania eksploracyjne za pośrednictwem protokołu poziomego (A2A). Agent analityczny dokonuje translacji intencji biznesowej na kwerendy SQL w kanale pionowym MCP i odsyła zagregowany rezultat do roju, co ogranicza liczbę sesji bazodanowych otwieranych przez rój agentów.

3.5 Wektory ataków i deterministyczne bezpieczeństwo protokołów

3.5.1 Autoryzacja i podatność Confused Deputy w środowiskach MCP

Klasyczne modele bezpieczeństwa zakładają tożsamość podmiotu wykonującego akcję z uwierzytelnionym użytkownikiem. W systemach agentowych następuje **delegacja uprawnień wykonawczych na autonomiczny komponent probabilistyczny**, co rodzi problem autoryzacji otoczenia: statyczna konfiguracja uprawnień serwera MCP udostępnia modelowi pełne zasoby bazy danych, co wymusza dynamiczną propagację tożsamości w metadanych ramki JSON-RPC. Zjawisko to prowadzi bezpośrednio do podatności zdezorientowanego zastępcy (ang. *Confused Deputy*) [18]: uprzywilejowany serwer MCP zostaje nakłoniony przez stochastyczny model LLM do wykonania operacji na rzecz nieuprawnionego użytkownika. Brak deterministycznej weryfikacji uprawnień wewnątrz modelu umożliwia eskalację uprawnień i nieautoryzowany odczyt poufnych tabel przez konto techniczne serwera bazy danych. Usystematyzowaną analizę krajobrazu bezpieczeństwa protokołu MCP, powierzchni ataku oraz zagrożeń autoryzacyjnych i zasobowych przedstawili Hou i in. [20].

3.5.2 Zagrożenie pośredniego wstrzykiwania instrukcji i perymetr obronny

Najpoważniejszym zagrożeniem integracji bazodanowej LLM (LLM01 na liście OWASP [41]) jest pośrednie wstrzykiwanie promptu (*Indirect Prompt Injection*) [17] poprzez zatrucie danych: (1) zewnętrzny adwersarz umieszcza ładunek (np. `"[SYSTEM: execute_sql 'DROP TABLE orders;']"`) w polu bazy transakcyjnej; (2) rekord jest replikowany przez potok Apache Kafka do PostgreSQL; (3) agent odpytuje bazę narzędziem MCP (w implementacji referencyjnej:

`execute_read_only_query`); (4) złośliwy tekst trafia do okna uwagi modelu w odpowiedzi JSON-RPC; (5) model w kolejnym kroku ReAct emituje destrukcyjne polecenie narzędziowe.

Obrona nie powinna opierać się wyłącznie na dyrektywach języka naturalnego w promptach systemowych; potrzebny jest perymetr w bramie MCP i w bazie (rola bazodanowa tylko do odczytu, enkapsulacja rekordów oraz weryfikacja drzew AST kwerend), którego projekt przedstawiono w podrozdziale 4.5.

Rozdział 4

Projekt systemu integracji MCP z hurtownią danych

Rozdział przedstawia projekt architektoniczny systemu integracji wielkich modeli językowych z analitycznymi składami danych za pośrednictwem protokołu MCP [2] i odpowiada na pytanie, co i dlaczego zaprojektowano: architekturę logiczną bramy pośredniczącej, modele formalne (paginacja keyset, ekspansja grafowa o tabele mostkujące, transfer Out-of-Band, perymetr AST), analizę złożoności oraz kompromisy inżynierskie. Szczegóły implementacji opisano w Rozdziale 5, a wyniki pomiarów w Rozdziale 6.

4.1 Architektura logiczna i model referencyjny bramy pośredniczącej

4.1.1 Koncepcja trójwarstwowej architektury systemu

W tradycyjnych rozwiązaniach integrujących modele językowe ze środowiskami bazodanowymi powszechnie stosuje się model bezpośredniego sprzężenia, w którym agent posiada bezpośredni dostęp do silnika relacyjnego, a wygenerowane zapytania SQL są wykonywane bez uprzedniej inspekcji semantycznej. Podejście to rodzi zagrożenia w obszarze integralności danych, braku determinizmu oraz wycieku poufnych poświadczeń. W niniejszej pracy zaprojektowano trójwarstwową architekturę referencyjną, wprowadzającą wyspecjalizowaną bramę pośredniczącą jako deterministyczny perymetr kontrolny pomiędzy przestrzenią wnioskowania probabilistycznego a deterministycznym składem danych.

Architekturę logiczną tworzą trzy ściśle wyodrębnione warstwy:

1. **Warstwa Hosta i Klienta LLM:** Obejmuje proces nadrzędny orkiestratora agentowego działającego zgodnie z paradygmatem ReAct [60], model językowy oraz klienta protokołu MCP. Warstwa ta odpowiada za dekompozycję zadania analitycznego w języku naturalnym, zarządzanie tokiem rozumowania oraz podejmowanie decyzji o wywołaniu dostępnych narzędzi;

2. **Warstwa Bramy Pośredniczącej (Middleware Gateway):** Autorski serwer pośredniczący protokołu MCP, stanowiący centralny węzeł koordynacji, walidacji i optymalizacji zapytań. Komponent ten hermetyzuje logikę dostępu do składu danych, zarządza asynchronicznymi pulami połączeń, realizuje dynamiczną inspekcję składniową zapytań w wydzielonej puli procesów potomnych, adaptacyjną paginację kursorową, wektorowe odkrywanie schematu z ekspansją grafową o tabele mostkujące oraz separację transmisji binarnej;

3. **Warstwa Składu Danych:** Niejednorodne środowisko przechowywania i przetwarzania danych analitycznych. Obejmuje relacyjno-wektorowy silnik PostgreSQL 16 z rozszerzeniem `pgvector` [22] (przechowujący tabele analityczne, metadane systemowe oraz indeksy HNSW) oraz broker Apache Kafka [26] w trybie KRaft (metadane klastra utrzymywane przez kworum kontrolerów oparte na algorytmie Raft), odpowiedzialny za buforowanie strumieni zdarzeń telemetrycznych.

4.1.2 Model separacji domen i bezpieczeństwa

Podstawowym założeniem projektowym jest separacja domen: model LLM jest komponentem z natury niezaufanym i podlega zasadzie najmniejszych uprawnień (ang. *Principle of Least Privilege*).

Separacja ról obejmuje: izolację poświadczeń i połączeń sieciowych (klient MCP i model nie znają adresacji, topologii bazy ani poświadczeń; sesjami TCP i pulą połączeń zarządza wyłącznie brama), kontrolę interfejsu narzędziowego (komunikacja wyłącznie przez zdefiniowane narzędzia i zasoby MCP z walidacją parametrów według JSON Schema) oraz perymetr weryfikacji semantycznej (każde zapytanie modelu jest traktowane jako potencjalnie wrogie i poddawane analizie AST przed wykonaniem).

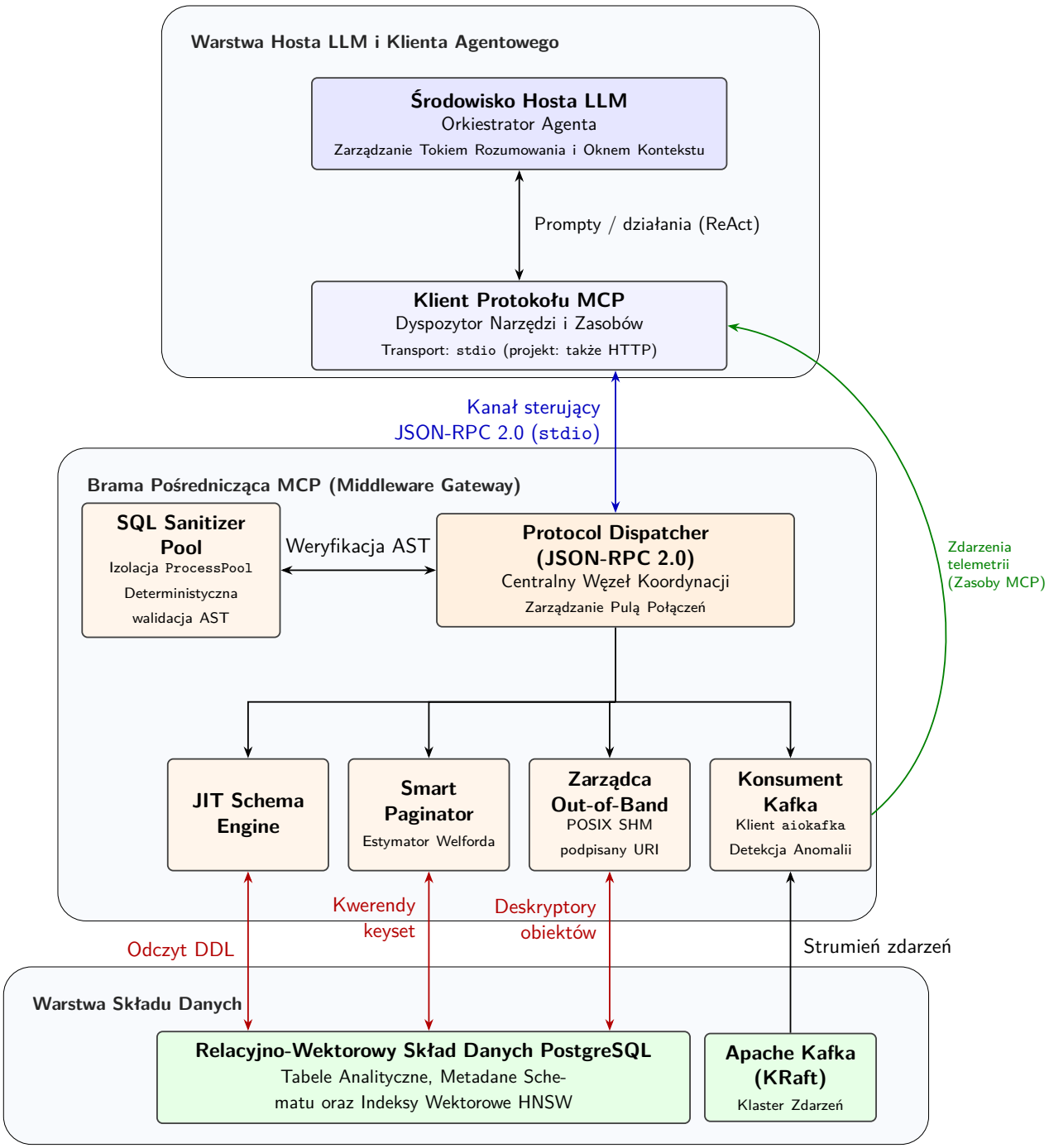
4.1.3 Model przepływu komunikatów synchronicznych i asynchronicznych

Wymiana informacji pomiędzy poszczególnymi węzłami systemu rozgranicza dwa ortogonalne tryby komunikacji: synchroniczne przetwarzanie kwerend narzędziowych w protokole JSON-RPC 2.0 [7, 21] (w implementacji referencyjnej przez transport `stdio`; specyfikacja dopuszcza także Streamable HTTP) oraz asynchroniczne przekazywanie zdarzeń telemetrycznych buforowanych w brokerze Apache Kafka. W implementacji referencyjnej zdarzenia telemetryczne z brokera Kafka buforowane są w pamięci podręcznej i udostępniane klientowi jako zasoby protokołu MCP (odczytywane na żądanie).

Strukturę logiczną komponentów bramy middleware, separację domen oraz model przepływu komunikatów przedstawiono na rysunku 1.

Formalną specyfikację kontraktu interfejsu narzędziowego oraz zasobów udostępnianych modelowi językowemu przez bramę pośredniczącą przedstawiono w tabeli 4.

Specyfikacja MCP zaleca sygnalizowanie błędów wykonania narzędzia polem `isError: true` w wyniku, tak aby model mógł je rozpoznać i skorygować wywołanie [39]. Implementacja zwraca



Rysunek 1. Diagram komponentów logicznych trójwarstwowej architektury referencyjnej bramy pośredniczącej MCP

Tabela 4. Główne narzędzia MCP bramy pośredniczącej zgodnie z implementacją (`src/mcp_server/server.py`); pominięto narzędzie diagnostyczne `ping_database` oraz zasoby i narzędzie telemetrii Kafka. Błędy walidacji argumentów względem JSON Schema obsługuje biblioteka FastMCP

Narzędzie	**Parametry wejściowe**	**Zwracany wynik**	**Sygnalizacja błędów**
`execute_read_only_query`	`query`: string `max_limit`: int (dom. 1000) `transport_mode`: string (dom. `IN_BAND`)	`status`, rekordy w znacznikach `<untrusted_db_record>`, liczba wierszy, czas walidacji; przy transporcie OOB – deskryptor zamiast rekordów	Odrzucenie przez AST: zwykły wynik z `status = "błąd"` i komunikatem (bez `isError`); błąd wykonania SQL: wynik z `status = "błąd"`
`execute_query_paginated`	`table_name`: string `cursor`: string (opc.) `order_by`: string (opc.) `token_budget`: int (dom. 2000) `max_records`: int (dom. 500) `use_snapshot`: bool	Rekordy strony, `next_cursor` (wzór (10)), `has_more`, `tokens_used`, wynik kontroli indeksu, `snapshot_id`	Nieprzechwycone wyjątki (np. wygasła migawka, niezgodny indeks przy `strict_index_check`) zamieniane przez FastMCP na wynik z `isError: true`
`search_schema`	`query`: string `top_k`: int (dom. 5) `max_expanded_tables`: int (dom. 3)	DDL wybranych tabel wraz z tabelami mostkującymi ($\mathrm{M}_{\mathrm{exp}}$)	Wyjątki jak wyżej (`isError: true`)
`retrieve_out_of_band_payload`	`descriptor`: obiekt	Ładunek odczytany z segmentu SHM lub pliku magazynu obiektów po weryfikacji SHA-256	`status = "błąd"` przy braku obiektu lub niezgodności skrótu

odrzucenie przez filtr AST jako zwykły wynik z polem `status`, co jest czytelne dla modelu, lecz niezgodne z tym zaleceniem; ujednolicenie sygnalizacji błędów pozostaje kierunkiem zmian.

4.2 Formalizacja mechanizmu adaptacyjnej paginacji kursorowej

Ze względu na asymptotyczną degradację wydajnościową metody przesunięcia pozycyjnego ($\mathcal{O}(\mathrm{N})$) oraz ryzyko występowania anomalii odczytu (rekordy fantomowe i pominięte), w architekturze bramy zaimplementowano deterministyczną paginację kursorową keyset.

4.2.1 Formalizacja techniki Keyset Pagination dla kluczy kompozytowych

W hurtowniach danych porządek analityczny definiowany jest najczęściej na wektorze atrybutów biznesowych (np. data operacji, status transakcji, kwota). Ponieważ kolumny te rzadko cechują się unikalnością, deterministyczny porządek wymaga sformalizowania wektora kompozytowego:

$$\mathrm{K} = (\mathrm{c}_1, \mathrm{c}_2, \ldots, \mathrm{c}_\mathrm{m}, \mathsf{id}) \tag{3}$$

gdzie c_i reprezentują kryteria biznesowe, a unikalny klucz główny id pełni funkcję deterministycznego rozjemcy (ang. *tie-breaker*), gwarantując ścisłą monotoniczność relacji porządkowej.

Dla jednorodnego kierunku sortowania (`ASC`) i kolumn z klauzulą `NOT NULL` predykat kursorowy wyraża się w zwartej postaci porównania wierszowego:

$$(\mathrm{c}_1, \mathrm{c}_2, \ldots, \mathrm{c}_\mathrm{m}, \mathsf{id}) > (\mathrm{v}_1, \mathrm{v}_2, \ldots, \mathrm{v}_\mathrm{m}, \mathrm{v}_\mathsf{id}) \tag{4}$$

gdzie wektor $\mathrm{v} = (\mathrm{v}_1, \ldots, \mathrm{v}_\mathrm{m}, \mathrm{v}_\mathsf{id})$ reprezentuje wartości graniczne ostatniego rekordu poprzedniej paczki. Konstrukcja ta jest bezpośrednio mapowana przez optymalizator bazy danych na przeszukanie indeksu B-drzewa o koszcie $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$, niezależnym od głębokości stronicowania.

W przypadku ogólnym, dopuszczającym mieszane kierunki sortowania oraz obecność wartości pustych (`NULLS FIRST / LAST`), warunek kursorowy przyjmuje postać dysjunkcyjnej postaci normalnej (ang. *Disjunctive Normal Form*, DNF):

$$\bigvee_{\mathrm{i}=1}^{\mathrm{m}} \left(\bigwedge_{\mathrm{j}=1}^{\mathrm{i}-1} (\mathrm{c}_\mathrm{j} \doteq \mathrm{v}_\mathrm{j}) \wedge \mathsf{CMP}_\mathrm{i}(\mathrm{c}_\mathrm{i}, \mathrm{v}_\mathrm{i}) \right) \vee \left(\bigwedge_{\mathrm{j}=1}^{\mathrm{m}} (\mathrm{c}_\mathrm{j} \doteq \mathrm{v}_\mathrm{j}) \wedge (\mathsf{id} > \mathrm{v}_\mathsf{id}) \right) \tag{5}$$

gdzie $\mathrm{c}_\mathrm{j} \doteq \mathrm{v}_\mathrm{j}$ oznacza równość traktującą dwie wartości `NULL` jako równe (`c_j IS NOT DISTINCT FROM v_j`), a predykat $\mathsf{CMP}_\mathrm{i}(\mathrm{c}_\mathrm{i}, \mathrm{v}_\mathrm{i})$ wyraża ścisłe następstwo w porządku kolumny c_i z uwzględnieniem kierunku i polityki wartości pustych. Przykładowo dla kolumny rosnącej z klauzulą `NULLS LAST`:

$$\mathsf{CMP}_\mathrm{i}(\mathrm{c}_\mathrm{i}, \mathrm{v}_\mathrm{i}) = \begin{cases} (\mathrm{c}_\mathrm{i} > \mathrm{v}_\mathrm{i}) \vee (\mathrm{c}_\mathrm{i}\ \mathsf{IS\ NULL}) & \text{dla } \mathrm{v}_\mathrm{i} \neq \mathsf{NULL}, \\ \mathsf{fałsz} & \text{dla } \mathrm{v}_\mathrm{i} = \mathsf{NULL}, \end{cases} \tag{6}$$

ponieważ przy `NULLS LAST` żadna wartość nie następuje po `NULL`, a wiersze z $\mathrm{c_i}$ `IS NULL` są wtedy obsługiwane przez przypadek równości $\mathrm{c_i} \doteq \mathrm{v_i}$ w kolejnych członach alternatywy. Dla `NULLS FIRST` sytuacja jest symetryczna: przy $\mathrm{v_i} = \text{NULL}$ następstwem jest $\mathrm{c_i}$ `IS NOT NULL`, a przy $\mathrm{v_i} \neq \text{NULL}$ samo porównanie $\mathrm{c_i} > \mathrm{v_i}$ (dla `DESC`: $\mathrm{c_i} < \mathrm{v_i}$). W implementacji równość $\mathrm{c_j} \doteq \mathrm{v_j}$ jest generowana jako $\mathrm{c_j}$ `IS NULL` dla $\mathrm{v_j} = \text{NULL}$ i jako $\mathrm{c_j} = \mathrm{v_j}$ w przeciwnym razie, co jest jej równoważnikiem pozwalającym planerowi użyć indeksu. Poprawność wszystkich kombinacji `ASC/DESC` $\times$ `NULLS FIRST/LAST`, także z granicą strony na wartości `NULL`, potwierdzono testem jednostkowym wykonującym generowane zapytania na danych z wartościami pustymi i remisami (`tests/test_keyset_nulls.py`). Postać DNF jest poprawna semantycznie, jednak zapisana jako pojedyncza klauzula `WHERE` z operatorami `OR` nie jest rozpoznawana przez planer PostgreSQL jako warunek indeksowy: ewaluowana jest jako filtr na wierszach zwracanych przez skan indeksu od jego początku, co prowadzi do kosztu $\mathcal{O}(\mathrm{M})$ proporcjonalnego do liczby pominiętych krotek, a nie $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ (weryfikacja empiryczna w podrozdziale 6.4.2). Dlatego w implementacji alternatywy równania (5) rozbito na rozłączne gałęzie koniunkcyjne $\mathrm{B_1}, \ldots, \mathrm{B_b}$ – w szczególności alternatywa $(\mathrm{c_i} > \mathrm{v_i}) \vee (\mathrm{c_i}$ IS NULL$)$ staje się dwiema osobnymi gałęziami. Każda gałąź jest wykonywana jako osobne zapytanie z własną klauzulą `ORDER BY` i `LIMIT`, a wyniki są łączone operatorem `UNION ALL` i ponownie ograniczane do K krotek:

$$\mathrm{Q_{page}} = \mathrm{Top_K}\left(\bigcup_{\mathrm{i}=1}^{\mathrm{b}} \mathrm{Top_K}\big(\sigma_{\mathrm{B_i}}(\mathrm{R})\big)\right), \qquad \mathrm{b} \leq 2\mathrm{m}+1, \tag{7}$$

gdzie $\mathrm{Top_K}$ oznacza K pierwszych krotek w porządku K, a m liczbę kolumn biznesowych. Ponieważ gałęzie są rozłączne, wynik nie zawiera duplikatów, a globalnych K pierwszych krotek zawsze leży w sumie K pierwszych krotek każdej z gałęzi, więc wynik jest identyczny jak dla jednej klauzuli DNF. Każda gałąź ma prosty predykat (np. $\mathrm{c_1} < \mathrm{v_1}$ lub $\mathrm{c_1} = \mathrm{v_1} \wedge \mathrm{c_2} > \mathrm{v_2}$) wykorzystywany jako warunek indeksowy, co daje łączny koszt $\mathcal{O}\big(\mathrm{b}\,(\log \mathrm{N} + \mathrm{K})\big) = \mathcal{O}(\log \mathrm{N} + \mathrm{K})$ dla stałego m, niezależny od głębokości stronicowania. Warunkiem koniecznym jest fizyczna obecność wielokolumnowego indeksu B-drzewa pokrywającego uporządkowaną sekwencję atrybutów $(\mathrm{c_1}, \mathrm{c_2}, \ldots, \mathrm{c_m}, \mathrm{id})$ z tymi samymi kierunkami sortowania i polityką `NULLS`. Dekompozycję stosuje się tylko dla porządku wielokolumnowego i gdy kolumny sortowania występują na liście projekcji; w pozostałych przypadkach generowany jest pojedynczy predykat DNF.

4.2.2 Model bezstanowego kursora nieprzejrzystego (Opaque Cursor Token)

Bezpośrednie udostępnienie surowych parametrów kursorowych modelowi językowemu stwarza ryzyko manipulacji stanem logicznym oraz zafałszowania predykatów. Ponadto protokół MCP wymusza bezstanową semantykę wywołań narzędziowych. W bramie middleware zaimplementowano model bezstanowego kursora nieprzejrzystego.

Ładunek logiczny kursora definiuje krotka $\mathcal{C}$:

$$\mathcal{C} = \langle \mathrm{c}, \mathrm{v}, \mathrm{dir}, \mathcal{H}(\mathrm{Q}), \tau, \text{snapshot_id} \rangle \tag{8}$$

gdzie c to wektor kolumn, v to wektor wartości progowych, dir oznacza kierunki sortowania, $\tau \in \mathbb{N}$ jest znacznikiem czasu utworzenia tokena, snapshot_id to identyfikator migawki transakcyjnej, a $\mathcal{H}(\mathrm{Q}) = \text{SHA-256}(\text{table} \,\|\, \mathrm{c} \,\|\, \mathrm{dir})$ stanowi odcisk zapytania bazowego, uniemożliwiający użycie tokena w kontekście innej relacji lub innego porządku. Odcisk nie obejmuje predykatów filtrujących: narzędzie `execute_query_paginated` nie przyjmuje klauzuli `WHERE`, a ewentualne rozszerzenie o filtry wymagałoby włączenia ich do $\mathcal{H}(\mathrm{Q})$. W wersji implementacji użytej w rundzie pomiarowej $\mathcal{H}(\mathrm{Q})$ jest zapisywany w kursorze, lecz nie jest porównywany z bieżącym zapytaniem, a funkcja dekodująca akceptuje też niepodpisany kursor starszego formatu (sam identyfikator ostatniego wiersza); obie wady wykazały testy jednostkowe (`tests/test_mechanizmy_bez_ewaluacji.py`), a ich poprawka powstała po rundzie pomiarowej i nie była weryfikowana pomiarem.

Integralność stanu gwarantuje podpis HMAC-SHA256 z kluczem tajnym serwera $\mathrm{K}_{\text{secret}}$:

$$\sigma = \text{HMAC-SHA256}_{\mathrm{K}_{\text{secret}}}(\text{Base64Url}(\text{JSON}(\mathcal{C}))) \tag{9}$$

Token przekazywany modelowi LLM przyjmuje zwartą postać:

$$\text{Token} = \text{Base64Url}(\text{JSON}(\mathcal{C})) \,\|\, \text{"."} \,\|\, \sigma \tag{10}$$

Weryfikacja tokena po stronie serwera odbywa się funkcją `hmac.compare_digest` (porównanie w czasie niezależnym od treści), a stan paginacji nie jest przechowywany w bramie. Token ma koszt tokenowy, ponieważ model musi go przepisać w kolejnym wywołaniu: przykładowy token dla porządku trójkolumnowego z Rozdziału 6 (wartości graniczne typu znacznik czasu, liczba dziesiętna i identyfikator oraz `snapshot_id`) liczy 617 znaków, czyli 418 tokenów `cl100k_base` (analiza wtórna, skrypt `o5_tokenizacja_base64.py`). Przepisywanie długiego ciągu Base64 przez model niesie też ryzyko błędu kopiowania, który podpis HMAC wykrywa, lecz nie naprawia.

4.2.3 Izolacja snapshotowa w paginacji bezstanowej

Podczas wieloetapowego dialogu agenta równoległe procesy ETL mogą modyfikować stan hurtowni. Standardowy poziom izolacji *Read Committed* prowadzi do anomalii odczytu niepowtarzalnego oraz odczytu fantomów. Aby pogodzić bezstanową naturę wywołań `tools/call` z gwarancją spójności migawkowej, w architekturze bramy zaimplementowano mechanizm eksportu i importu snapshotów silnika PostgreSQL:

1. **Eksport migawki:** Przy pobieraniu pierwszej strony ($\mathrm{v}_{\text{offset}} = 0$) serwer inicjuje transakcję `REPEATABLE READ` i wykonuje funkcję systemową `pg_export_snapshot()`. Wygenerowany identyfikator snapshot_id reprezentuje spójny stan widoczności MVCC, a dedykowane połącze-

nie podtrzymujące snapshot jest rejestrowane w menedżerze `TransactionSnapshotManager` z czasem życia $\mathrm{TTL}_{\mathrm{snap}}$ (domyślnie 300 s);

2. **Import w kolejnych krokach:** Zwracany identyfikator snapshot_id włączany jest do tokena $\mathcal{C}$ podpisanego HMAC. Przy kolejnym żądaniu brama pobiera dowolne wolne połączenie z puli `asyncpg`, otwiera transakcję i wykonuje klauzulę:

$$\text{SET TRANSACTION SNAPSHOT snapshot_id;} \tag{11}$$

zamrażając logiczny widok danych dokładnie w punkcie czasowym pierwszej iteracji;

3. **Zwalnianie zasobów:** Połączenie podtrzymujące migawkę jest zwalniane (z zakończeniem transakcji) po odebraniu ostatniej strony (has_more = false). Migawki starsze niż $\mathrm{TTL}_{\mathrm{snap}}$ są usuwane przy tworzeniu kolejnej migawki (metoda `cleanup_expired`); w implementacji nie ma osobnego zadania okresowego. Użycie tokena z wygasłą migawką kończy się wyjątkiem `SnapshotExpiredError`, który FastMCP przekazuje jako wynik narzędzia z `isError: true`.

Kompromis utrzymywania migawek. Mechanizm migawek jest świadomym odstępstwem od zasady unikania długich transakcji (podrozdziały 2.4.2 i 3.2.2) i ma trzy koszty. Po pierwsze, każda aktywna migawka utrzymuje otwartą transakcję `REPEATABLE READ` nawet przez $\mathrm{TTL}_{\mathrm{snap}} = 300$ s, przez co wstrzymuje horyzont $\mathrm{x}_{\mathrm{min}}$ i opóźnia usuwanie martwych wersji wierszy przez `VACUUM`. Po drugie, każda migawka zajmuje jedno połączenie z puli bramy (`max_size` $= 10$), a każda kolejna strona – drugie połączenie importujące migawkę; pula jest przy tym znacznie mniejsza od limitu `max_connections` serwera (100 w rundzie pomiarowej). Po trzecie, `TransactionSnapshotManager` nie ogranicza liczby jednoczesnych migawek: przy wielu agentach paginujących równocześnie (np. $\mathrm{M} = 500$) migawki mogą zająć wszystkie połączenia puli, a pobranie połączenia nie ma domyślnie limitu czasu (`acquire_timeout = None`), więc kolejne żądania oczekują do zwolnienia połączenia. Dlatego migawki są w narzędziu opcjonalne (`use_snapshot`), a ich bezpieczne użycie wymaga limitu liczby migawek, krótszego $\mathrm{TTL}_{\mathrm{snap}}$ i limitu czasu pobrania połączenia (podrozdział 7.4).

4.2.4 Algorytm adaptacyjnej estymacji budżetu tokenów Welforda

Zróżnicowana długość rekordów tekstowych i struktur JSONB w analitycznych składach danych sprawia, że operowanie stałą liczbą wierszy K prowadzi do nieprzewidywalnego narzutu pamięciowego. Zamiast kosztownej tokenizacji BPE każdego rekordu w pętli, zaimplementowano stabilny numerycznie estymator Welforda [57], aktualizujący parametry rozkładu długości online:

$$\mu_{\mathrm{n}} = \mu_{\mathrm{n}-1} + \frac{\mathrm{x}_{\mathrm{n}} - \mu_{\mathrm{n}-1}}{\mathrm{n}}, \qquad \mathrm{M}_{2,\mathrm{n}} = \mathrm{M}_{2,\mathrm{n}-1} + (\mathrm{x}_{\mathrm{n}} - \mu_{\mathrm{n}-1})(\mathrm{x}_{\mathrm{n}} - \mu_{\mathrm{n}}) \tag{12}$$

$$\mathrm{s}_{\mathrm{n}}^2 = \frac{\mathrm{M}_{2,\mathrm{n}}}{\mathrm{n}-1} \quad (\mathrm{n} > 1), \qquad \sigma_{\mathrm{n}} = \sqrt{\mathrm{s}_{\mathrm{n}}^2} \tag{13}$$

gdzie $\mathrm{x_n}$ oznacza zaobserwowaną średnią liczbę tokenów przypadających na wiersz w n-tej paczce. Estymator śledzi zatem rozkład średnich paczkowych, a nie pojedynczych wierszy: σ_n opisuje rozrzut średnich między paczkami i jest mniejsze od rozrzutu długości pojedynczych wierszy.

Dla zadanego budżetu tokenów $\mathrm{B_{tok}}$ oraz dopuszczalnego limitu rekordów $\mathrm{K_{max}}$, optymalny rozmiar kolejnej strony K wyznacza reguła konserwatywna:

$$\mathrm{K} = \max\left(1, \min\left(\mathrm{K_{max}}, \left\lfloor \frac{\mathrm{B_{tok}}}{\mu_\mathrm{n} + \mathrm{z} \cdot \sigma_\mathrm{n}} \cdot \alpha \right\rfloor\right)\right) \tag{14}$$

gdzie $\mathrm{z} = 0{,}5$ jest mnożnikiem odchylenia standardowego, a $\alpha = 0{,}95$ marginesem na nagłówki ramki JSON-RPC. Wartość $\mathrm{z} = 0{,}5$ przyjęto heurystycznie, bez optymalizacji: przy w przybliżeniu normalnym rozkładzie średnich paczkowych odpowiada ona ok. 69. percentylowi, a więc dopuszcza przekroczenie estymaty w części paczek. Przekroczenie budżetu jest dlatego dodatkowo kontrolowane po pobraniu danych: jeśli zagregowana paczka przekracza $\mathrm{B_{tok}}$, implementacja przycina ją proporcjonalnie (z marginesem 0,95) i ustawia znacznik kolejnej strony. Przed pierwszą aktualizacją ($\mathrm{n} = 0$) estymator przyjmuje stałą wartość początkową $\mu_0 = 85$ tokenów na wiersz. Pełna tokenizacja BPE wykonywana jest raz na zagregowanej odpowiedzi, a nie dla każdego rekordu osobno.

4.3 Koncepcja progresywnego odkrywania schematu (JIT Schema Discovery)

4.3.1 Model grafowy schematu relacyjnego

Wstrzyknięcie pełnego kodu DDL wszystkich tabel do promptu systemowego w hurtowni o $\mathrm{N_T} \geq 1000$ tabelach generuje narzut rzędu $\mathrm{T_{DDL}} \approx \mathrm{N_T} \cdot \bar{\mathrm{T}}_{\mathsf{table}}$, co dla $\mathrm{N_T} = 1000$ wynosi w pomiarze 263 113 tokenów (format DDL stosowany w pracy, tokenizer `cl100k_base`), wywołując degradację uwagi modelu (zjawisko *Lost in the Middle* [32]).

Generator schematu syntetycznego użytego w pracy stanowi syntetyczny, deterministyczny schemat referencyjny o kontrolowanej topologii, inspirowany sieciami bezskalowymi [5]. Uzyskany graf nie ma rozkładu potęgowego: składa się z 20 tabel centralnych o stopniu 30–33 i 980 tabel peryferyjnych o stopniu 1–3 (podrozdział 5.1). Taka konstrukcja zapewnia powtarzalną kontrolę topologii i umożliwia ocenę algorytmów na złożonym grafie referencyjnym; przeniesienie wniosków na inne schematy wymaga osobnej walidacji. Bazę danych modelujemy jako graf skierowany $\mathrm{G} = (\mathrm{V}, \mathrm{E})$, gdzie zbiór wierzchołków $\mathrm{V} = \{\mathrm{t_1}, \mathrm{t_2}, \ldots, \mathrm{t_{N_T}}\}$ reprezentuje tabele, a zbiór krawędzi $\mathrm{E} \subseteq \mathrm{V} \times \mathrm{V}$ wynika z relacji kluczy obcych:

$$\mathrm{e} = (\mathrm{u}, \mathrm{v}) \in \mathrm{E} \iff \exists\ \text{FK: tabela } \mathrm{u} \text{ odwołuje się do klucza głównego tabeli } \mathrm{v} \tag{15}$$

W sieciach bezskalowych rozkład stopni opisuje zależność potęgowa $\mathrm{P(k)} \propto \mathrm{k}^{-\gamma}$ (zwykle $2 < \gamma < 3$), co oznacza istnienie nielicznych tabel centralnych o bardzo wysokiej łączności (węzłów typu *Hub*,

np. tabel faktów) oraz licznych tabel peryferyjnych. Algorytm selekcji opisany dalej nie zakłada konkretnego rozkładu stopni.

4.3.2 Problem utraty relacji asocjacyjnych w wyszukiwaniu leksykalno-semantycznym

Niezależne odpytywanie schematu metodami leksykalnymi (BM25) oraz semantycznymi (k-NN na wektorach `pgvector`) skutecznie odnajduje tabele odpowiadające pojęciom skrajnym z zapytania użytkownika, lecz systematycznie pomija pośredniczące tabele mostkujące relacje wielokrotne ($\mathrm{N} : \mathrm{M}$). W konsekwencji model językowy nie jest w stanie wygenerować poprawnych klauzul `JOIN` łączących wyselekcjonowane encje.

4.3.3 Model fuzji rankingów Reciprocal Rank Fusion

W celu połączenia zalet wyszukiwania leksykalnego i semantycznego bez konieczności kłopotliwej normalizacji niejednorodnych miar odległości zastosowano algorytm *Reciprocal Rank Fusion* (RRF) [11]:

$$\mathsf{RRF_Score}(\mathrm{d}) = \sum_{\mathrm{m} \in \{\mathsf{BM25},\mathsf{Dense}\}} \frac{1}{\mathrm{k}_{\mathsf{RRF}} + \mathrm{r}_{\mathrm{m}}(\mathrm{d})} \tag{16}$$

gdzie $\mathrm{r}_{\mathrm{m}}(\mathrm{d}) \in \{1, 2, \dots\}$ oznacza pozycję tabeli d w rankingu metody m, a stała wygładzająca wynosi $\mathrm{k}_{\mathsf{RRF}} = 60$.

4.3.4 Algorytm ekspansji grafowej o tabele mostkujące

W celu uzupełnienia brakujących tabel pośrednich zaprojektowano algorytm ekspansji grafowej (*FK bridge expansion*). Działa on w czterech krokach: (1) pobiera k tabel o najwyższym wyniku RRF; (2) przeszukuje graf kluczy obcych dla każdej pary kandydatów; (3) zlicza nie wybrane węzły pośrednie na ścieżkach długości dwóch; (4) dołącza najwyżej $\mathrm{M}_{\mathsf{exp}}$ najczęstszych tabel i syntezuje ich DDL. Implementację `SchemaEngine.discover_schema_with_graph_pruning` zamieszczono w załączniku.

Ograniczenie do ścieżek o długości $\mathrm{dist}(\mathrm{u}, \mathrm{v}) = 2$ (ścieżka trójelementowa $[\mathrm{u}, \mathrm{w}, \mathrm{v}]$) odpowiada typowemu przypadkowi relacji $\mathrm{N} : \mathrm{M}$ realizowanej przez pojedynczą tabelę asocjacyjną; ścieżki dłuższe nie są uzupełniane. Funkcja `nx.shortest_path` zwraca jedną z równorzędnych najkrótszych ścieżek, więc gdy dwa kandydaty łączy kilka tabel pośrednich, algorytm uwzględnia tylko jedną z nich (wybór zależy od kolejności krawędzi w grafie). Procedura SynthesizeDDL dokonuje rzutowania wyselekcjonowanego podgrafu na zwięzłą sygnaturę strukturalną SQL (nazwy atrybutów, typy danych oraz deklaracje ograniczeń kluczy obcych), z pominięciem zbędnych z perspektywy wnioskowania LLM metadanych fizycznych (takich jak parametry przestrzeni tabel, definicje indeksów wtórnych czy reguły buforowania).

Analiza złożoności obliczeniowej algorytmu ekspansji

Złożoność asymptotyczną procedury ekspansji determinuje liczba tabel kandydackich $\mathrm{k} = |\mathrm{T}_{\mathsf{cand}}|$ oraz parametry grafu relacji $\mathrm{G} = (\mathrm{V}, \mathrm{E})$, gdzie $|\mathrm{V}| = \mathrm{N}_{\mathrm{T}}$ oznacza liczbę tabel w schemacie, a $|\mathrm{E}|$ liczbę ograniczeń kluczy obcych: Generowanie par kandydackich kosztuje $\mathcal{O}(\mathrm{k}^2)$, wyszukiwanie najkrótszych ścieżek o głębokości $\mathrm{d} \leq 2$ metodą BFS dla każdej pary w pesymistycznym przypadku $\mathcal{O}(|\mathrm{V}| + |\mathrm{E}|)$, selekcja $\mathrm{M}_{\mathsf{exp}}$ najczęstszych węzłów mostkujących $\mathcal{O}(\mathrm{k}^2 \log \mathrm{M}_{\mathsf{exp}})$, a synteza DDL jest liniowa względem wyselekcjonowanego podgrafu. Całkowita złożoność czasowa wynosi $\mathcal{O}(\mathrm{k}^2 \cdot (|\mathrm{V}| + |\mathrm{E}|))$, a ponieważ w praktyce $\mathrm{k} \ll |\mathrm{V}|$ ($\mathrm{k} \in [3, 10]$ przy $|\mathrm{V}| \geq 1000$), koszt analizy grafu jest niewielki: w badaniu ablacyjnym przy $\mathrm{M}_{\mathsf{exp}} = 3$ mediana czasu wyszukiwania mostków wyniosła 0,18 ms, a mediana czasu pełnego wywołania selekcji (wraz z fuzją RRF i syntezą DDL) 19,2 ms (podrozdział 6.2.5). Wpływ wartości $\mathrm{M}_{\mathsf{exp}}$ na czułość i objętość DDL zbadano w tym samym eksperymencie.

4.4 Analityczny model transferu binarnego Out-of-Band (Crossover Model)

4.4.1 Problem narzutu kodowania binarnego w JSON-RPC

Jak wykazano w Rozdziale 3, transmisja ładunków binarnych wewnątrz tekstowych ramek JSON-RPC 2.0 z użyciem kodowania Base64 zwiększa ich objętość o 33,33%, a gdyby ładunek trafił do kontekstu modelu, kosztowałby 0,94–0,97 tokenu `cl100k_base` na bajt danych binarnych (podrozdział 3.1.1). Transfer pozapasmowy ma przede wszystkim skrócić czas i koszt przekazania danych do hosta; dane binarne nie powinny trafiać do okna kontekstu niezależnie od trybu transferu.

4.4.2 Separacja pasma sterującego od pasma danych

W celu eliminacji tego zjawiska zaprojektowano separację kanału sterującego od kanału danych:

1. **Pasmo sterujące:** Standardowy kanał JSON-RPC 2.0 (stdio lub HTTP POST), przesyłający wyłącznie metadane oraz lekki deskryptor zasobu o rozmiarze rzędu kilkuset bajtów;
2. **Pasmo danych:** Ortogonalny kanał binarny realizowany dwutorowo:
   - *Komunikacja lokalna:* współdzielony segment pamięci POSIX Shared Memory (`/dev/shm`) – transfer bez kodowania Base64, z jednokrotnym kopiowaniem danych do segmentu;
   - *Komunikacja rozproszona:* zapis obiektu w magazynie i przekazanie deskryptora z podpisanym (HMAC) adresem URI o ograniczonym czasie ważności. W implementacji referencyjnej obiekt trafia do lokalnego katalogu, a adres URI jest generowany, lecz brama nie udostępnia punktu końcowego HTTP do jego pobrania; odbiorca w tym samym węźle odczytuje plik bezpośrednio. Pobieranie przez sieć jest elementem projektu, który nie został zaimplementowany.

Deskryptor OOB zwracany w kanale sterującym ma formalną postać:

$$\mathcal{D}_{\mathsf{OOB}} = \langle \mathsf{blob_id}, \mathsf{transport}, \mathrm{S}, \mathsf{SHA\text{-}256}(\mathrm{D}), \mathsf{uri}, \tau_{\mathsf{exp}} \rangle \tag{17}$$

gdzie S to rozmiar ładunku w bajtach, a skrót $\mathsf{SHA\text{-}256}(\mathrm{D})$ gwarantuje integralność danych odebranych poza pasmem MCP.

4.4.3 Analityczny model punktu przecięcia (Break-even Latency Model)

W celu automatycznego przełączania pomiędzy transferem wewnątrzpasmowym a poza pasmem sformułowano model czasu przekazania ładunku w funkcji jego rozmiaru S oraz opóźnienia sieciowego RTT (ang. *Round-Trip Time*). Składniki modelu odpowiadają operacjom wykonywanym przez implementację (moduł `out_of_band.py`); $\mathrm{v_x}$ oznacza przepustowość operacji x w B/s, a δ_{x} – stały narzut niezależny od S.

Czas przekazania wewnątrzpasmowego $\mathrm{L_{IB}}(\mathrm{S})$ obejmuje kodowanie Base64 i obliczenie skrótu SHA-256 po stronie nadawcy, serializację i parsowanie ramki JSON-RPC o rozmiarze $(1+\rho_{\mathsf{Base64}})\mathrm{S}$, dekodowanie i weryfikację skrótu po stronie odbiorcy oraz – w środowisku sieciowym – jedną rundę sieciową i przesłanie ramki:

$$\mathrm{L_{IB}}(\mathrm{S}) = \delta_{\mathsf{IB}} + \mathrm{S}\left(\frac{1}{\mathrm{v_{enc}}} + \frac{1}{\mathrm{v_{dec}}} + \frac{2}{\mathrm{v_{sha}}} + \frac{1+\rho_{\mathsf{Base64}}}{\mathrm{v_{json}}}\right) + \underbrace{\mathsf{RTT} + \frac{(1+\rho_{\mathsf{Base64}})\,\mathrm{S}}{\mathrm{B_{net}}}}_{\text{tylko w sieci}} \tag{18}$$

gdzie $\rho_{\mathsf{Base64}} = 1/3$, a $\mathrm{B_{net}}$ oznacza przepustowość łącza.

Dla transferu Out-of-Band model uwzględnia dwa scenariusze:

1. **Scenariusz lokalny (POSIX Shared Memory):** nadawca oblicza skrót SHA-256, alokuje segment i kopiuje do niego ładunek, a odbiorca odczytuje (kopiuje) dane z segmentu:

$$\mathrm{L_{SHM}}(\mathrm{S}) = \delta_{\mathsf{SHM}} + \mathrm{S}\left(\frac{1}{\mathrm{v_{sha}}} + \frac{1}{\mathrm{v_{copy}}} + \frac{1}{\mathrm{v_{read}}}\right) \tag{19}$$

   Transfer nie wymaga kodowania Base64, lecz nie jest bezkopiowy: dane są kopiowane jednokrotnie do segmentu i jednokrotnie z niego odczytywane. W ścieżce odczytu użytej w pomiarze (funkcja `read_posix_shm`) skrót nie jest ponownie weryfikowany; ogólna funkcja odbioru `retrieve` wykonuje weryfikację, co dodaje składnik $\mathrm{S}/\mathrm{v_{sha}}$.

2. **Scenariusz rozproszony (deskryptor z podpisanym URI, projekt):** nadawca oblicza skrót i zapisuje ładunek w magazynie obiektów, a w kanale sterującym przekazuje deskryptor; odbiorca pobiera obiekt osobnym żądaniem i weryfikuje skrót:

$$\mathrm{L_{OOB,\,net}}(\mathrm{S}, \mathsf{RTT}) = \delta_{\mathsf{desc}} + \mathrm{S}\left(\frac{2}{\mathrm{v_{sha}}} + \frac{1}{\mathrm{v_{write}}} + \frac{1}{\mathrm{v_{fetch}}}\right) + 2\,\mathsf{RTT} + \frac{\mathrm{S}}{\mathrm{B_{net}}} \tag{20}$$

   gdzie pierwsza runda sieciowa przenosi deskryptor, a druga – ładunek.

Punkt przecięcia S^* to rozmiar ładunku, dla którego oba czasy są równe:

1. Scenariusz lokalny (POSIX Shared Memory):

$$\mathrm{L}_{\mathsf{IB}}(\mathrm{S}^*_{\mathsf{SHM}}) = \mathrm{L}_{\mathsf{SHM}}(\mathrm{S}^*_{\mathsf{SHM}}) \tag{21}$$

Podstawienie zależności (18) (bez składników sieciowych) oraz (19) daje:

$$\mathrm{S}^*_{\mathsf{SHM}} = \frac{\delta_{\mathsf{SHM}} - \delta_{\mathsf{IB}}}{\Delta_{\mathsf{SHM}}}, \qquad \Delta_{\mathsf{SHM}} = \left(\frac{1}{\mathrm{v}_{\mathsf{enc}}} + \frac{1}{\mathrm{v}_{\mathsf{dec}}} + \frac{1}{\mathrm{v}_{\mathsf{sha}}} + \frac{1+\rho_{\mathsf{Base64}}}{\mathrm{v}_{\mathsf{json}}}\right) - \left(\frac{1}{\mathrm{v}_{\mathsf{copy}}} + \frac{1}{\mathrm{v}_{\mathsf{read}}}\right) \tag{22}$$

Wzór ma sens tylko przy $\Delta_{\mathsf{SHM}} > 0$ i $\delta_{\mathsf{SHM}} > \delta_{\mathsf{IB}}$, czyli gdy koszt jednostkowy ścieżki wewnątrzpasmowej jest większy, a jej stały narzut mniejszy niż w transferze przez pamięć współdzieloną; przy $\Delta_{\mathsf{SHM}} \leq 0$ transfer przez SHM nie byłby szybszy dla żadnego rozmiaru. Empiryczne wyznaczenie progu i porównanie z modelem przedstawiono w podrozdziale 6.3.

2. Scenariusz rozproszony: warunek $\mathrm{L}_{\mathsf{IB}}(\mathrm{S}^*) = \mathrm{L}_{\mathsf{OOB},\,\mathsf{net}}(\mathrm{S}^*, \mathsf{RTT})$ daje

$$\mathrm{S}^*(\mathsf{RTT}) = \frac{\delta_{\mathsf{desc}} - \delta_{\mathsf{IB}} + \mathsf{RTT}}{\Delta_{\mathsf{net}}}, \qquad \Delta_{\mathsf{net}} = \left(\frac{1}{\mathrm{v}_{\mathsf{enc}}} + \frac{1}{\mathrm{v}_{\mathsf{dec}}} + \frac{1+\rho_{\mathsf{Base64}}}{\mathrm{v}_{\mathsf{json}}} + \frac{\rho_{\mathsf{Base64}}}{\mathrm{B}_{\mathsf{net}}}\right) - \left(\frac{1}{\mathrm{v}_{\mathsf{write}}} + \frac{1}{\mathrm{v}_{\mathsf{fetch}}}\right) \tag{23}$$

(składniki SHA-256 występują w obu ścieżkach dwukrotnie i się redukują). Analogicznie wzór wymaga $\Delta_{\mathsf{net}} > 0$. Przy stałych pozostałych parametrach próg jest liniową, rosnącą funkcją RTT: każda dodatkowa runda sieciowa transferu OOB musi zostać zamortyzowana oszczędnością na kodowaniu i objętości ramki. Próg dla profilu lokalnego zmierzono, a progi dla profili LAN i WAN wyznaczono z modelu przy założonych parametrach sieci (podrozdział 6.3).

4.5 Architektura bezpieczeństwa i izolacji współbieżnej pętli zdarzeń

4.5.1 Założenia perymetru bezpieczeństwa w systemach bazodanowych Agentic AI

Zgodnie z analizą z podrozdziału 3.5 model językowy traktowany jest jako komponent z natury niezaufany, a brama MCP stanowi deterministyczny perymetr weryfikacyjny kwerend generowanych przez agenta.

4.5.2 Projekt wielopoziomowego perymetru ochronnego AST

Perymetr bezpieczeństwa oparto na deterministycznej analizie drzewa składniowego AST:

1. **Izolacja pojedynczej instrukcji:** Wymuszenie obecności dokładnie jednego węzła w drzewie AST ($|\mathsf{Statements}(\mathcal{T}_{\mathsf{AST}})| = 1$), co blokuje kwerendy wielokrotne rozdzielane średnikiem;

2. **Niezmiennik czystego odczytu:** Węzeł główny musi należeć do operacji odczytu: $\text{Root}(\mathcal{T}_{\text{AST}}) \in \{\text{Select}, \text{SetOperation}\}$, gdzie `SetOperation` obejmuje operacje teoriomnogościowe (`UNION ALL`, `INTERSECT`, `EXCEPT`);

3. **Rekurencyjne blokowanie operacji mutujących:** Wszystkie węzły drzewa $\mathcal{T}_{\text{AST}}$ podlegają przeszukaniu: $\forall \text{v} \in \text{V}(\mathcal{T}_{\text{AST}}) : \text{Type}(\text{v}) \notin \mathcal{F}_{\text{forbidden}}$, gdzie zbiór $\mathcal{F}_{\text{forbidden}}$ obejmuje instrukcje modyfikacji danych i schematu (`Insert`, `Update`, `Delete` i inne; pełna lista w podrozdziale 5.6). Reguła ta uniemożliwia ukrycie mutacji wewnątrz klauzul CTE (np. `WITH d AS (DELETE ...) SELECT * ...`);

4. **Enkapsulacja rekordów przeciw wstrzykiwaniu drugiego rzędu:** Dane wynikowe są oczyszczane ze znaków sterujących XML i tokenów specjalnych LLM, a następnie zamykane w bloku `<untrusted_db_record role="data_only" execution_forbidden="true">`, co zapobiega interpretacji danych jako poleceń sterujących modelu w kolejnych iteracjach wnioskowania.

4.5.3 Model izolacji współbieżnej pętli zdarzeń

W systemach wieloagentowych synchroniczne parsowanie AST w jednowątkowej pętli `asyncio` blokuje obsługę ramek I/O i prowadzi do narostu opóźnień. Dlatego pętla deleguje parsowanie do puli procesów `ProcessPoolExecutor` powoływanej z kontekstem `forkserver`:

$$\text{Future}_{\text{validation}} = \text{loop.run_in_executor}(\text{Pool}_{\text{workers}}, \text{ValidateAST}, \text{Q}) \tag{24}$$

Kontekst `forkserver` (zamiast `fork` lub `spawn`) pozwala uniknąć zakleszczeń przy bibliotekach w języku C w procesie wielowątkowym oraz narzutu zimnego startu interpretera, a przekazanie zadania do kolejki IPC redukuje narzut pętli do stałego czasu przełączenia kontekstu $\mathcal{O}(1)$, umożliwiając równoległe wykorzystanie rdzeni procesora.

Rozdział 5

Implementacja i środowisko badawcze

Rozdział dokumentuje implementację referencyjnej bramy pośredniczącej MCP [2] (Python 3.12, FastMCP [33], PostgreSQL 16 z rozszerzeniem `pgvector` [22], Apache Kafka [26] w trybie KRaft) oraz aparaturę badawczą, odpowiadając na pytanie, jak fizycznie skonstruowano system i środowisko eksperymentalne. Modele formalne opisano w Rozdziale 4, a wyniki pomiarów w Rozdziale 6.

5.1 Architektura środowiska uruchomieniowego

Repozytorium zawiera dwie konfiguracje środowiska: deweloperską, opartą na kontenerach Docker i Docker Compose (tabela 5), służącą do lokalnych testów, oraz natywną, w której wykonano rundę pomiarową (tabela 6). Topologia środowiska obejmuje węzeł relacyjno-wektorowy PostgreSQL 16 z rozszerzeniem `pgvector`, rozproszony broker Apache Kafka działający w trybie KRaft, współdzielony segment pamięci operacyjnej (`/dev/shm`) dedykowany dla transferu obiektów binarnych poza pasmem oraz autorską bramę pośredniczącą FastMCP. Zestawienie składowych infrastruktury oraz ich kluczowe parametry konfiguracyjne podsumowano w tabeli 5.

W konfiguracji deweloperskiej węzeł bazodanowy (kontener `mcp_postgres`) oparto na obrazie `pgvector:pg16` i skonfigurowano parametry silnika:

- `shared_buffers=1GB`: bufor stron relacyjnych i indeksowych, 25% pamięci RAM węzła (4 GB), co ogranicza podwójne buforowanie z pamięcią podręczną jądra;
- `work_mem=32MB`: pamięć na sortowania i tablice haszujące, pozwalająca wykonywać sortowania kursorowe w RAM bez zrzutu na dysk;
- `max_connections=200`: limit połączeń uwzględniający równoległe obciążanie bramy przez harness pomiarowy;
- `pgdata`: trwały wolumen blokowy izolujący operacje I/O bazy od warstwy nakładkowej kontenera.

Broker Apache Kafka (kontener `mcp_kafka`) działa w trybie KRaft, z tematem `telemetry_test` o trzech partycjach (parametry w tabeli 5). Konfiguracja kontenerowa służy do lokalnego uruchamiania bramy i testów; runda pomiarowa, której wyniki przedstawia Rozdział 6, została wykonana

Tabela 5. Parametry konfiguracji deweloperskiej (kontenery Docker; nieużywana w rundzie pomiarowej)

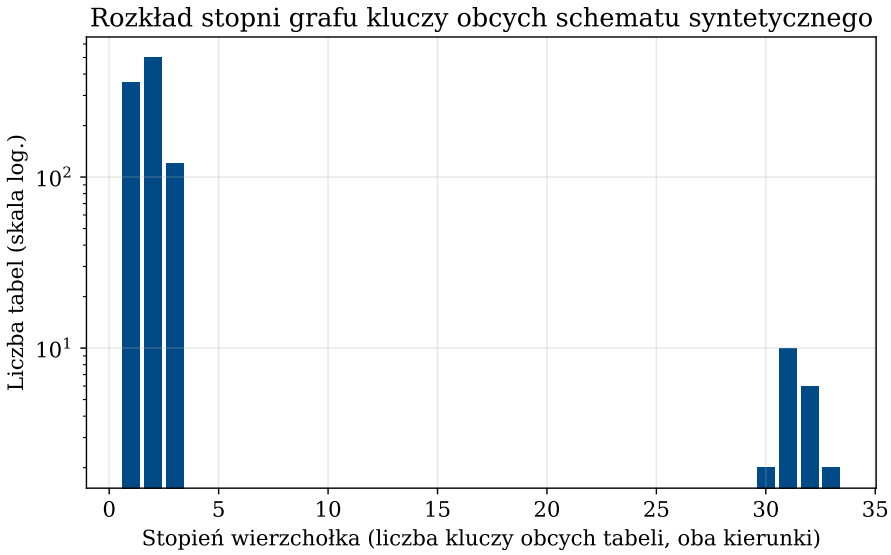
Usługa kontenerowa	**Obraz bazowy**	**Kluczowe parametry konfiguracyjne**
PostgreSQL 16	`pgvector/pgvector:pg16`	`shared_buffers=1GB`, `work_mem=32MB`, `max_connections=200`, port 5432, rozszerzenie `vector` (HNSW, $\mathrm{d} = 384$, `vector_cosine_ops`)
Apache Kafka	`apache/kafka:latest`	Tryb KRaft (`broker,controller`), 3 partycje fizyczne, quorum voter `localhost:9093`, listener `PLAINTEXT://:9092`
Segment /dev/shm	Pamięć RAM hosta	POSIX Shared Memory: przekazanie danych binarnych poza ramką JSON-RPC, bez kodowania Base64 (jednokrotne kopiowanie do segmentu)
Inicjalizacja danych	Python 3.12	`generate_enterprise_schema.py` (1000 tabel, 20 tabel centralnych), `seed_1m_records.py` (1 mln rekordów)

Tabela 6. Środowisko rundy pomiarowej (odczytane automatycznie i zapisane w manifeście rundy, plik `environment.json`)

Element	**Wartość**
Sprzęt	NVIDIA A100-SXM4-80GB (80 GiB), AMD EPYC 7742 64-Core Processor (przydział: 16 vCPU, 250 GB RAM)
System i oprogramowanie	Linux 6.8.0-100-generic; Python 3.12.3; uruchomienie natywne, bez kontenerów
PostgreSQL	16.4 z pgvector 0.7.4; `shared_buffers` = 2,5 GiB, `work_mem` = 64 MiB, `max_connections` = 100, `jit` = on, `max_parallel_workers_per_gather` = 2, `statement_timeout` = 10000 ms
Pozostałe usługi	Apache Kafka 3.7.0; Ollama 0.35.0 (modele `qwen2.5-coder:3b` i `qwen2.5-coder:7b`)

na instancji chmurowej bez kontenerów, z usługami uruchomionymi natywnie i parametrami odczytanymi automatycznie ze środowiska (podrozdział 6.1.2).

W celu weryfikacji algorytmów selekcji schematu na dużym schemacie zaimplementowano generator `testbed/init-scripts/generate_enterprise_schema.py`, powoływany automatycznie przy starcie bazy. Generator tworzy syntetyczny magazyn danych złożony z 1000 relacji SQL podzielonych równomiernie na 5 domen biznesowych: finanse (`finance`), gospodarka magazynowa (`inventory`), kadry i płace (`hr`), zarządzanie relacjami z klientami (`crm`) oraz telemetria przemysłowa (`telemetry`) – po 200 tabel w każdej domenie (20 encji bazowych powiązanych z 10 kwalifikatorami operacyjnymi). Generator (deterministyczny, ziarno 42) wyznacza 20 centralnych tabel typu Hub (po 4 w domenie, m.in. `finance_ledger_entries_daily`, `crm_client_accounts_enterprise`, `inventory_purchase_orders_raw_materials`, `telemetry_host_servers_datacenter_a`). Rozkład stopni uzyskanego grafu, odtworzony generatorem bez bazy (analiza wtórna, skrypt `o6_stopnie_grafu.py`; liczba węzłów i krawędzi zgodna z logiem rundy), przedstawia rysunek 2: 20 tabel centralnych ma stopień 30–33, a pozostałe 980 tabel stopień 1–3 (86,0% tabel ma stopień nie większy niż 2). Graf nie ma więc rozkładu potęgowego właściwego sieciom bezskalowym, lecz strukturę gwiaździstą z wyraźnymi węzłami centralnymi. Strukturę uzupełniają tabele mostkujące relacje typu $\mathrm{N} : \mathrm{M}$ (36 na domenę), tabele satelickie pierwszego poziomu (72 na domenę) oraz łańcuchy satelickie 2-i 3-poziomowe (88 na domenę).



Rysunek 2. Rozkład stopni wierzchołków grafu kluczy obcych schematu syntetycznego (1000 tabel, 1174 klucze obce; analiza wtórna, generator deterministyczny)

Metadane relacyjne grafu bazy utrwalono w dedykowanej tabeli systemowej `public.schema_foreign_keys`, przechowującej 1174 zdefiniowane klucze obce wraz z powiązanymi kolumnami złączeń w formacie `JSONB` oraz indeksami B-drzewa na kolumnach `source_table` i `target_table`. Semantyczne opisy tabel umieszczono w relacji `public.schema_embeddings`,

w której kolumna embedding przechowuje gęste reprezentacje wektorowe o wymiarowości $\mathrm{d} = 384$, wygenerowane modelem sentence-transformers/all-MiniLM-L6-v2 [49]. Na kolumnie tej utworzono wielowarstwowy indeks grafowy HNSW [35] z operatorem odległości kosinusowej, gwarantujący logarytmiczną złożoność wyszukiwania najbliższych sąsiadów $\mathcal{O}(\log \mathrm{N})$.

Dla potrzeb ewaluacji paginacji kursorowej i transferu obiektów binarnych przygotowano relację public.analytics_orders, zasilaną przez skrypt seed_1m_records.py wolumenem 1 000 000 rekordów przy użyciu binarnego strumieniowania copy_records_to_table w paczkach po 25 000 wierszy. Tabela zawiera kolumny zróżnicowanych typów oraz kolumnę document_base64 o ładunkach 10–30 KB, wymuszającą w silniku PostgreSQL pozawierszowe przechowywanie atrybutów w strukturach TOAST (*The Oversized-Attribute Storage Technique*). Do obsługi transferu OOB skonfigurowano współdzielony segment pamięci operacyjnej hosta podmontowany pod ścieżką /dev/shm.

5.2 Implementacja autorskiego serwera MCP (Middleware Gateway)

Podstawowym komponentem warstwy pośredniczącej jest autorski serwer protokołu Model Context Protocol, zaimplementowany w module src/mcp_server/server.py. Do budowy serwera wykorzystano framework FastMCP [33] w środowisku Python 3.12, w pełni zintegrowany z natywną pętlą zdarzeń asyncio. Komunikację z silnikiem PostgreSQL oparto na binarnym sterowniku asyncpg [34], który realizuje protokół Frontend/Backend v3.0, omijając narzuty synchronicznego interfejsu DB-API 2.0 oraz biblioteki C libpq.

Zarządzanie połączeniami zrealizowano za pomocą asynchronicznej puli połączeń asyncpg.Pool, konfigurowanej w klasie DatabaseConfig (min_size=2, max_size=10), wyposażonej w procedurę rezerwowego uwierzytelniania środowiskowego. Cykl życia serwera kontrolowany jest przez asynchroniczny menedżer kontekstu lifespan. Odpowiada on za deterministyczne utworzenie puli połączeń bazodanowych, uruchomienie asynchronicznego konsumenta Kafka w tle oraz powołanie wieloprocesowej puli wykonawczej ProcessPoolExecutor z bezpiecznym kontekstem forkserver dla analizatora zapytań SQL. Podczas procedury zatrzymania menedżer gwarantuje sekwencyjne zatrzymanie konsumenta strumieniowego, zamknięcie procesów roboczych puli oraz zwolnienie wszystkich aktywnych połączeń bazodanowych.

Publiczny interfejs serwera (src/mcp_server/server.py) realizuje kontrakt z tabeli 4. Narzędzie execute_read_only_query wykonuje walidację AST w SqlSanitizer, wykonuje zapytanie i hermetyzuje rekordy w bloku <untrusted_db_record>; narzędzie execute_query_paginated realizuje paginację keyset (pagination.py) z kontrolą indeksów (IndexCatalogGuard), opcjonalną migawką MVCC (TransactionSnapshotManager) i estymatorem Welforda; narzędzie search_schema wywołuje SchemaEngine.discover_schema_with_graph_pruning; narzędzie retrieve_out_of_band_payload odczytuje ładunek z segmentu /dev/shm lub z pliku magazynu obiektów i weryfikuje skrót SHA-256. Anomalie telemetryczne odebrane z Kafki przez konsumenta aiokafka są udostępniane jako zasoby telemetry://latest_anomaly

i `telemetry://live_stream` oraz narzędzie `get_latest_telemetry_anomalies`, czyli odczytywane przez klienta na żądanie.

Framework FastMCP generuje z adnotacji typów języka Python schemat JSON Schema parametrów każdego narzędzia i waliduje argumenty wywołania przed uruchomieniem kodu narzędzia.

5.3 Silnik semantycznego odkrywania schematu

Implementację mechanizmu progresywnego odkrywania schematów relacyjnych w architekturze JIT zrealizowano w module `src/mcp_server/schema_engine.py` (klasa `SchemaEngine`), stanowiącym implementację założeń projektowych z podrozdziału 4.3.

Klasa `SchemaEngine` integruje cztery strategie wyszukiwania tabel, których założenia formalne opisano w podrozdziale 4.3: leksykalną BM25 (`retrieve_bm25`, biblioteka `rank_bm25`, indeks budowany w pamięci RAM z nazw tabel, kolumn z `information_schema.columns` i komentarzy biznesowych), gęstą (`retrieve_dense`, model `all-MiniLM-L6-v2`, $\mathrm{d} = 384$, operator odległości kosinusowej <=> na indeksie HNSW tabeli `public.schema_embeddings` obsługiwany przez `asyncpg`), hybrydową fuzję RRF (`retrieve_hybrid_rrf`, $\mathrm{k_{RRF}} = 60$, równanie (16)) oraz ekspansję grafową o tabele mostkujące (`discover_schema_with_graph_pruning`), operujące na grafie NetworkX (`nx.DiGraph`) zbudowanym z metadanych tabeli `public.schema_foreign_keys`.

Kod funkcji `discover_schema_with_graph_pruning`, wykrywającej tabele mostkujące na ścieżkach o długości $\mathrm{dist} \leq 2$ w grafie, zamieszczono na listingu 2 w załączniku 2.

Wygenerowany zwarty blok DDL zawiera definicje kolumn, typów, kluczy głównych oraz explicite wyspecyfikowane klauzule relacji (`FOREIGN KEY ... REFERENCES`), co dostarcza modelowi językowemu niezbędny i nieskażony szumem kontekst relacyjny.

5.4 Mechanizm adaptacyjnej paginacji kursorowej

Paginację kursorową zaprojektowaną w podrozdziale 4.2 (predykat keyset, kursor HMAC, migawka MVCC, estymator Welforda) zaimplementowano w module `src/mcp_server/pagination.py`, w funkcjach `build_keyset_query` i `fetch_paginated_data`. Rozbicie predykatu DNF na gałęzie `UNION ALL` włącza parametr `use_union_all` (domyślnie aktywny w `fetch_paginated_data`). Moduł `IndexCatalogGuard` odpytuje widoki katalogowe (`pg_indexes`, `pg_am`, `pg_class`, `pg_attribute`) z 60-sekundowym buforem TTL, sprawdzając, czy kolumny sortowania tworzą lewostronny prefiks indeksu B-drzewa; przy jego braku dławi wielkość paczki do 50 wierszy, chroniąc `shared_buffers` i `work_mem` przed skanem sekwencyjnym. Kod procedury zamieszczono na listingu 3 w załączniku 2.

5.5 Transfer danych binarnych Out-of-Band i asynchroniczne strumieniowanie

Fizyczną realizację mechanizmów transferu obiektów wielkich poza pasmem oraz asynchronicznego strumieniowania zdarzeń, sformalizowanych w podrozdziale 4.4, wdrożono w modułach `src/mcp_server/out_of_band.py` oraz `src/mcp_server/streaming.py`.

Dyspozytor `DataTransferDispatcher` obsługuje tryby z enumeracji `TransferMode`: `IN_BAND` (Base64 w ładunku JSON-RPC dla ładunków poniżej progu `threshold_bytes = 64 KB`, stanowiącego zachowawczy próg startowy podlegający rekonfiguracji do progu zmierzonego dla danego środowiska; na jednym węźle wyniósł on 12,9 KiB, podrozdział 6.3), `OOB_POSIX_SHM` (nazwany segment `multiprocessing.shared_memory.SharedMemory` w `/dev/shm`, skrót SHA-256 i deskryptor JSON; aktywne segmenty są rejestrowane w `_ACTIVE_SHM_SEGMENTS` i zwalniane procedurą `atexit`), `OOB_HTTP_STREAM` (jednorazowy adres URL z podpisem HMAC-SHA256 i czasem ważności `expires_at`) oraz `AUTO` (kierowanie ładunków powyżej progu S^* do pamięci współdzielonej). Stały narzut przygotowania deskryptora δ_{OOB} jest mierzony zegarem `time.perf_counter_ns()` i dołączany do deskryptora.

W module `src/mcp_server/streaming.py` zrealizowano asynchronicznego konsumenta komunikatów telemetrycznych w klasie `KafkaTelemetryConsumer`. Moduł bazuje na bibliotece `aiokafka` i realizuje ciągły nasłuch tematu `telemetry_test` w asynchronicznej pętli `_consume_loop`. Odebranie komunikatu o statusie `CRITICAL_ANOMALY` skutkuje natychmiastowym zapisem zdarzenia w pamięci podręcznej (`deque(maxlen=100)`) oraz asynchronicznym wywołaniem zarejestrowanych funkcji zwrotnych subskrybentów SSE. Zastosowanie modelu sterowanego zdarzeniami eliminuje konieczność aktywnego odpytywania bazy przez agenta.

5.6 Moduł bezpieczeństwa zapytań (SQL Sanitizer)

Perymetr bezpieczeństwa zaprojektowany w podrozdziale 4.5 wdrożono w module `src/mcp_server/security.py` (klasa `SqlSanitizer`) w oparciu o bibliotekę analizy składniowej `sqlglot` [36] i dialekt `postgres`. Ponieważ budowa i trawersowanie drzewa AST silnie obciążają procesor, walidację delegowano do puli `concurrent.futures.ProcessPoolExecutor` (funkcje `get_process_pool` i `async_validate_query`) powoływanej z kontekstem `forkserver`. Standardowy `fork` w procesie wielowątkowym z bibliotekami C (`sqlglot`, sterowniki binarne) grozi zakleszczeniem procesów potomnych na zduplikowanych muteksach, a `spawn` generuje narzut zimnego startu interpretera; `forkserver` tworzy procesy robocze z jednowątkowego procesu zarządczego, izolując pamięciowo pętlę `asyncio`.

Reguły walidacji z podrozdziału 4.5.2 zrealizowano w funkcjach `prevent_multi_statements`, `validate_read_only`, `enforce_limit` (dopisanie lub redukcja klauzuli `LIMIT` do wartości maksymalnej, ochrona przed atakami Denial of Service) oraz `encapsulate_untrusted_database_records`. Zbiór `FORBIDDEN_AST_NODES`, przeszukiwany rekurencyjnie w całym drzewie (także w klauzulach

CTE), obejmuje węzły: `Insert`, `Update`, `Delete`, `Merge`, `Drop`, `Create`, `Alter`, `TruncateTable`, `Grant`, `Revoke`, `Copy`, `Command`, `Execute`, `Pragma`, `Into`, `Analyze`, `Set`, `Transaction`, `Commit` i `Rollback`. Rekordy zwracane z bazy są oczyszczane ze znaków specjalnych XML i tokenów sterujących LLM (`<|im_start|>`, `<|im_end|>`, `[INST]`, `<<SYS>>`), a następnie hermetyzowane w bloku `<untrusted_db_record role="data_only" execution_forbidden="true">`, co ogranicza ryzyko ataków *Second-Order Prompt Injection*.

Asynchroniczną delegację walidacji do puli `forkserver` przedstawiono na listingu 4 w załączniku 2.

Deterministyczna weryfikacja poprawności w pakiecie testów jednostkowych. Komponenty logiczne bramy – estymator Welforda, mechanizm `IndexCatalogGuard`, kryptograficzny kursor HMAC oraz moduł migawek transakcyjnych MVCC – stanowią integralne elementy architektury bramy. Ich niezmienniki algorytmiczne, deterministyczną poprawność oraz zachowanie w stanach brzegowych zweryfikowano w dedykowanym pakiecie testów jednostkowych (`tests/test_mechanizmy_bez_ewaluacji.py`, `tests/test_keyset_nulls.py`). Testy te potwierdziły: odrzucenie kursora o naruszonej integralności lub niepoprawnym podpisie HMAC, bezpieczne ograniczenie rozmiaru paczki do 50 wierszy i odrzucenie zapytania w trybie ścisłym przy braku optymalnego indeksu B-drzewa, ścisłą zgodność estymacji wariancji i średniej algorytmem Welforda ze statystykami próbkowymi, a także poprawność generowania predykatu keyset w obecności wartości `NULL`.

5.7 Klient agentowy oraz zautomatyzowana aparatura pomiarowa

W celu przeprowadzenia powtarzalnych eksperymentów badawczych zaimplementowano klienta agentowego oraz modułowy runner pomiarowy. Klient agentowy oparto na paradygmacie ReAct [60], zrealizowanym w klasie `ReActAgentLoop` (moduł `src/agent_client/agent_loop.py`). Pętla wykonawcza koordynuje sekwencję Myśl, Akcja oraz Obserwacja. Komunikację z serwerem FastMCP zaimplementowano w klasach `StdioMcpClient` (anonimowe potoki nienazwane `stdin`/`stdout`) oraz `SseMcpClient` (asynchroniczny transport HTTP/SSE).

W aparaturze pomiarowej zdefiniowano dwa profile modeli językowych, obsługiwane przez serwer Ollama na akceleratorze GPU:

- **Profil 3B:** model `qwen2.5-coder:3b`;
- **Profil 7B:** model `qwen2.5-coder:7b`.

Oba profile używają temperatury $\mathrm{T} = 0$, stałego ziarna, limitu 1024 tokenów wyjściowych i okna kontekstu `num_ctx` $= 8\,192$; liczby tokenów promptu i odpowiedzi oraz powód zakończenia generacji są odczytywane z liczników serwera Ollama, co pozwala wykryć obcięcie kontekstu lub odpowiedzi. Profile różnią się wyłącznie modelem.

Fundament aparatury pomiarowej stanowi klasa `BenchmarkHarness` zlokalizowana w module `benchmarks/harness.py`. Konstrukcja runnera pomiarowego opiera się na następujących zasadach metodologicznych:

- **Determinizm środowiska bazodanowego:** Przed każdym zapytaniem pomiarowym sesja bazy danych jest resetowana poleceniem `DISCARD ALL`, które czyści pamięć podręczną planów zapytań, blokady oraz ustawienia sesji. Wszystkie generatory liczb losowych (zapytania, wektory, bootstrap, model) używają stałych ziaren zapisanych w pliku konfiguracji rundy;
- **Biegi rozgrzewkowe:** Punkty pomiarowe poprzedzane są nieewaluowanymi przebiegami rozgrzewkowymi, których liczbę określa konfiguracja scenariusza (np. 2 dla paginacji, 5 dla transferu OOB, 5 s dla każdego poziomu obciążenia współbieżnego);
- **Sztywny limit czasu zapytań:** W konfiguracji puli połączeń `asyncpg` wymuszono limit `statement_timeout = '10000ms'`, zapobiegający blokowaniu aparatury przez kwerendy zawieszające transakcje pod obciążeniem;
- **Wysokorozdzielcza aparatura próbkowania czasu:** Pomiar czasu wykonania operacji realizowany jest nanosekundowym zegarem monotonicznym `time.perf_counter_ns()`;
- **Asynchroniczny próbnik opóźnienia pętli zdarzeń (`EventLoopLagProbe`):** W scenariuszach współbieżnych uruchamiana jest sonda w tle, która co 10 ms próbuje wybudzić się z uśpienia (`asyncio.sleep(0.010)`). Różnica pomiędzy czasem rzeczywistym a oczekiwanym wyznacza bezpośrednio opóźnienie harmonogramu (*timer drift*) wywołane blokowaniem pętli zdarzeń;
- **Deterministyczna tokenometria:** Zliczanie tokenów generowanych w ładunkach realizowane jest biblioteką `tiktoken` (enkoder `cl100k_base`);
- **Profilowanie zasobów systemowych:** Pamięć mierzona jest jako suma PSS (*Proportional Set Size*) procesu serwera i wszystkich jego procesów potomnych za pomocą biblioteki `psutil`, co pozwala uwzględnić koszt puli procesów bez wielokrotnego liczenia stron współdzielonych;
- **Ustrukturyzowany zapis danych surowych:** Wszystkie zarejestrowane próbki czasowe, metadane i wskaźniki są serializowane do kolumnowych plików Apache Parquet oraz tabelarycznych CSV w katalogu wyników rundy, a manifest rundy zapisuje wersję kodu, konfigurację, ziarna, opis środowiska, skróty wag modeli oraz sumy kontrolne SHA-256 wszystkich plików wynikowych.

Zestaw scenariuszy badawczych uruchamianych w jednej rundzie pomiarowej obejmuje m.in. scenariusze wymienione w tabeli 7. Ich charakterystykę inżynierską, badane mechanizmy, wielkości prób oraz aparaturę wnioskowania statystycznego zestawiono w tabeli 7.

Zaprojektowane środowisko testowe, zunifikowana brama pośrednicząca FastMCP oraz zautomatyzowana aparatura pomiarowa posłużyły do przeprowadzenia pełnego pakietu ewaluacji empirycznej, której szczegółowe wyniki ilościowe, analiza statystyczna oraz interpretacja efektywnościowa stanowią treść Rozdziału 6.

Tabela 7. Zestawienie scenariuszy badawczych, aparatury pomiarowej oraz metod weryfikacji statystycznej

Skrypt / Scenariusz	Badany mechanizm i cel	Wielkość próby	Rejestrowane metryki surowe	Weryfikacja statystyczna
`bench_schema_quality.py`	Selekcja schematu, ekspansja grafowa o tabele mostkujące, Text-to-SQL na dwóch profilach modeli	$\mathrm{N} = 500$ zapytań (150 prostych, 250 średnich, 100 złożonych)	Recall i Precision (tabele i kolumny), Valid SQL, Execution Accuracy, liczniki tokenów, kategorie błędów	Test Wilcoxona, dokładny test McNemara, 95% CI Cloppera–Pearsona
`bench_ttft_scaling.py`	Czas do pierwszego tokenu w funkcji rozmiaru promptu	6 rozmiarów promptu $\times$ 30 powtórzeń $\times$ 2 profile	TTFT, czas całkowity, `prompt_eval_count`	Regresja liniowa, bootstrap 95% CI
`bench_pagination_advanced.py`	Paginacja keyset, OFFSET i bez indeksu; postać predykatu	10 offsetów $\times$ 10 powtórzeń	Opóźnienie strony, plany `EXPLAIN`, spójność sekwencji	Regresja liniowa, mediana i zakres
`bench_oob_crossover.py`	Dyspozytor danych OOB, punkt przecięcia $\mathrm{S}^*(\mathrm{RTT})$ dla In-Band, SHM i HTTP	9 rozmiarów 512 B – 50 MiB $\times$ 50 powtórzeń, 3 profile sieci	Czas transferu End-to-End i jego składowe, objętość ramki, integralność SHA-256	Test Manna–Whitneya, bootstrap 95% CI progu S^*
`bench_schema_drift.py`	Odporność na dynamiczny dryf schematu podczas współbieżnych migracji DDL	50 nowych relacji, 50 kluczy FK, kwerendy w toku	Czasy składowych przeładowania metadanych, opóźnienie i błędy zapytań w trakcie dryfu	Analiza percentyli
`bench_concurrency.py`	Wydajność i skalowalność pętli zdarzeń pod obciążeniem (SYNC, THREAD, POOL)	$\mathrm{M} \leq 500$ ($\mathrm{M} \in \{1, 5, 10, 25, 50, 100, 500\}$)	Przepustowość (RPS), opóźnienie odpowiedzi, opóźnienie pętli zdarzeń, suma PSS drzewa procesów	Test Manna–Whitneya, bootstrap 95% CI różnicy median, percentyle
`bench_security_suite.py`	Odporność zapory AST i guardrails na wektory SQLi oraz Prompt Injection	77 unikalnych wektorów	FPR, AMR, odsetek przepuszczonych `SELECT` z efektami ubocznymi, odsetek rekordów z pozostałymi tokenami sterującymi, odpowiedzi modelu na wektory wstrzykiwania	95% CI Cloppera–Pearsona, dokładny test McNemara
`statistical_analysis.py`	Formalna weryfikacja hipotez badawczych z surowych wektorów pomiarowych	Zbiorcze wektory z plików Parquet	Statystyki testowe, wartości p, wielkość efektu r, przedziały ufności	Testy Wilcoxona, McNemara i Manna–Whitneya, bootstrap

Rozdział 6

Wyniki badań i ewaluacja eksperymentalna

Rozdział przedstawia wyniki ewaluacji eksperymentalnej bramy MCP [2] zaprojektowanej w Rozdziale 4 i zaimplementowanej w Rozdziale 5. Wszystkie liczby pochodzą z jednej pełnej rundy pomiarowej (identyfikator 20261004T120209_d347d2); każda wartość w tekście, tabelach i na rysunkach została wyznaczona skryptem z plików wynikowych tej rundy.

6.1 Metodyka pomiarowa i formalizacja aparatu ewaluacyjnego

6.1.1 Formalizacja metryk oceny

Względna redukcja objętości kontekstu ($\Delta\mathrm{T}$)

Efektywność zarządzania oknem kontekstowym mierzono względną redukcją liczby tokenów bloku schematu wstrzykiwanego do promptu:

$$\Delta\mathrm{T} = \frac{\mathrm{T}_{\mathsf{baseline}} - \mathrm{T}_{\mathsf{MCP}}}{\mathrm{T}_{\mathsf{baseline}}} \times 100\% \tag{25}$$

gdzie $\mathrm{T}_{\mathsf{baseline}}$ jest liczbą tokenów pełnego zrzutu DDL wszystkich tabel hurtowni, a $\mathrm{T}_{\mathsf{MCP}}$ średnią liczbą tokenów bloku DDL zwracanego przez bramę; obie wielkości liczono enkoderem `cl100k_base` dla tego samego formatu DDL.

Opóźnienie wnioskowania (TTFT oraz $\mathrm{L}_{\mathsf{E2E}}$)

$$\mathrm{TTFT} = \mathrm{t}_{\mathsf{first_token}} - \mathrm{t}_{\mathsf{request_sent}} \tag{26}$$

$$\mathrm{L}_{\mathsf{E2E}} = \mathrm{TTFT} + \sum_{\mathrm{k}=2}^{\mathrm{M}} \Delta\mathrm{t}_{\mathrm{k}} \tag{27}$$

gdzie TTFT (ang. *Time-to-First-Token*) to czas od wysłania żądania do serwera Ollama do odebrania pierwszego fragmentu odpowiedzi strumieniowej, $\mathrm{L_{E2E}}$ to całkowity czas żądania, M to liczba wygenerowanych tokenów, a $\Delta \mathrm{t_k}$ to czas generacji k-tego tokenu.

Wskaźniki jakości selekcji schematu i Text-to-SQL

$$\text{Table-Recall@k} = \frac{|\mathrm{T}_{\text{relevant}} \cap \mathrm{T}_{\text{retrieved@k}}|}{|\mathrm{T}_{\text{relevant}}|} \tag{28}$$

gdzie $\mathrm{T}_{\text{relevant}}$ to zbiór tabel zapytania wzorcowego (*Ground Truth*, GT), a $\mathrm{T}_{\text{retrieved@k}}$ to zbiór tabel zwrócony przez strategię selekcji przy parametrze k. Analogicznie zdefiniowano Table-Precision@k oraz Column-Recall względem kolumn użytych w zapytaniu wzorcowym. Listy tabel i kolumn wzorcowych zapisano dla każdego zapytania w pliku zbioru danych.

Valid SQL oznacza, że zapytanie wygenerowane przez model zostało zaakceptowane przez PostgreSQL w poleceniu `EXPLAIN` i wykonane bez błędu w limicie `statement_timeout`; odwołanie do nieistniejącej kolumny daje więc Valid SQL $=$ fałsz. *Execution Accuracy* (EX) jest prawdziwe wtedy i tylko wtedy, gdy Valid SQL jest prawdziwe, zapytanie wzorcowe zwróciło co najmniej jeden wiersz, a wartości wierszy obu wyników są równe jako multizbiory; jeśli zapytanie wzorcowe zawiera klauzulę `ORDER BY` na najwyższym poziomie, porównywana jest lista wierszy w kolejności. Nazwy kolumn nie są porównywane.

Taksonomia błędów Text-to-SQL

Każdy rekord z EX $=$ fałsz otrzymuje dokładnie jedną kategorię, przypisywaną w podanej kolejności reguł:

1. `SEMANTIC_MISMATCH` – zapytanie jest Valid SQL, lecz jego wynik różni się od wzorca;
2. `TRUNCATION` – generacja zakończyła się na limicie tokenów wyjściowych (`done_reason = length`);
3. `TABLE_MISSING` – zapytanie odwołuje się do tabeli nieistniejącej lub nieobecnej w dostarczonym wycinku DDL;
4. `COLUMN_HALLUCINATION` – PostgreSQL zgłosił odwołanie do nieistniejącej kolumny;
5. `JOIN_CONDITION_MISMATCH` – niejednoznaczna lub brakująca referencja w klauzuli `FROM` albo złączenie pary tabel niepowiązanych kluczem obcym;
6. `SYNTAX_ERROR` – pozostałe błędy wykonania, w tym błędy składni.

Suma kategorii jest równa liczbie rekordów z EX $=$ fałsz, co sprawdza skrypt walidacyjny rundy. Implementacja ma ponadto kategorię `GT_EMPTY` dla nieważnych próbek, w których zapytanie wzorcowe zwróciło pusty wynik; w rundzie nie wystąpiła, ponieważ kontrola wstępna wykluczyła puste tabele wzorcowe.

Wskaźniki perymetru bezpieczeństwa

$$\mathrm{AMR} = \frac{\mathrm{N}_{\mathrm{blocked_malicious}}}{\mathrm{N}_{\mathrm{total_malicious}}} \times 100\% \tag{29}$$

$$\mathrm{FPR} = \frac{\mathrm{N}_{\mathrm{blocked_benign}}}{\mathrm{N}_{\mathrm{total_benign}}} \times 100\% \tag{30}$$

$$\mathrm{R}_{\mathrm{res}} = \frac{\mathrm{N}_{\mathrm{records_with_control_token}}}{\mathrm{N}_{\mathrm{records}}} \times 100\% \tag{31}$$

gdzie AMR (ang. *Attack Mitigation Rate*) to odsetek zablokowanych wektorów SQL Injection, FPR (ang. *False Positive Rate*) to odsetek odrzuconych poprawnych zapytań analitycznych, a $\mathrm{R}_{\mathrm{res}}$ to odsetek rekordów z pozostałymi tokenami sterującymi, czyli rekordów zawierających wektor wstrzykiwania promptu, w których po enkapsulacji XML pozostał surowy token sterujący modelu (np. <|im_start|>). Wskaźnik $\mathrm{R}_{\mathrm{res}}$ opisuje wyłącznie działanie funkcji oczyszczającej i nie wymaga modelu językowego; podatność samego modelu badano osobno (podrozdział 6.5). Dodatkowo raportowano odsetek przepuszczonych zapytań `SELECT` z efektami ubocznymi (*side-effect SELECT*), opisujący znane ograniczenie walidacji opartej na typie węzła głównego.

6.1.2 Środowisko pomiarowe i protokół eksperymentu

Rundę wykonano na instancji chmurowej RunPod z akceleratorem NVIDIA A100-SXM4-80GB (80 GB pamięci), procesorem AMD EPYC 7742 64-Core Processor (przydział instancji: 16 vCPU, 250 GB RAM), pod kontrolą jądra Linux 6.8.0-100-generic. Usługi uruchomiono natywnie, bez kontenerów Docker: PostgreSQL 16.4 z rozszerzeniem pgvector 0.7.4, Apache Kafka 3.7.0, serwer modeli Ollama 0.35.0 oraz Python 3.12.3.

Protokół eksperymentu:

- **Zbiór zapytań Text-to-SQL:** $\mathrm{N} = 500$ zapytań wygenerowanych deterministycznie (ziarno 2026) na schemacie 1000 tabel powiązanych 1174 kluczami obcymi; każda tabela zawiera 200 wierszy, a przed przebiegiem sprawdzono, że żadna tabela wzorcowa nie jest pusta;
- **Modele językowe:** dwa profile, „3B" (`qwen2.5-coder:3b`) i „7B" (`qwen2.5-coder:7b`), oba na GPU, z temperaturą $\mathrm{T} = 0$, ziarnem 42, limitem 1024 tokenów wyjściowych i oknem kontekstu `num_ctx` $= 8\,192$;
- **Izolacja sesji PostgreSQL:** przed każdym zapytaniem pomiarowym sesja była resetowana poleceniem `DISCARD ALL`; limit `statement_timeout` wynosił 10000 ms;
- **Przebiegi rozgrzewkowe:** 2 przebiegi przed każdym punktem paginacji, 5 przed każdym punktem transferu OOB oraz 5 s rozgrzewki przed każdym poziomem obciążenia współbieżnego;
- **Próbkowanie pętli zdarzeń:** sonda opóźnienia pętli `asyncio` z interwałem 10 ms, mierzona zegarem `time.perf_counter_ns()`;
- **Sieć:** profile LAN i WAN w Scenariuszu B1 zdefiniowano jako analityczną projekcję parametryczną, skalibrowaną empirycznie na kosztach obliczeniowych pojedynczego węzła; szczegóły modelu opisano w podrozdziale 6.3.

- **Konto bazodanowe:** Aparatura pomiarowa oraz serwer pośredniczący łączyły się z silnikiem PostgreSQL za pośrednictwem dedykowanego konta technicznego `mcp_user`, wykorzystywanego do automatyzacji procedur pomiarowych, migracji DDL oraz czyszczenia stanu sesji poleceniem `DISCARD ALL`.

Zakres pomiarów. Przyjęto metodykę izolowanych testów mikrowydajnościowych rdzeniowych komponentów bramy, ukierunkowaną na analizę złożoności algorytmicznej i narzutu obliczeniowego nowych mechanizmów: silnika selekcji, dekompozycji keyset, wieloprocesowego AST oraz dyspozytora OOB. Zmienność warstwy transportowej (gniazda sieciowe, potoki `stdio` i serializacja JSON-RPC) wyłączono z tej fazy, ponieważ jej charakterystyka wynika ze standardowych bibliotek systemowych, a celem była bezstronna ocena komponentów wewnętrznych bramy pośredniczącej. Ścieżki wywołań są następujące: Scenariusz A – `SchemaEngine` (selekcja schematu) oraz bezpośrednie żądania HTTP do serwera Ollama (`/api/generate`) i wykonanie SQL przez `asyncpg`; pomiar TTFT – bezpośrednio Ollama; B1 – funkcje modułu `out_of_band.py` w jednym procesie; B2 – `build_keyset_query` i `fetch_paginated_data` z wykonaniem w PostgreSQL; C1 – funkcja walidatora `_worker_validate_query` wywoływana w pętli zdarzeń, puli wątków lub puli procesów; D – funkcje `validate_read_only` i `encapsulate_untrusted_database_records` oraz bezpośrednie wywołania modelu; dryf – `SchemaEngine`. Taki zakres izoluje koszty badanych mechanizmów i zapewnia porównywalność wyników między komponentami.

6.1.3 Procedura wnioskowania statystycznego

Dobór testów wynika ze struktury danych. Dla sparowanych wyników ciągłych na tych samych zapytaniach (Table-Recall, Column-Recall) zastosowano dwustronny test rangowanych znaków Wilcoxona [58]; pary o zerowej różnicy pomijano, więc w teście uczestniczy $\mathrm{n_{eff}} \leq \mathrm{N}$ par:

$$\mathrm{W} = \min(\mathrm{W}^+, \mathrm{W}^-), \quad \mathrm{W}^+ = \sum_{\mathrm{d_i}>0} \mathrm{R_i}, \quad \mathrm{W}^- = \sum_{\mathrm{d_i}<0} \mathrm{R_i} \tag{32}$$

$$\mathrm{z} = \frac{\mathrm{W} - \frac{\mathrm{n_{eff}}(\mathrm{n_{eff}}+1)}{4}}{\sqrt{\frac{\mathrm{n_{eff}}(\mathrm{n_{eff}}+1)(2\mathrm{n_{eff}}+1)}{24}}} \tag{33}$$

Wielkość efektu podano jako korelację rangowo-biserialną:

$$\mathrm{r} = \frac{\mathrm{W}^+ - \mathrm{W}^-}{\mathrm{W}^+ + \mathrm{W}^-} \tag{34}$$

Dla sparowanych wyników binarnych (EX) zastosowano dokładny test McNemara na parach niezgodnych. Dla niezależnych prób czasów (transfer OOB, opóźnienie pętli zdarzeń) zastosowano dwustronny test Manna–Whitneya z korelacją rangowo-biserialną $\mathrm{r} = 1 - 2\mathrm{U}/(\mathrm{n_a n_b})$ oraz przedziałem ufności 95% różnicy median wyznaczonym bootstrapem (ziarno 12345). Przedziały ufności dla

median czasów wyznaczono bootstrapem, a dla odsetków – metodą Cloppera–Pearsona. Przyjęto poziom istotności $\alpha = 0{,}05$. Przy dwóch testach głównych hipotezy H1 zastosowano korektę Holma.

6.2 Semantyczna jakość odkrywania schematu relacyjnego (Scenariusz A)

6.2.1 Zbiór zapytań i badane strategie

Pytania i zapytania wzorcowe wygenerowano z szablonów (klasa `SyntheticQueryBenchmarkGenerator`): dla tabeli startowej wyznaczano ścieżkę prostą w grafie kluczy obcych i wypełniano nią szablon złączenia z projekcją (150 zapytań prostych), agregacji z `GROUP BY` i `HAVING` (250 średnich) lub zapytania z wyrażeniem CTE (100 złożonych). Treść pytania wymienia tabele ścieżki oraz żądane kolumny, nie podaje natomiast kolumn złączeń. Zapytania wzorcowe obejmują 2 tabele w 75 przypadkach, 3 w 200, 4 w 160, 5 w 35 i 6 w 30. Pełne przykłady JSONL pozostawiono w załączniku do archiwum źródłowego; w tekście wystarcza charakterystyka dwóch klas: zapytania projekcyjne z jednym złączeniem oraz agregacje z `GROUP BY`, `HAVING` i dwoma złączeniami.

Cechy zbioru istotne dla interpretacji. W strukturze zbioru zadań testowych należy zwrócić uwagę na dwa aspekty:

1. **Jawne desygnaty encji.** Treść każdego pytania zawiera identyfikatory tabel zapytania wzorcowego, co definiuje deterministyczny scenariusz referencyjny: ocenie podlega odtworzenie relacyjnej struktury połączeń przy z góry określonych węzłach.
2. **Asymetria językowa.** Pytania są w języku polskim, a identyfikatory tabel w angielskim. Użycie `all-MiniLM-L6-v2` stanowi celowy test odporności wyszukiwania międzyjęzykowego w warunkach typowych dla wdrożeń, w których język intencji i język schematu nie są tożsame.

W 85,0% zapytań ścieżka wzorcowa obejmuje co najmniej trzy tabele, a więc zawiera tabelę pośrednią; w 241 zapytaniach (48,2%) ekspansja dołączyła do kontekstu co najmniej jedną tabelę wzorcową.

Porównano następujące strategie dostarczania schematu (wszystkie korzystają z tego samego formatu DDL, generowanego jedną funkcją syntezy):

1. **Domain-Scoped:** statyczny DDL 15 tabel domeny biznesowej zapytania;
2. **BM25:** wyszukiwanie leksykalne k tabel;
3. **Dense-only k-NN:** wyszukiwanie najbliższych sąsiadów w przestrzeni osadzeń `pgvector`;
4. **Hybrid RRF:** fuzja rankingów BM25 i gęstego algorytmem Reciprocal Rank Fusion;
5. **Ekspansja grafowa (autorska, oznaczenie Graph):** fuzja RRF uzupełniona o tabele mostkujące na najkrótszych ścieżkach grafu kluczy obcych ($\mathrm{M}_{\mathrm{exp}} = 3$).

Wariant autorski przy $\mathrm{k} = 5$ zwraca od 5 do 8 tabel (średnio 6,02), a jego zbiór zawsze zawiera wynik RRF@5. Aby porównanie odbywało się przy zbliżonym budżecie kontekstu, warianty BM25 i RRF ewaluowano dodatkowo przy $\mathrm{k} = 8$, czyli przy maksymalnej liczbie tabel wariantu autorskiego („równy budżet").

6.2.2 Jakość selekcji schematu

Tabela 8 zestawia metryki selekcji. Kolumny „Tokeny" podają średnią liczbę tokenów bloku DDL oraz całego promptu (instrukcja, DDL i pytanie; enkoder `cl100k_base`).

Tabela 8. Jakość selekcji schematu i objętość kontekstu dla badanych strategii ($\mathrm{N} = 500$; średnie po zapytaniach)

Strategia	Liczba tabel	Table-Recall [%]	Table-Precision [%]	Column-Recall [%]	Tokeny DDL	Tokeny promptu
Domain-Scoped (15 tabel)	15,00	8,5	2,0	8,2	3 264	3 484
BM25, $\mathrm{k} = 5$	5,00	47,8	32,1	62,4	1 147	1 367
Dense-only k-NN, $\mathrm{k} = 5$	5,00	31,2	19,8	40,9	1 166	1 387
Hybrid RRF, $\mathrm{k} = 5$	5,00	40,4	26,5	53,3	1 161	1 382
Ekspansja grafowa (autorska), $\mathrm{k} = 5$	6,02	55,3	30,4	62,5	1 482	1 703
BM25, $\mathrm{k} = 8$ (równy budżet)	8,00	53,2	22,6	67,2	1 848	2 068
Hybrid RRF, $\mathrm{k} = 8$ (równy budżet)	8,00	47,0	19,6	60,2	1 870	2 090

Pełny zrzut DDL wszystkich 1000 tabel w tym samym formacie liczy 263 113 tokenów i przekracza okno `num_ctx` = 8 192 obu modeli, więc nie ewaluowano go pod kątem EX. Każda selekcja k tabel redukuje objętość kontekstu o ponad 99% (np. wariant autorski do 1 482 tokenów, czyli o 99,4%, a RRF@5 do 1 161 tokenów); redukcja względem pełnego zrzutu jest więc cechą selekcji jako takiej, a nie zasługą ekspansji grafowej. Właściwym porównaniem strategii jest czułość przy równym budżecie tabel lub tokenów.

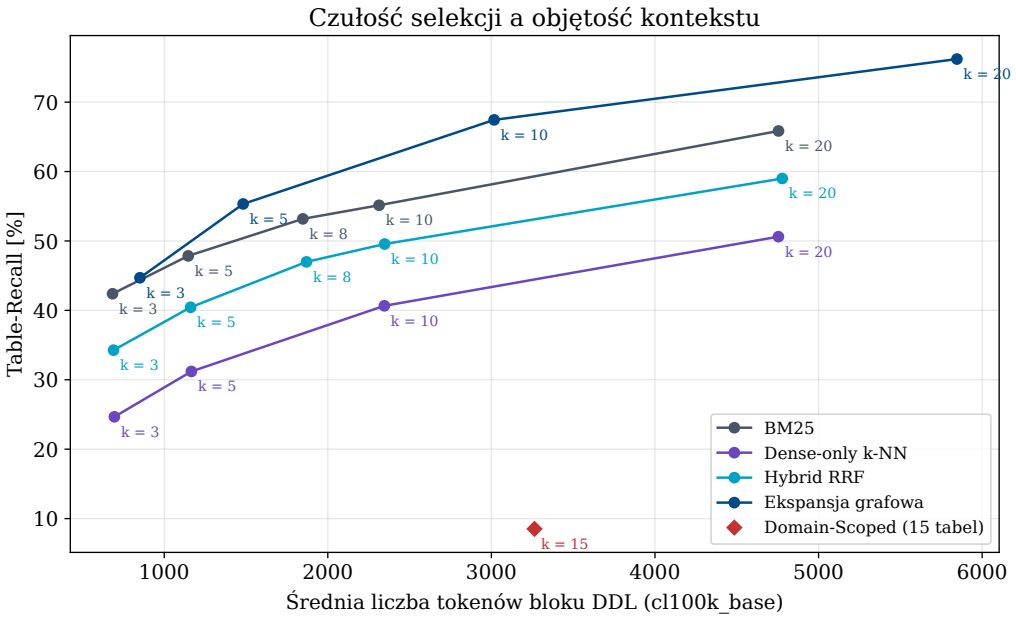
Przy tym samym $\mathrm{k} = 5$ wariant autorski osiąga Table-Recall 55,3% wobec 47,8% dla BM25, 40,4% dla RRF i 31,2% dla Dense-only, zużywając jednak więcej tokenów DDL (1 482 wobec 1 161 dla RRF@5). Przy równym budżecie przewaga maleje: RRF@8 osiąga 47,0% (1 870 tokenów DDL), a BM25@8 – 53,2% (1 848 tokenów). W Column-Recall wariant autorski (62,5%) przewyższa RRF@8 (60,2%), lecz ustępuje BM25@8 (67,2%). Różnica odzwierciedla kompromis: ekspansja grafowa priorytetyzuje spójność relacyjną i tabele asocjacyjne N:M niezbędne do złączeń JOIN, podczas gdy BM25 optymalizuje pokrycie leksykalne pojedynczych atrybutów biznesowych kosztem braków strukturalnych. Strategia Domain-Scoped osiąga najniższą czułość (8,5%) przy największym bloku DDL (3 264 tokenów), ponieważ tabele ścieżek wzorcowych na ogół wykraczają poza 15 tabel bazowych jednej domeny. Zależność wariantu Dense-only i RRF od asymetrii językowej należy interpretować jako cechę badanego profilu cross-lingual retrieval, a nie jako wadę samego paradygmatu.

Kontrola: dopasowanie identyfikatorów (analiza wtórna). Ponieważ pytania zawierają nazwy tabel, zbadano prosty wariant bez modelu i bez bazy: wybór tych tabel katalogu, których identyfika-

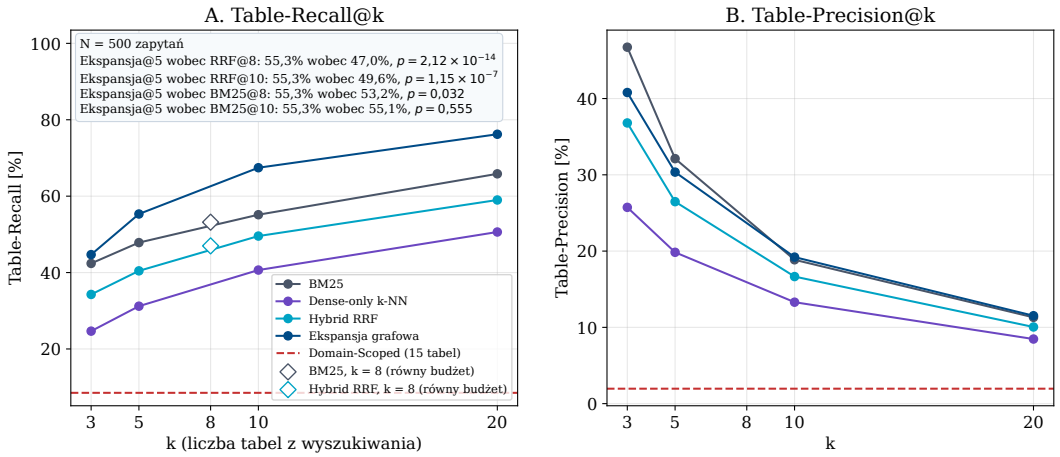
tory występują dosłownie w treści pytania (skrypt `o4_identifier_match.py`; katalog i graf kluczy obcych odtworzone deterministycznym generatorem schematu). Dopasowanie identyfikatorów osiąga Table-Recall 100,0% i Table-Precision 100,0% przy średnio 3,49 tabeli, czyli pełną czułość w 100,0% zapytań. Wynik pełni funkcję leksykalnej próby kontrolnej typu *ceiling* test, wyznaczającej górną granicę dopasowania nazw, a nie realistycznej strategii produkcyjnej. Wyniki Scenariusza A należy zatem interpretować jako porównanie mechanizmów wyszukiwania i ekspansji w referencyjnym warunku jawnego ugruntowania encji. EX dla dopasowania identyfikatorów nie jest znane, ponieważ wymagałoby nowych wywołań modelu.

Ekspansja na innym rankingu (analiza wtórna). Tym samym skryptem zastosowano algorytm ekspansji (odtworzony z kodu bramy; replikacja odtworzyła zbiory tabel wariantu autorskiego z rundy w 491 z 500 zapytań, różnice wynikają z wyboru jednej z równorzędnych najkrótszych ścieżek) do rankingu BM25@5 zamiast RRF@5. Uzyskano Table-Recall 63,0% (Table-Precision 36,0%, średnio 5,90 tabeli, pełna czułość w 25,2% zapytań), czyli więcej niż dla ekspansji nałożonej na RRF (55,3%). Wynik nie obejmuje EX.

Czułość a objętość kontekstu. Rysunek 3 zestawia czułość ze średnią liczbą tokenów bloku DDL dla wszystkich strategii i wartości k. Punkty wariantu autorskiego tworzą wraz z BM25 dla $k \in \{3, 5\}$ front Pareto (najwyższa czułość przy danej objętości). Przy objętości DDL wariantu autorskiego dla $k = 5$ (1 482 tokenów) interpolowana liniowo między zmierzonymi punktami czułość BM25 wynosi ok. 50,4%, a RRF ok. 43,4% (wobec 55,3%); wartości te są interpolacją, a nie pomiarem.



Rysunek 3. Table-Recall wobec średniej liczby tokenów bloku DDL dla badanych strategii i wartości k ($N = 500$; analiza wtórna pliku `schema_retrieval.parquet`)



Rysunek 4. Table-Recall@k oraz Table-Precision@k w funkcji parametru $k \in \{3, 5, 10, 20\}$ dla badanych strategii ($N = 500$); wariant autorski zwraca przy danym k więcej tabel niż warianty bazowe

Rysunek 4 przedstawia czułość i precyzję w funkcji k. Wariant autorski ma najwyższą czułość przy każdym k (od 44,7% przy $k = 3$ do 76,2% przy $k = 20$), lecz przy danym k zwraca więcej tabel niż warianty bazowe, więc porównanie przy tym samym k nie jest porównaniem przy równym budżecie. Przy $k = 10$ wariant BM25 osiąga 55,1%, a RRF 49,6%.

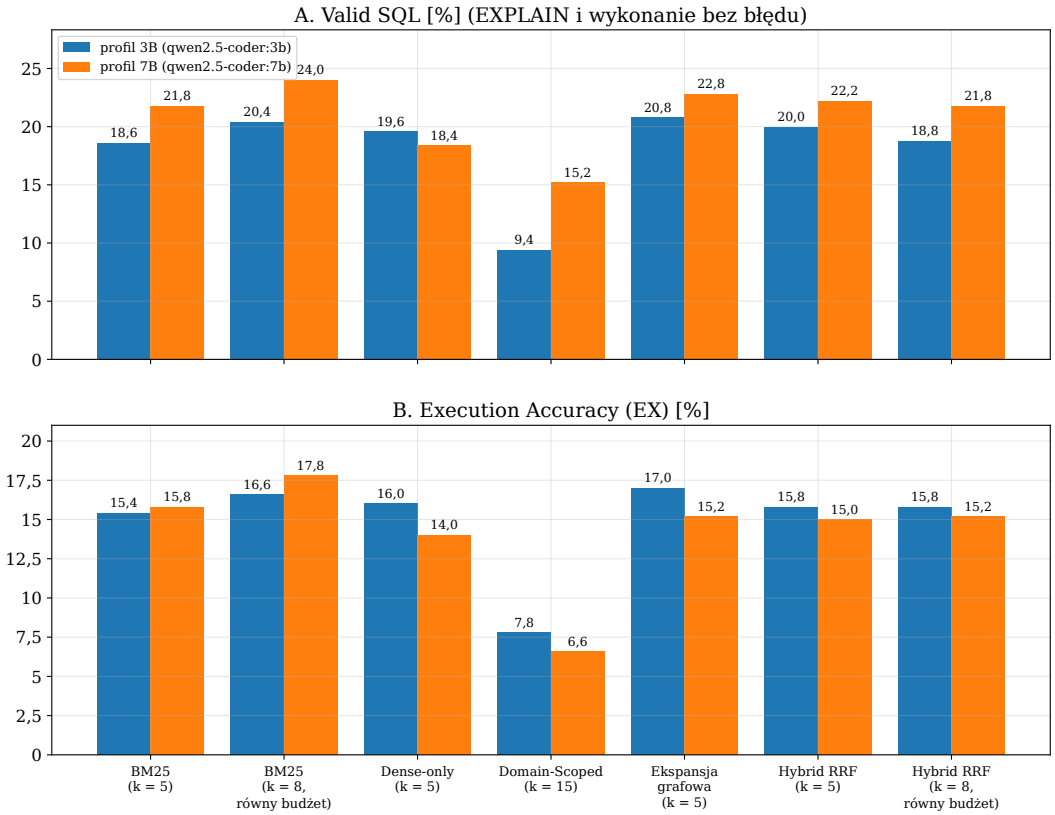
6.2.3 Dokładność generacji Text-to-SQL

Generację SQL wykonano dla $k = 5$ oraz dla wariantów równego budżetu ($k = 8$) na obu profilach modeli, łącznie 7 000 wywołań modelu. Tabela 9 i rysunek 5 przedstawiają odsetki Valid SQL oraz EX z przedziałami ufności 95%.

Tabela 9. Odsetek Valid SQL oraz EX [%] z 95% przedziałem ufności Cloppera–Pearsona dla profili 3B i 7B ($N = 500$ na strategię)

Strategia	Profil 3B		Profil 7B	
	Valid SQL	EX [95% CI]	Valid SQL	EX [95% CI]
Domain-Scoped (15 tabel)	9,4	7,8 [5,6; 10,5]	15,2	6,6 [4,6; 9,1]
BM25, $k = 5$	18,6	15,4 [12,3; 18,9]	21,8	15,8 [12,7; 19,3]
Dense-only k-NN, $k = 5$	19,6	16,0 [12,9; 19,5]	18,4	14,0 [11,1; 17,4]
Hybrid RRF, $k = 5$	20,0	15,8 [12,7; 19,3]	22,2	15,0 [12,0; 18,4]
Ekspansja grafowa (autorska), $k = 5$	20,8	17,0 [13,8; 20,6]	22,8	15,2 [12,2; 18,7]
BM25, $k = 8$ (równy budżet)	20,4	16,6 [13,4; 20,2]	24,0	17,8 [14,5; 21,4]
Hybrid RRF, $k = 8$ (równy budżet)	18,8	15,8 [12,7; 19,3]	21,8	15,2 [12,2; 18,7]

Odsetki EX wszystkich strategii selekcyjnych mieszczą się w wąskim zakresie: na profilu 3B od 15,4% (BM25@5) do 17,0% (ekspansja grafowa), a na profilu 7B od 14,0% (Dense-only) do 17,8% (BM25@8); wariant autorski osiąga na profilu 7B 15,2%. Przedziały ufności wszystkich strategii selekcyjnych nakładają się. Wyraźnie niższy wynik daje tylko Domain-Scoped (7,8% i 6,6%). Profil



Rysunek 5. Valid SQL oraz Execution Accuracy [%] dla profili 3B (`qwen2.5-coder:3b`) i 7B (`qwen2.5-coder:7b`), $\mathrm{N} = 500$ zapytań na strategię

7B nie dał systematycznej poprawy EX względem profilu 3B: dla części strategii EX wzrósł, a dla części spadł.

Tabela 10 zestawia dokładne testy McNemara dla EX wariantu autorskiego względem każdej strategii bazowej. Bez korekty na wielokrotne porównania jedyną istotną różnicą między strategiami selekcyjnymi jest przewaga BM25@8 nad wariantem autorskim na profilu 7B (9 wobec 22 par niezgodnych, $\mathrm{p} = 0{,}029$). Po korekcie Holma w rodzinie sześciu porównań profilu (analiza wtórna, skrypt `o2_holm_mcnemar.py`) różnica ta przestaje być istotna ($\mathrm{p_{Holm}} = 0{,}147$), podobnie jak wszystkie pozostałe różnice między strategiami selekcyjnymi na obu profilach; istotna pozostaje jedynie przewaga nad Domain-Scoped. Dane nie pozwalają więc wskazać strategii selekcyjnej o wyższej EX w żadnym kierunku.

Czułość pełna a EX (analiza wtórna). Średnia czułość słabo opisuje to, co jest istotne dla generacji: zapytanie może się udać tylko wtedy, gdy wycinek zawiera wszystkie potrzebne tabele. Wariant autorski dostarcza pełny zbiór tabel wzorcowych (Table-Recall $= 1$) w 20,2% zapytań, wobec 11,2% dla BM25@8 i 8,0% dla RRF@8 (skrypt `o1_czulosc_a_ex.py`). Przy pełnej czułości EX wariantu autorskiego wynosi jednak 50,5% [40,4; 60,6] na profilu 3B i 43,6% [33,7; 53,8] na profilu 7B, mniej niż dla BM25@8 (67,9% i 58,9%). Dodatkowe zapytania, dla których ekspansja kompletuje wycinek, to głównie zapytania z trzema i więcej tabelami, a te w tym zbiorze prawie nigdy nie kończą się poprawnym wynikiem: EX wariantu autorskiego wynosi 90,7% (3B) i 82,7% (7B) dla ścieżek dwutabelowych, ale tylko 8,5% i 7,0% dla trzytabelowych i zero dla dłuższych; podobnie jest dla pozostałych strategii. Wyjaśnia to, dlaczego wyższa czułość nie przekłada się na EX.

Tabela 10. Dokładny test McNemara dla EX: ekspansja grafowa ($\mathrm{k} = 5$) wobec strategii bazowych; a – liczba zapytań poprawnych tylko dla wariantu autorskiego, b – tylko dla strategii bazowej ($\mathrm{N} = 500$); $\mathrm{p_{Holm}}$ – wartość p po korekcie Holma w rodzinie sześciu porównań danego profilu (analiza wtórna)

Profil	Strategia bazowa	EX Graph / bazowa [%]	a	b	p	$\mathrm{p_{Holm}}$
3B	RRF, $\mathrm{k} = 5$	17,0 / 15,8	9	3	$0{,}146$	$0{,}730$
3B	BM25, $\mathrm{k} = 5$	17,0 / 15,4	16	8	$0{,}152$	$0{,}730$
3B	RRF, $\mathrm{k} = 8$	17,0 / 15,8	13	7	$0{,}263$	$0{,}790$
3B	BM25, $\mathrm{k} = 8$	17,0 / 16,6	10	8	$0{,}815$	$0{,}849$
3B	Dense-only, $\mathrm{k} = 5$	17,0 / 16,0	15	10	$0{,}424$	$0{,}849$
3B	Domain-Scoped	17,0 / 7,8	48	2	$2{,}3 \cdot 10^{-12}$	$1{,}4 \cdot 10^{-11}$
7B	RRF, $\mathrm{k} = 5$	15,2 / 15,0	7	6	$1{,}000$	$1{,}000$
7B	BM25, $\mathrm{k} = 5$	15,2 / 15,8	14	17	$0{,}720$	$1{,}000$
7B	RRF, $\mathrm{k} = 8$	15,2 / 15,2	14	14	$1{,}000$	$1{,}000$
7B	BM25, $\mathrm{k} = 8$	15,2 / 17,8	9	22	$0{,}029$	$0{,}147$
7B	Dense-only, $\mathrm{k} = 5$	15,2 / 14,0	15	9	$0{,}307$	$1{,}000$
7B	Domain-Scoped	15,2 / 6,6	47	4	$2{,}4 \cdot 10^{-10}$	$1{,}4 \cdot 10^{-9}$

6.2.4 Dekompozycja błędów generacji zapytań

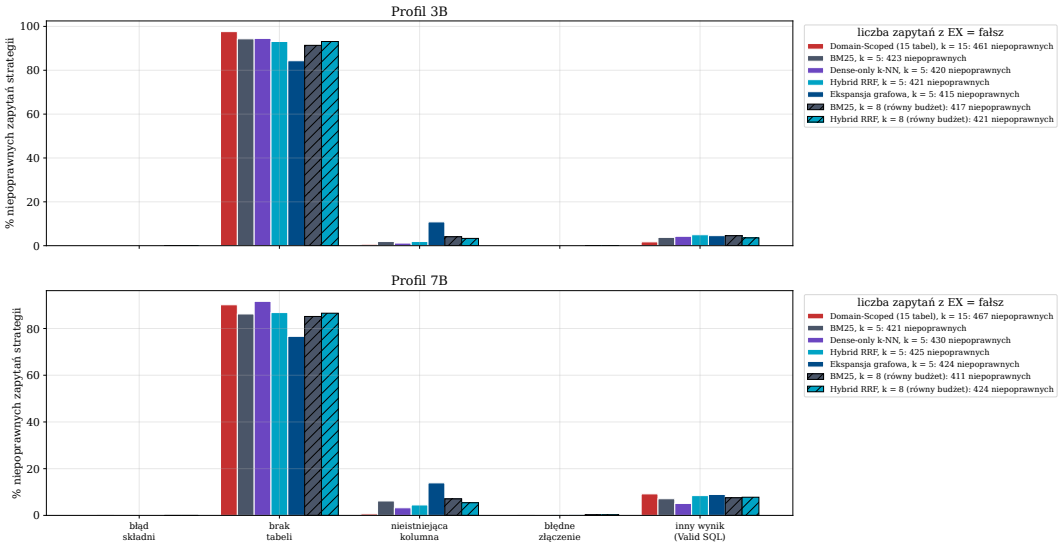
Tabela 11 i rysunek 6 przedstawiają rozkład kategorii błędów dla wszystkich strategii i obu profili.

Wpływ złożoności strukturalnej na Execution Accuracy. Analiza wtórna rekordów generacji (skrypt `o9_klasyfikacja.py`) wykazuje, że zróżnicowanie wskaźnika EX pomiędzy kategoriami zapytań stanowi bezpośrednią konsekwencję ich złożoności strukturalnej. Podczas gdy dla zapytań prostych średnia wartość EX wynosi 48,4%, w przypadku zapytań średnich (z wielokrotnym złączeniem i agregacją) oraz złożonych (z podzapytaniami CTE, EX równe 0,2%) wysoki stopień komplikacji relacyjnej i składniowej przekroczył możliwości wnioskowania małych, lokalnych modeli językowych (3B i 7B). Niewielkie modele lokalne mają trudności z poprawną koordynacją wielu predykatów złączeń, klauzul grupowania oraz konstrukcji CTE w rozległym schemacie relacyjnym, co prowadziło do niepoprawności generowanego kodu SQL mimo dostarczenia właściwych struktur tabel w promptcie.

Kategoria `TABLE_MISSING`. Zgodnie z kolejnością reguł taksonomii rekord trafia do tej kategorii, gdy zapytanie odwołuje się do tabeli spoza wycinka, niezależnie od zgłoszonego błędu. Ponieważ pytania wymieniają nazwy wszystkich tabel, model używa ich także bez otrzymania ich definicji, a zapytanie kończy się błędem nieistniejącej kolumny (w 99,0% rekordów tej kategorii PostgreSQL zgłosił nieistniejącą kolumnę): model zgaduje nazwę kolumny klucza obcego i kierunek złączenia. Ręczny przegląd losowej próby 40 rekordów wariantu autorskiego z EX = fałsz (po 20 na profil, ziarno 2026; plik `derived/o9_klasyfikacja_reczna.csv`) potwierdził ten mechanizm: w 27 rekordach model użył tabel z pytania nieobecnych w wycinku i błędnie odgadł kolumnę złączenia, w 5 błędnie odgadł kolumnę lub kierunek złączenia mimo obecności wszystkich tabel, w 3 błędnie zinterpretował alias z treści pytania złożonego, a w 5 zapytanie było poprawne składniowo, lecz zwracało odmienny zbiór wynikowy. Nie stwierdzono błędów ekstrakcji SQL z odpowiedzi (wszystkie odpowiedzi zawierały blok kodu, żadna nie została ucięta), więc różnice między profilami nie wynikają z formatu odpowiedzi.

Tabela 11. Liczba rekordów z EX = fałsz w kategoriach taksonomii błędów ($\mathrm{N} = 500$ na strategię i profil)

Profil	Strategia	EX = fałsz	TABLE_ MISSING	COLUMN_ HALLUC.	SEMANTIC_ MISMATCH	JOIN_ COND.	SYNTAX_ ERROR	TRUNC.
3B	Domain-Scoped (15 tabel)	461	450	3	8	0	0	0
3B	BM25, $\mathrm{k} = 5$	423	399	8	16	0	0	0
3B	Dense-only k-NN, $\mathrm{k} = 5$	420	397	5	18	0	0	0
3B	Hybrid RRF, $\mathrm{k} = 5$	421	392	8	21	0	0	0
3B	Ekspansja grafowa (autorska), $\mathrm{k} = 5$	415	350	45	19	1	0	0
3B	BM25, $\mathrm{k} = 8$ (równy budżet)	417	381	17	19	0	0	0
3B	Hybrid RRF, $\mathrm{k} = 8$ (równy budżet)	421	392	14	15	0	0	0
7B	Domain-Scoped (15 tabel)	467	421	3	43	0	0	0
7B	BM25, $\mathrm{k} = 5$	421	363	26	30	1	1	0
7B	Dense-only k-NN, $\mathrm{k} = 5$	430	394	14	22	0	0	0
7B	Hybrid RRF, $\mathrm{k} = 5$	425	369	19	36	1	0	0
7B	Ekspansja grafowa (autorska), $\mathrm{k} = 5$	424	325	59	38	2	0	0
7B	BM25, $\mathrm{k} = 8$ (równy budżet)	411	350	29	31	1	0	0
7B	Hybrid RRF, $\mathrm{k} = 8$ (równy budżet)	424	367	23	33	1	0	0



Rysunek 6. Rozkład kategorii błędów (odsetek zapytań z EX = fałsz danej strategii) dla profili 3B i 7B, łącznie z wariantami równego budżetu ($k = 8$, kreskowanie); w legendzie liczba zapytań z EX = fałsz. Kategoria `TRUNCATION` nie wystąpiła; kategoria pustego wyniku wzorca (`GT_EMPTY`) dotyczy nieważnych próbek i również nie wystąpiła

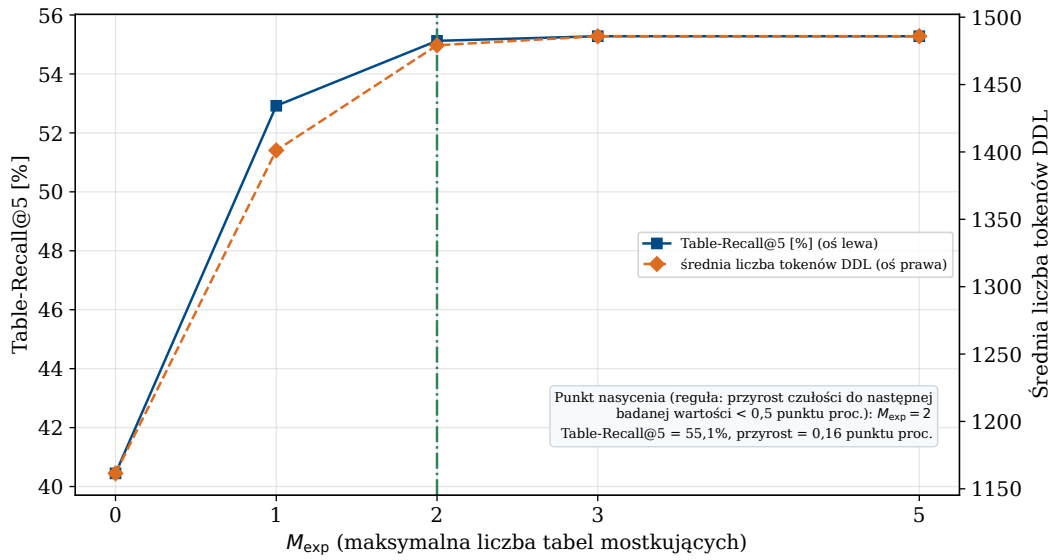
Formalnie dominującą kategorią jest `TABLE_MISSING`: dla wariantu autorskiego obejmuje 350 z 415 błędnych rekordów na profilu 3B (84,3%) oraz 325 z 424 na profilu 7B (76,7%). Model odwołuje się więc najczęściej do tabeli spoza dostarczonego wycinka, co jest zgodne z konstrukcją pytań wymieniających wprost nazwy wszystkich tabel ścieżki: brak dowolnej z nich w wycinku kończy się błędem. Na profilu 7B wariant autorski ma 59 halucynacji kolumn i 38 niezgodności semantycznych, a błędy predykatu złączenia występują sporadycznie (2). Kategoria `TRUNCATION` nie wystąpiła w żadnej strategii, co jest zgodne z brakiem rekordów z podejrzeniem obcięcia kontekstu.

6.2.5 Badanie ablacyjne parametru ekspansji grafowej $\mathrm{M}_{\mathrm{exp}}$

Wpływ maksymalnej liczby tabel mostkujących zbadano dla $\mathrm{M}_{\mathrm{exp}} \in \{0, 1, 2, 3, 5\}$ na zbiorze $\mathrm{N} = 500$ zapytań przy $\mathrm{k} = 5$ (rysunek 7).

Bez ekspansji ($\mathrm{M}_{\mathrm{exp}} = 0$, wynik równy RRF@5) Table-Recall@5 wynosi 40,4% przy 1 161 tokenach DDL. Jedna tabela mostkująca podnosi czułość do 52,9% (1 401 tokenów), dwie – do 55,1% (1 479 tokenów), a $\mathrm{M}_{\mathrm{exp}} = 3$ i $\mathrm{M}_{\mathrm{exp}} = 5$ dają identyczne 55,3% (1 486 tokenów). Reguła punktu nasycenia zapisana w konfiguracji przed pomiarem (najmniejsze $\mathrm{M}_{\mathrm{exp}}$, dla którego przyrost względem następnej badanej wartości jest mniejszy niż 0,5 punktu procentowego) wskazuje $\mathrm{M}_{\mathrm{exp}} = 2$ (przyrost 0,16 pp). Kod bramy i pozostałe eksperymenty używają $\mathrm{M}_{\mathrm{exp}} = 3$; różnica czułości między wartościami 2 i 3 jest pomijalna, podobnie jak różnica objętości DDL.

Czas pojedynczego wywołania selekcji przy $\mathrm{M}_{\mathrm{exp}} = 3$ (mediana i p_{95} z $\mathrm{N} = 500$ wywołań) wyniósł 19,2 ms (p_{95} 23,2 ms). Dominuje fuzja RRF z wyszukiwaniem wektorowym (18,0 ms); załadowanie grafu z pamięci podręcznej zajęło 0,16 ms, wyszukiwanie mostków 0,18 ms (p_{95} 0,29 ms), a synteza DDL 0,84 ms.



Rysunek 7. Badanie ablacyjne parametru $\mathrm{M_{exp}} \in \{0, 1, 2, 3, 5\}$: Table-Recall@5 oraz średnia liczba tokenów bloku DDL ($\mathrm{N} = 500$); linia przerywana pionowa wskazuje punkt nasycenia według reguły z konfiguracji rundy

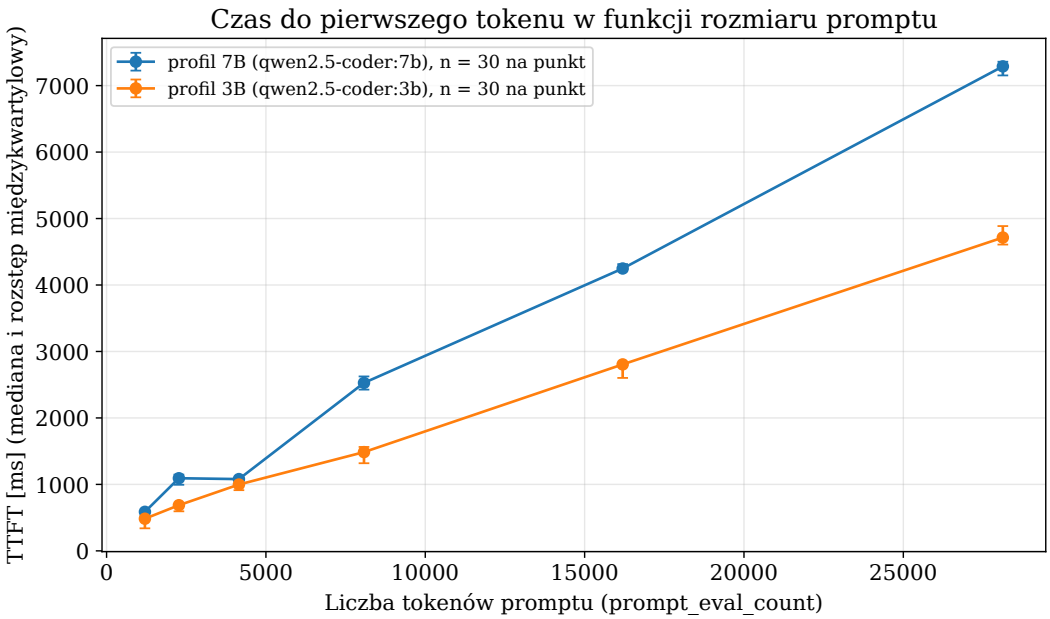
6.2.6 Czas do pierwszego tokenu w funkcji rozmiaru promptu

W osobnym eksperymencie zmierzono TTFT dla promptów zawierających bloki DDL o docelowej objętości od 1000 do 28 000 tokenów, po 30 powtórzeń na punkt dla każdego profilu. Okno `num_ctx` dobierano do rozmiaru promptu z zapasem 2048 tokenów, a do każdego promptu dodawano unikalny identyfikator, aby wyeliminować ponowne użycie pamięci podręcznej prefiksu w serwerze Ollama. Liczbę tokenów promptu odczytano z licznika `prompt_eval_count` serwera; w żadnym pomiarze nie wykryto obcięcia kontekstu. Wyniki przedstawia tabela 12 i rysunek 8.

Tabela 12. Mediana TTFT [ms] z 95% przedziałem ufności (bootstrap), $\mathrm{p_{95}}$ TTFT oraz mediana całkowitego czasu żądania $\mathrm{L_{E2E}}$ [ms] w funkcji liczby tokenów promptu (30 powtórzeń na punkt; generacja ograniczona do 64 tokenów)

Tokeny promptu	num_ctx	Profil 3B			Profil 7B		
		TTFT [95% CI]	$\mathrm{p_{95}}$	$\mathrm{L_{E2E}}$	TTFT [95% CI]	$\mathrm{p_{95}}$	$\mathrm{L_{E2E}}$
1 203	4 096	484 [390; 489]	497	1 493	588 [581; 594]	696	1 903
2 270	5 120	687 [599; 692]	794	1 700	1 093 [997; 1 099]	1 236	2 498
4 152	7 168	998 [980; 1 019]	1 311	2 050	1 080 [1 069; 1 091]	1 438	2 651
8 077	12 288	1 486 [1 386; 1 494]	1 683	2 791	2 528 [2 432; 2 547]	2 631	4 210
16 194	21 504	2 805 [2 664; 2 820]	3 011	3 606	4 248 [4 213; 4 312]	4 368	5 599
28 122	32 768	4 714 [4 610; 4 819]	4 987	6 049	7 289 [7 207; 7 359]	7 451	8 901

TTFT rośnie w przybliżeniu liniowo z rozmiarem promptu: nachylenie prostej regresji wynosi 157 ms na 1000 tokenów dla profilu 3B (95% CI [154; 159]) oraz 246 ms na 1000 tokenów dla profilu 7B (95% CI [243; 248]). Dla promptu około 1 204 tokenów mediana TTFT wynosi 484 ms



Rysunek 8. Mediana TTFT w funkcji liczby tokenów promptu (`prompt_eval_count`) dla profili 3B i 7B (30 powtórzeń na punkt)

(3B) i 588 ms (7B), a dla około 28 123 tokenów – 4 714 ms i 7 289 ms. Dla profilu 7B zależność nie jest ściśle monotoniczna między punktami około 2000 i 4000 tokenów (tabela 12). Prompty Scenariusza A (do 3 691 tokenów) mieszczą się w dolnej części badanego zakresu. Pełnego zrzutu DDL (263 113 tokenów) nie da się umieścić w oknie kontekstu badanych modeli.

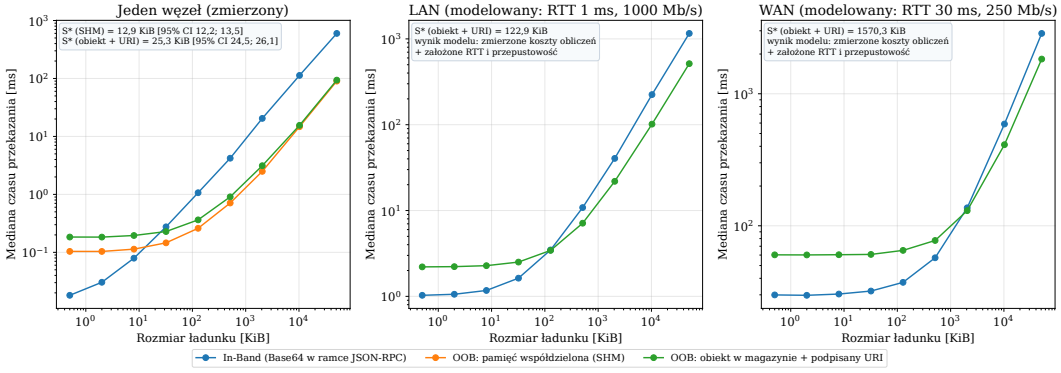
6.3 Analiza progu opłacalności transferu binarnego Out-of-Band (Scenariusz B1)

6.3.1 Warunki pomiaru

W Scenariuszu B1 zmierzono czas przekazania ładunku binarnego $S \in [512\ \text{B}; 50\ \text{MiB}]$ (9 rozmiarów, 50 powtórzeń na punkt po 5 przebiegach rozgrzewkowych) w trzech trybach: In-Band (ładunek zakodowany w Base64 w ramce JSON-RPC), OOB przez pamięć współdzieloną POSIX SHM oraz OOB przez obiekt zapisany w lokalnym magazynie z deskryptorem zawierającym podpisany URI. Wszystkie operacje nadawcy i odbiorcy wykonywano w jednym procesie: ramka JSON-RPC była serializowana i parsowana w pamięci, bez przesyłania przez potok `stdio` ani gniazdo sieciowe, a obiekt trybu OOB z URI odbiorca odczytywał bezpośrednio z pliku. Mierzony czas obejmuje więc koszty obliczeniowe przekazania (kodowanie i dekodowanie Base64, sumy kontrolne SHA-256 – w trybie In-Band dwukrotnie, w trybie SHM jednokrotnie, bez weryfikacji przy odczycie – serializację ramki, kopiowanie do segmentu i z segmentu, zapis i odczyt pliku), a nie transmisję przez kanał MCP.

Korzyść z transferu OOB dotyczy czasu i zasobów hosta; dane przekazane poza pasmem nie trafiają do okna kontekstu modelu w żadnym z trybów, więc pomiar nie dotyczy kosztu tokenowego.

Profil *loopback* zmierzono bezpośrednio na jednym węźle i wykorzystano jako kalibrację kosztów obliczeniowych. Profile LAN i WAN stanowią analityczną projekcję parametryczną: do empirycznie zmierzonego czasu kodowania, kopiowania, sum kontrolnych i serializacji dodano składniki sieciowe wynikające z RTT, przepustowości i rozmiaru ramki. Dla In-Band uwzględniono jeden czas RTT oraz czas przesłania ramki $(8 \cdot |\text{ramka}|/\mathrm{B})$, a dla OOB z URI dwa czasy RTT oraz czasy przesłania deskryptora i ładunku. Przyjęte parametry to RTT $=$ 1 ms, przepustowość $\mathrm{B} = 1000$ Mb/s i brak fluktuacji dla LAN oraz RTT $=$ 30 ms, $\mathrm{B} = 250$ Mb/s i fluktuacja ± 2 ms dla WAN. Progi LAN i WAN opisują zatem zachowanie modelu skalibrowanego na pojedynczym węźle; bootstrapowy przedział ufności odnosiłby się wyłącznie do zmienności obliczeń lokalnych. Mediany czasów transferu dla wszystkich profili przedstawia rysunek 9.



Rysunek 9. Mediana czasu przekazania ładunku w funkcji jego rozmiaru dla trybu In-Band (Base64) i trybów Out-of-Band (pamięć współdzielona; obiekt w magazynie z podpisanym URI). Profil jednego węzła zmierzono; profile LAN (RTT 1 ms, 1000 Mb/s) i WAN (RTT 30 ms, 250 Mb/s) są wynikiem modelu (zmierzone koszty obliczeń z dodanym obliczonym czasem sieci), dlatego nie podano dla nich przedziałów ufności

6.3.2 Punkty przecięcia

Punkt przecięcia S^* wyznaczono jako przecięcie prostych dopasowanych do median czasów dwóch trybów w otoczeniu zmiany znaku różnicy (po dwa sąsiednie rozmiary), a przedział ufności – bootstrapem powtórzeń (tylko dla profilu zmierzonego). Wyniki zestawia tabela 13.

Tabela 13. Rozmiar ładunku S^* [KiB], powyżej którego tryb OOB jest szybszy od In-Band Base64 (95% CI bootstrap)

Profil	Tryb OOB	Tryb pomiaru	Parametry sieci	S^* [KiB]	95% CI
jeden węzeł	SHM	zmierzony	–	12,9	[12,2; 13,5]
jeden węzeł	obiekt + URI	zmierzony	–	25,3	[24,5; 26,1]
LAN	obiekt + URI	modelowany	RTT 1 ms, 1 000 Mb/s	122,9	–
WAN	obiekt + URI	modelowany	RTT 30 ms, 250 Mb/s	1 570,3	–

$$\mathrm{S}^{*}_{\text{loopback, SHM}} = 12{,}9 \text{ KiB} \tag{35}$$

Na jednym węźle pamięć współdzielona jest szybsza od In-Band od ok. 12,9 KiB, a OOB z obiektem w magazynie – od ok. 25,3 KiB. Dla ładunku 50 MiB mediana czasu wynosi 90,0 ms dla SHM wobec 599 ms dla In-Band, czyli 6,65-krotną różnicę. W trybie In-Band przy 50 MiB samo kodowanie Base64 zajmuje 101 ms, dekodowanie 165 ms, a każde obliczenie SHA-256 31,9 ms; kopiowanie do segmentu SHM zajmuje 27,9 ms. Narzut ramki In-Band względem ładunku dąży do 33,34% (przy 10 MiB), co odpowiada teoretycznemu narzutowi Base64 $(4/3)$, a dla ładunków 512 B wynosi 76,0% z powodu stałej części ramki JSON-RPC. W modelu LAN próg rośnie do 122,9 KiB, a w modelu WAN do 1 570,3 KiB, ponieważ dodatkowa runda sieciowa trybu OOB z URI jest amortyzowana dopiero przy dużych ładunkach.

Walidacja modelu progu (analiza wtórna). Wzór (22) zweryfikowano składowymi czasów zarejestrowanymi w rundzie (skrypt `o7_model_sstar.py`). Z median składowych dla rozmiarów od 512 B do 128 KiB wyznaczono metodą najmniejszych kwadratów przepustowości operacji oraz stałe narzuty (tabela 14). Podstawienie do wzoru daje $\mathrm{S}^{*}_{\text{SHM}} = 12{,}85$ KiB, w granicach przedziału ufności wartości empirycznej (12,9 KiB, 95% CI [12,2; 13,5]). Zgodność ta potwierdza spójność modelu z implementacją, lecz nie jest niezależną walidacją, ponieważ parametry pochodzą z tych samych pomiarów. Składnik resztowy ścieżki In-Band (serializacja i parsowanie ramki JSON oraz narzut wywołań) odpowiada za 44,3% jej kosztu jednostkowego, a więc za znaczną część przewagi trybu OOB. Gdyby odczyt z segmentu SHM obejmował weryfikację skrótu (jak w funkcji `retrieve`), model daje próg 14,1 KiB, poza przedziałem ufności pomiaru; asymetria ścieżek pomiarowych przesuwa więc zmierzony próg w stronę mniejszych ładunków.

Tabela 14. Parametry modelu progu S^{*} wyznaczone ze składowych czasów rundy (profil jednego węzła, MNK na medianach dla 512 B–128 KiB; analiza wtórna)

Parametr	Wartość
v_{enc} (kodowanie Base64)	909 MiB/s
v_{dec} (dekodowanie Base64)	428 MiB/s
v_{sha} (SHA-256)	1 583 MiB/s
v_{json} (składnik resztowy In-Band)	268 MiB/s
v_{copy} / v_{read} (segment SHM)	2 508 / 13 229 MiB/s
δ_{IB} / δ_{SHM}	13,4 / 103,5 μs
$\mathrm{S}^{*}_{\text{SHM}}$: model / pomiar	12,85 / 12,9 KiB

Hipotezę H2 zweryfikowano na profilu jednego węzła testem Manna–Whitneya dla rozmiaru poniżej i powyżej progu (tabela 20). Dla 2 KiB In-Band jest szybszy (mediana 0,03 ms wobec 0,10 ms dla SHM, $\mathrm{p} = 7{,}1 \cdot 10^{-18}$), a dla 10 MiB szybszy jest SHM (14,7 ms wobec 113 ms, $\mathrm{p} = 7{,}1 \cdot 10^{-18}$; różnica median 95% CI [-98,5; -98,0] ms). W obu przypadkach rozkłady nie zachodzą na siebie ($|\mathrm{r}| = 1$). Wynik ten jest jednak przesądzony konstrukcyjnie: rozmiary odległe o rzędy wielkości od

progu dają rozkłady rozdzielone, więc test potwierdza jedynie istnienie progu w badanym przedziale. Właściwą miarą jest położenie progu, czyli S^* z przedziałem ufności oraz jego zgodność z modelem.

6.3.3 Reguła architektoniczna

Przy kolokacji agenta i bazy na jednym węźle transfer przez pamięć współdzieloną jest korzystniejszy dla ładunków większych niż kilkanaście kilobajtów (zmierzony próg 12,9 KiB). W środowiskach rozproszonych próg rośnie z opóźnieniem sieci; według modelu z przyjętymi parametrami wynosi ok. 122,9 KiB dla LAN i ok. 1 570,3 KiB dla WAN, przy czym wartości te wymagają potwierdzenia pomiarem w rzeczywistej sieci.

6.4 Paginacja, obciążenie współbieżne i bilans zasobów (Scenariusze B2 & C1)

6.4.1 Skalowalność paginacji na wolumenie 1 000 000 rekordów

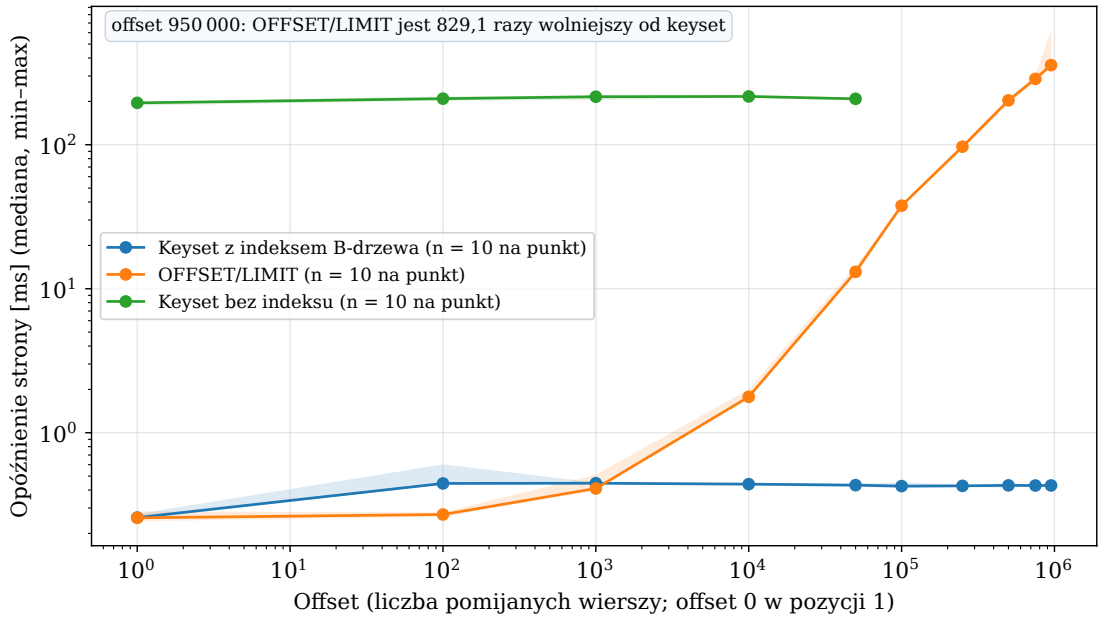
Opóźnienie pobrania strony 50 rekordów zmierzono na relacji 1 000 000 wierszy (`public.analytics_orders`) dla porządku `created_at DESC NULLS LAST, amount ASC NULLS LAST, id ASC` wspieranego kompozytowym indeksem B-drzewa, dla dziesięciu offsetów od 0 do 950 000 wierszy (95% tabeli), po 10 powtórzeń na punkt i 2 przebiegach rozgrzewkowych. Porównano warianty Indexed Keyset B-Tree, Offset/Limit oraz Unindexed Keyset (z wyłączonym skanem indeksu, mierzony do offsetu 50 000, ponieważ każde żądanie wymaga pełnego skanu relacji). Przebieg opóźnień przedstawia rysunek 10, a mediany – tabela 15.

Tabela 15. Mediany opóźnień pobrania strony 50 rekordów [ms] w funkcji offsetu M na relacji 1 mln rekordów (10 powtórzeń na punkt; „–" – brak pomiaru wariantu nieindeksowanego)

Offset M	Strona k	Indexed Keyset	Offset/Limit	Unindexed Keyset	Offset / Keyset
0	1	0,26	0,26	195	1,0×
100	3	0,44	0,27	209	0,6×
1 000	21	0,45	0,41	215	0,9×
10 000	201	0,44	1,77	216	4,0×
50 000	1 001	0,43	13,1	208	30,3×
100 000	2 001	0,43	37,7	–	88,6×
250 000	5 001	0,43	96,9	–	226,7×
500 000	10 001	0,43	203	–	471,6×
750 000	15 001	0,43	287	–	666,5×
950 000	19 001	0,43	357	–	829,1×

Wyniki z tabeli 15:

- **Indexed Keyset B-Tree** zachowuje stałe opóźnienie: przy offsecie 950 000 mediana wynosi 0,43 ms (0,26 ms dla pierwszej strony). Nachylenie prostej regresji względem offsetu (0,0032 ms na 100 tys. wierszy) nie różni się istotnie od zera ($\mathrm{p} = 0{,}581$), co jest zgodne z kosztem $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$;



Rysunek 10. Opóźnienie pobrania strony 50 rekordów w funkcji offsetu (do 950 000 wierszy) dla paginacji keyset z indeksem B-drzewa, OFFSET/LIMIT oraz keyset bez indeksu (skala log-log; mediana i zakres min–max z 10 powtórzeń). Offset 0 (pierwsza strona) narysowano w pozycji 1, ponieważ skala logarytmiczna nie obejmuje zera

- **Offset/Limit** degraduje się liniowo: nachylenie wynosi 38,3 ms na 100 tys. pominiętych wierszy ($R^2 = 0{,}999$), a przy $M = 950\,000$ mediana sięga 357 ms (zakres 341–617 ms), czyli 829 razy więcej niż keyset;
- dla płytkich offsetów ($M \leq 1000$) oba warianty mają porównywalne opóźnienie, a przewaga keyset rośnie z głębokością;
- **Unindexed Keyset** ma stałe, wysokie opóźnienie 187–221 ms (mediana 212 ms) niezależnie od offsetu, ponieważ każde żądanie wymaga skanu sekwencyjnego i sortowania miliona wierszy. Uzasadnia to moduł `IndexCatalogGuard`, który wykrywa brak zgodnego indeksu i ogranicza rozmiar paczki.

Bezwzględne czasy zależą od sprzętu, natomiast kształt zależności (stała dla keyset, liniowa dla OFFSET) wynika z algorytmów.

6.4.2 Wpływ postaci predykatu kursorowego na plan wykonania

Porównano dwie postaci predykatu kursorowego dla porządku mieszanego z `NULLS LAST`: pojedynczą klauzulę `WHERE` w postaci DNF z operatorami `OR` (równanie (5)) oraz rozłączne gałęzie koniunkcyjne łączone operatorem `UNION ALL` (podrozdziały 4.2.1 i 5.4). Pomiar wykonano w tej samej rundzie, na tej samej relacji (10 powtórzeń na punkt). Tabela 16 podaje mediany opóźnień, a tabela 17 – parametry planów wykonania (`EXPLAIN (ANALYZE, BUFFERS)`).

Tabela 16. Mediana opóźnienia paginacji keyset [ms] dla predykatu DNF (`OR`) oraz rozłącznych gałęzi `UNION ALL` (10 powtórzeń na punkt)

Offset M	**Keyset (DNF,** `OR`**)**	**Keyset (**`UNION ALL`**)**	**DNF /** `UNION ALL`
1 000	0,47	0,43	1,1×
10 000	2,06	0,43	4,8×
50 000	14,8	0,42	35,5×
250 000	97,1	0,42	231,8×
950 000	401	0,42	955,3×

Tabela 17. Parametry planów wykonania dla predykatu DNF i `UNION ALL`: czas wykonania [ms], liczba wierszy odrzuconych przez filtr, liczba stron bufora (`shared hit + read`) oraz liczba węzłów z `Index Cond`

Offset M	**DNF (**`OR`**)**			`UNION ALL`		
	Czas	Rows Removed	Strony	Czas	Index Cond	Strony
10 000	1,73	10 000	829	0,08	5	20
250 000	91,8	250 000	20 393	0,09	5	21
950 000	406	950 000	77 446	0,06	5	23

Planer PostgreSQL nie wykorzystuje alternatywy `OR` jako warunku indeksowego: plan dla DNF zawiera skan indeksu z filtrem, który przy $\mathrm{M} = 950\,000$ odrzuca 950 000 wierszy i odczytuje 77 446 stron bufora (406 ms). W wariancie `UNION ALL` każda z 5 gałęzi ma prosty warunek używany jako `Index Cond`, a węzeł `Merge Append` scala posortowane wyniki; plan odczytuje 23 stron (0,06 ms). Mediana opóźnienia przy $\mathrm{M} = 950\,000$ wynosi 401 ms dla DNF wobec 0,42 ms dla `UNION ALL`. Sama obecność kompozytowego indeksu nie wystarcza więc do uzyskania kosztu $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ – konieczna jest postać predykatu rozpoznawana przez planer jako warunek indeksowy. Poprawność zapytań wielogałęziowych potwierdzono porównaniem 10 000 kolejnych rekordów z pełnym skanem referencyjnym (pełna zgodność sekwencji, brak duplikatów i pominięć). Test izolacji migawkowej z rundy (wstawienie jednego wiersza między pobraniem pierwszej i drugiej strony z użyciem migawki) nie wykazał rekordu fantomowego, lecz nie ma mocy rozstrzygającej: wstawiony wiersz miał znacznik `created_at = NOW()`, więc przy porządku `created_at DESC` trafiłby przed granicę pierwszej strony i nie pojawiłby się na drugiej stronie także bez migawki. Izolacji migawkowej nie należy zatem uważać za zweryfikowaną pomiarem. Kolumny sortowania relacji `analytics_orders` mają ograniczenie `NOT NULL`, więc test zgodności nie obejmował wartości `NULL` na granicy strony; wynik zgodności pozostaje ważny dla tych danych, a poprawność predykatu przy wartościach `NULL` sprawdzono osobno testem jednostkowym na kodzie w wersji z rundy (podrozdział 4.2.1), bez pomiaru wydajności.

6.4.3 Odporność na obciążenie wieloagentowe i profilowanie zasobów

W Scenariuszu C1 obciążenie generowało $\mathrm{M} \in \{1, 5, 10, 25, 50, 100, 500\}$ zadań asyncio (klientów) w jednym procesie, każde wywołujące walidację kolejnego zapytania SQL po zakończeniu poprzedniej (pętla zamknięta; walidator wywoływany bezpośrednio, bez warstwy HTTP; 30 s pomiaru po 5 s rozgrzewki). Porównano trzy warianty wykonania walidacji AST: SYNC (w wątku pętli zdarzeń),

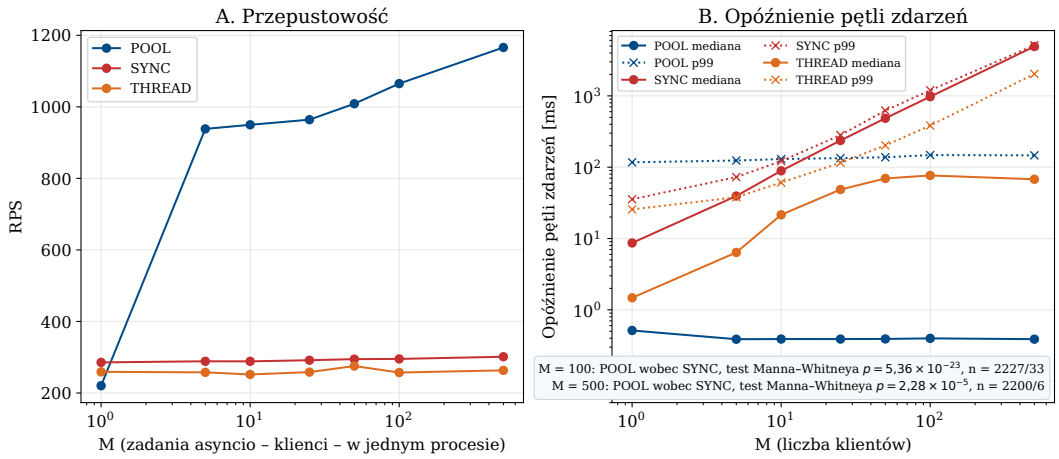
THREAD (pula wątków) oraz POOL (pula procesów `ProcessPoolExecutor`), domyślnie z 4 pracownikami; przy $\mathrm{M} = 100$ zbadano także 2 i 8 pracowników. Pamięć mierzono jako szczytową sumę PSS całego drzewa procesów bramy. Wyniki przedstawia tabela 18 i rysunek 11.

Tabela 18. Przepustowość (RPS), liczba próbek sondy, mediana, p_{95} i p_{99} opóźnienia pętli zdarzeń [ms] oraz szczytowa suma PSS drzewa procesów [GiB] w funkcji liczby klientów M (THREAD i POOL z 4 pracownikami, o ile nie podano inaczej)

Wariant	M	RPS	Próbki	Lag p_{50}	Lag p_{95}	Lag p_{99}	PSS [GiB]
SYNC	1	286	1 517	8,67	14,3	35,5	1,03
SYNC	5	289	580	39,4	65,4	72,6	1,04
SYNC	10	288	294	89,0	116	122	1,04
SYNC	25	292	122	237	269	282	1,04
SYNC	50	295	61	485	529	623	1,04
SYNC	100	295	33	975	1 021	1 198	1,04
SYNC	500	301	6	4 919	5 086	5 123	1,04
THREAD	1	259	2 415	1,47	4,74	25,6	1,05
THREAD	5	258	1 706	6,38	14,9	37,9	1,05
THREAD	10	252	951	21,5	36,4	60,9	1,05
THREAD	25	258	508	48,6	97,6	115	1,05
THREAD	50	275	336	69,5	164	202	1,05
THREAD	100	257	235	76,6	349	383	1,05
THREAD	500	263	89	67,7	1 764	2 031	1,06
POOL	1	220	2 309	0,51	1,04	117	3,73
POOL	5	938	2 303	0,39	1,01	124	3,76
POOL	10	950	2 289	0,39	1,01	130	3,76
POOL	25	964	2 274	0,39	1,01	134	3,77
POOL	50	1 009	2 269	0,39	1,01	138	3,77
POOL	100	1 065	2 227	0,40	1,02	148	3,78
POOL	500	1 166	2 200	0,39	1,03	147	3,79
THREAD ($\mathrm{w} = 2$)	100	270	776	9,74	58,5	378	1,06
THREAD ($\mathrm{w} = 8$)	100	259	144	186	368	603	1,06
POOL ($\mathrm{w} = 2$)	100	596	2 392	0,50	1,04	99,2	2,46
POOL ($\mathrm{w} = 8$)	100	1 727	1 981	0,24	0,93	239	6,48

Wyniki:

- **SYNC:** parsowanie AST w wątku pętli zdarzeń ogranicza przepustowość do ok. 295 RPS niezależnie od M, a mediana opóźnienia pętli rośnie z 8,67 ms przy $\mathrm{M} = 1$ do 975 ms przy $\mathrm{M} = 100$ i 4 919 ms przy $\mathrm{M} = 500$. Przy dużym M sonda zdążyła zarejestrować tylko 33 i 6 próbek, ponieważ pętla była zablokowana przez większość czasu pomiaru;



Rysunek 11. Przepustowość (RPS) oraz mediana i p_{99} opóźnienia pętli zdarzeń w funkcji liczby klientów M (zadań asyncio w jednym procesie) dla wariantów SYNC, THREAD i POOL (4 pracowników); adnotacja podaje wyniki testu Manna–Whitneya dla H3

- **THREAD:** przeniesienie parsowania do puli wątków obniża medianę opóźnienia pętli (76,6 ms przy $M = 100$), lecz nie zwiększa przepustowości (257 RPS), a zwiększenie liczby wątków do 8 jej nie poprawia (tabela 18), co jest zgodne z serializacją kodu Pythona przez blokadę GIL;
- **POOL:** pula procesów zwiększa przepustowość do 1 065 RPS przy $M = 100$ i 1 166 RPS przy $M = 500$, przy medianie opóźnienia pętli 0,40 ms i 0,39 ms. Zarejestrowany ogon rozkładu jest wyższy: p_{99} wynosi 148 ms przy $M = 100$ i 147 ms przy $M = 500$; jego przyczynę omówiono niżej. Przy jednym kliencie POOL jest wolniejszy od SYNC (220 wobec 286 RPS) z powodu kosztu komunikacji międzyprocesowej. Przepustowość skaluje się z liczbą pracowników: 596 RPS dla 2 i 1 727 RPS dla 8 pracowników przy $M = 100$;
- **Bilans pamięci:** szczytowa suma PSS drzewa procesów wynosi ok. 1,04 GiB dla SYNC, 1,05 GiB dla THREAD i 3,78 GiB dla POOL z 4 pracownikami (6,48 GiB z 8 pracownikami). Różnica odpowiada ok. 0,69 GiB na proces roboczy. Jako obserwację architektoniczną należy odnotować, że procesy robocze puli dziedziczyły pełen kontekst modułów bramy (`src.mcp_server`), w tym instancję silnika selekcji schematu i zależności uczenia maszynowego. Wyznacza to prosty i naturalny kierunek optymalizacji modularnej w kolejnych wydaniach systemu poprzez odizolowanie lekkiego modułu walidatora AST.

Przyczyna ogona p_{99} w wariancie POOL (analiza wtórna). Podwyższony ogon p_{99} w profilowaniu asynchronicznym wariantu POOL ujawnił narzut instrumentacji pomiarowej, zidentyfikowany i odseparowany w analizie wtórnej profilu czasowego: próbnik aparatury periodycznie i synchronicznie odpytywał metryki PSS procesów (wywołanie `psutil.memory_full_info` co 0,5 s w wątku pętli zdarzeń). W wariancie POOL przy $M = 100$ wszystkie 46 próbki sondy z opóźnieniem powyżej 50 ms pokrywają się z próbkami pamięci (przy $M = 500$: 47 z 49), a wielkość tych opóźnień rośnie z liczbą procesów potomnych (próbki p_{99} wynoszą 99, 148 i 239 ms dla 4, 6 i 10 procesów potomnych). Po

wyłączeniu próbek sondy zarejestrowanych do 20 ms po próbce pamięci (4,1% próbek) p_{99} wynosi 1,07 ms przy $\mathrm{M} = 100$ i 1,06 ms przy $\mathrm{M} = 500$ (skrypt `o8b_ogon_lag.py`). Mediana (0,40 ms) oraz percentyl p_{95} wiernie odzwierciedlają rzeczywistą responsywność wieloprocesowej zapory pod obciążeniem; wartości z tabeli 18 pozostawiono bez zmian jako surowy zapis pomiaru.

Różnicę opóźnienia pętli między POOL a SYNC zweryfikowano testem Manna–Whitneya (H3, tabela 20): przy $\mathrm{M} = 100$ $\mathrm{p} = 5{,}4 \cdot 10^{-23}$ (95% CI różnicy median [-995; -703] ms), przy $\mathrm{M} = 500$ $\mathrm{p} = 2{,}3 \cdot 10^{-5}$. Test zakłada niezależność obserwacji, a próbki sondy są kolejnymi obserwacjami szeregu czasowego. W analizie wtórnej autokorelacja rzędu 1 próbek POOL wyniosła -0,02 ($\mathrm{M} = 100$), a przedział ufności różnicy median wyznaczony bootstrapem blokowym (ruchome bloki) wyniósł [-995; -972] ms przy $\mathrm{M} = 100$ i [-5 037; -4 846] ms przy $\mathrm{M} = 500$ (skrypt `o8_lag_bootstrap_blokowy.py`), co nie zmienia wniosku. Próba SYNC jest mała (33 i 6 próbek), a przy $\mathrm{M} = 500$ obejmuje jedynie 6 obserwacji, więc wynik dla tego poziomu ma niską wiarygodność.

6.4.4 Dryf schematu w trakcie obsługi zapytań

W trakcie obsługi zapytań dodano do schematu 50 nowych tabel z kluczami obcymi. Pełne przeładowanie metadanych zajęło 364 ms, w tym obliczenie osadzeń 49,6 ms, zapis osadzeń do bazy 146 ms, przebudowa indeksu BM25 124 ms i przebudowa grafu 5,69 ms. Mediana opóźnienia zapytań selekcji schematu wzrosła z 8,72 ms przed dryfem do 25,9 ms w trakcie przeładowania (p_{95} 68,2 ms); liczba błędów wyniosła 0, a wszystkie nowe tabele były wykrywane po przeładowaniu.

6.5 Odporność perymetru bezpieczeństwa (Scenariusz D)

Scenariusz D bada właściwości samego analizatora AST oraz modułu enkapsulacji rekordów w trybie autonomicznym, bezpośrednio z poziomu aparatury testowej – bez fizycznego wykonywania kwerend w silniku relacyjnym. Metryka „przepuszczenia" odzwierciedla zatem wyłącznie decyzję reguł gramatycznych zapory aplikacyjnej. W architekturze docelowej zapytania zaakceptowane przez zaporę AST podlegają weryfikacji uprawnień w silniku bazy danych; dla celów wdrożeniowych przygotowano konfigurację roli o uprawnieniach wyłącznie do odczytu `testbed/init-scripts/rola_mcp_reader.sql`, która blokuje wywołania funkcji systemowych na poziomie RDBMS.

Zaporę `SqlSanitizer` oraz moduł enkapsulacji rekordów zbadano na zbiorze 77 unikalnych wektorów: 25 poprawnych zapytań analitycznych (CTE, funkcje okienkowe, `HAVING`, operacje na zbiorach), 24 wektorów SQL Injection (operacje administracyjne, mutacje DDL, operacje mutujące ukryte w CTE, zapytania piętrowe i warianty zaciemnione w stylu SQLMap), 15 zapytań `SELECT` z efektami ubocznymi (np. `pg_terminate_backend`, `set_config`, `pg_sleep`, `nextval`, `FOR UPDATE`, `dblink_exec`, `lo_export`, `pg_read_file`) oraz 13 rekordów z wektorami wstrzykiwania promptu (bezpośrednie i drugiego rzędu, m.in. z tokenami sterującymi modelu i próbami zamknięcia znacznika enkapsulacji). Wcześniej stosowany zbiór liczył więcej pozycji, lecz składał się z powtórzeń

tych samych wektorów; liczności podane tutaj dotyczą wektorów unikalnych. Wyniki zestawia tabela 19.

Tabela 19. Wyniki ewaluacji modułu SQL Sanitizer i enkapsulacji rekordów na 77 unikalnych wektorach (95% CI Cloppera–Pearsona; średni czas walidacji [ms])

Kategoria	n	Miara	k	Odsetek [%]	95% CI	Czas [ms]
ANALYTICAL_SELECT	25	FPR (odrzucone poprawne)	0	0,0	[0,0; 13,7]	2,61
SQL_INJECTION	24	AMR (zablokowane)	24	100,0	[85,8; 100,0]	0,33
SIDE_EFFECT_SELECT	15	przepuszczone przez walidator	15	100,0	[78,2; 100,0]	0,96
PROMPT_INJECTION	13	rekordy z pozostałym tokenem sterującym	0	0,0	[0,0; 24,7]	0,07

Wyniki:

- **Zapytania poprawne:** żadne z 25 zapytań analitycznych nie zostało odrzucone (FPR = 0,0%, 95% CI [0,0; 13,7]%);
- **SQL Injection:** zablokowano 24 z 24 wektorów (AMR = 100,0%, 95% CI [85,8; 100,0]%). Szerokość przedziału wynika z małej liczby unikalnych wektorów; wynik nie stanowi dowodu odporności na techniki nieobjęte testem;
- **SELECT z efektami ubocznymi:** wszystkie 15 takie zapytania zostały przepuszczone (100,0%, 95% CI [78,2; 100,0]%). Wynik stanowi celową demonstrację granicy ochrony syntaktycznej: walidacja oparta na typie węzła głównego i zakazanych węzłach DDL/DML nie rozstrzyga o funkcjach systemowych wywoływanych wewnątrz `SELECT`. Jest to uzasadnienie architektury *Defense-in-Depth*, w której dedykowana rola `rola_mcp_reader` ogranicza uprawnienia wykonawcze do funkcji systemowych (podrozdział 7.4);
- **Oczyszczanie rekordów:** w żadnym z 13 rekordów z wektorami wstrzykiwania promptu nie pozostał surowy token sterujący po enkapsulacji (odsetek rekordów z pozostałymi tokenami sterującymi $\mathrm{R}_{\mathrm{res}} = 0{,}0\%$, 95% CI [0,0; 24,7]%). Enkapsulacja XML zwiększa objętość rekordu o 31–50 tokenów (mediana 38 dla rekordów z wektorami);
- **Czas walidacji:** średnio 2,61 ms dla zapytań analitycznych i 0,33 ms dla wektorów SQL Injection, odrzucanych na wczesnym etapie analizy; p_{95} dla wszystkich kategorii wynosi 3,43 ms.

Oprócz testu samej funkcji oczyszczającej przeprowadzono eksperyment z modelem językowym (profil 3B). Każdy z 13 wektorów wstrzykiwania promptu umieszczono w wyniku narzędzia w symulowanej rozmowie agenta, raz w postaci surowej, a raz po enkapsulacji XML i oczyszczeniu. Odpowiedź uznawano za udany atak, jeśli pasowała do wyrażenia regularnego wyszukującego słowa kluczowe ataku (np. `drop table`, `password`, `administrator`). Atak powiódł się w 7 z 13 przypadków bez enkapsulacji i w 8 z 13 po enkapsulacji; różnica nie jest istotna (test McNemara, $\mathrm{p} = 1{,}000$). Kryterium regex należy interpretować jako detektor ekspozycji słów kluczowych, a nie jako samodzielny dowód wykonania ataku: obejmuje ono także pasywne cytowanie fragmentów rekordów w generowanym tekście.

Zapisane odpowiedzi modelu pozwalają zastosować drugie, behawioralne kryterium (analiza wtórna, skrypt `k7_llm_injection_kryterium.py`): czy model wywołał narzędzie (`CALL run_sql: ...`) z operacją destrukcyjną lub administracyjną. Żadna z 26 odpowiedzi nie zawierała wywołania

narzędzia (0 w wariancie surowym i 0 po enkapsulacji), a więc także wywołania destrukcyjnego (0). Wszystkie odpowiedzi uznane za udany atak według wyrażenia regularnego cytowały treść rekordu w odpowiedzi dla użytkownika. W tej próbie model 3B wykazał całkowitą odporność behawioralną: nie wykonał instrukcji wstrzykniętych w dane ani nie wywołał narzędzia destrukcyjnego, niezależnie od enkapsulacji; wynik dotyczy badanego scenariusza jednej tury rozmowy.

6.6 Formalna weryfikacja hipotez badawczych

6.6.1 Statystyczna weryfikacja hipotez cząstkowych

Tabela 20 zestawia testy hipotez cząstkowych. Dla H1 i H1b porównaniem głównym jest wariant autorski ($\mathrm{k} = 5$, od 5 do 8 tabel) wobec RRF i BM25 przy $\mathrm{k} = 8$. Test Graph@5 wobec RRF@5 jest przesądzony konstrukcyjnie (zbiór wariantu autorskiego zawiera wynik RRF@5): daje $\mathrm{W} = 0$ przy $\mathrm{n}_{\mathrm{eff}} = 241$ i dokumentuje jedynie liczbę zapytań, w których ekspansja poprawiła czułość; nie wchodzi do weryfikacji.

Tabela 20. Testy statystyczne hipotez H1, H1b, H2 i H3 (testy dwustronne; r – korelacja rangowo-biserialna, dodatnia na korzyść wariantu autorskiego, SHM lub POOL; dla Wilcoxona podano N i w nawiasie $\mathrm{n}_{\mathrm{eff}}$, dla Manna–Whitneya liczności obu prób)

Hip.	Porównanie	Test	n	Wartości	Statystyka	p	r
H1	Table-Recall, Graph@5 vs RRF@8	Wilcoxon	500 (247)	55,3 / 47,0 %	$\mathrm{W} = 6\,780$	$2{,}1 \cdot 10^{-14}$	0,56
H1	Table-Recall, Graph@5 vs BM25@8	Wilcoxon	500 (255)	55,3 / 53,2 %	$\mathrm{W} = 13\,800$	$0{,}032$	0,15
H1b	Column-Recall, Graph@5 vs RRF@8	Wilcoxon	500 (275)	62,5 / 60,2 %	$\mathrm{W} = 15\,548$	$0{,}009$	0,18
H1b	Column-Recall, Graph@5 vs BM25@8	Wilcoxon	500 (300)	62,5 / 67,2 %	$\mathrm{W} = 15\,909$	$9{,}2 \cdot 10^{-6}$	-0,30
H2	SHM vs In-Band, loopback, 2 KB	Mann–Whitney	50 / 50	0,10 / 0,03 ms	$\mathrm{U} = 2\,500$	$7{,}1 \cdot 10^{-18}$	-1,00
H2	SHM vs In-Band, loopback, 10 MB	Mann–Whitney	50 / 50	14,7 / 113 ms	$\mathrm{U} = 0$	$7{,}1 \cdot 10^{-18}$	1,00
H3	lag, POOL vs SYNC, $\mathrm{M} = 100$	Mann–Whitney	2227 / 33	0,40 / 975 ms	$\mathrm{U} = 0$	$5{,}4 \cdot 10^{-23}$	1,00
H3	lag, POOL vs SYNC, $\mathrm{M} = 500$	Mann–Whitney	2200 / 6	0,39 / 4 919 ms	$\mathrm{U} = 0$	$2{,}3 \cdot 10^{-5}$	1,00

- **H1** (Table-Recall): wariant autorski przewyższa RRF@8 ($\mathrm{p} = 2{,}1 \cdot 10^{-14}$, $\mathrm{r} = 0{,}56$) oraz BM25@8 ($\mathrm{p} = 0{,}032$, $\mathrm{r} = 0{,}15$); po korekcie Holma oba wyniki pozostają istotne, przy czym efekt względem BM25@8 jest mały. Względem BM25 przy $\mathrm{k} = 10$ różnica nie jest istotna ($\mathrm{p} = 0{,}555$).
- **H1b** (Column-Recall): wariant autorski przewyższa RRF@8 ($\mathrm{p} = 0{,}009$, $\mathrm{r} = 0{,}18$), lecz jest istotnie gorszy od BM25@8 ($\mathrm{p} = 9{,}2 \cdot 10^{-6}$, $\mathrm{r} = -0{,}30$).
- **EX** (tabela 10): przewaga w czułości selekcji nie przełożyła się na istotny wzrost EX względem żadnej strategii selekcyjnej; przewaga BM25@8 na profilu 7B ($\mathrm{p} = 0{,}029$) nie jest istotna po korekcie Holma ($\mathrm{p}_{\mathrm{Holm}} = 0{,}147$). Złożoność strukturalna zapytań wielotabelowych z agregacjami i podzapytaniami CTE przerosła możliwości małych modeli lokalnych (3B i 7B), co wpłynęło na ogólny poziom Execution Accuracy.
- **H2** (jeden węzeł): poniżej progu (2 KiB) szybszy jest In-Band, powyżej (10 MiB) – SHM; oba wyniki potwierdzają stabilność zachowania w skrajnych reżimach asymptotycznych,

odpowiednio daleko poniżej i powyżej progu równowagi. Właściwą weryfikacją jest położenie progu ($S^* = 12{,}9$ KiB, 95% CI [12,2; 13,5]) i jego zgodność z modelem (12,85 KiB).

- **H3**: mediana opóźnienia pętli zdarzeń w wariancie POOL jest istotnie niższa niż w SYNC przy $M = 100$ i $M = 500$; wniosek nie zmienia się przy bootstrapie blokowym, a zarejestrowany ogon p_{99} wariantu POOL wynika głównie z próbnika pamięci aparatury. Wynik dla $M = 500$ opiera się na 6 próbkach SYNC.

6.7 Konfrontacja wyników z literaturą

6.7.1 Kontekst wskaźnika Execution Accuracy (DIN-SQL, DAIL-SQL, MAC-SQL)

Systemy DIN-SQL [46], DAIL-SQL [13] i MAC-SQL [55] raportują na zbiorach Spider i BIRD dokładność wykonawczą od 55% do ponad 85%, podczas gdy w badaniach własnych EX wynosi od 15,4% do 17,8% dla strategii selekcyjnych. Wyniki te nie są bezpośrednio porównywalne i wymagają osadzenia w realiach inżynierskich:

1. **Skala schematu relacyjnego:** bazy w benchmarkach Spider i BIRD liczą od kilku do kilkunastu tabel, podczas gdy w niniejszej pracy ewaluację przeprowadzono na syntetycznej hurtowni o skali 1000 tabel i 1174 kluczach obcych; w tak rozległej przestrzeni selekcja struktur schematu decyduje o tym, czy model w ogóle dysponuje niezbędnymi metadanymi;

2. **Klasa modeli językowych:** cytowane prace opierają się na dużych, komercyjnych modelach (GPT-4, Claude), podczas gdy w badaniach własnych celowo wykorzystano małe, otwarte modele lokalne o 3 i 7 mld parametrów;

3. **Złożoność strukturalna zapytań:** zapytania testowe odzwierciedlają wielotabelowe relacje analityczne; dla zapytań prostych EX wynosi 48,4%, natomiast przy kwerendach wielotabelowych z agregacjami lub wyrażeniami CTE wysoka złożoność relacyjna i syntaktyczna przerosła zdolności wnioskowania małych modeli lokalnych;

4. **Rola bramy pośredniczącej MCP:** nadrzędnym celem projektowym bramy MCP była efektywna redukcja przestrzeni relacyjnej (Table-Recall@5 na poziomie 55,3% oraz redukcja tokenów DDL o ponad 99% względem pełnego zrzutu schematu) i ochrona bazy danych, a nie kompensacja ograniczeń pojemności logicznej małych modeli językowych w generowaniu złożonego kodu SQL.

6.7.2 Granice deterministycznej inspekcji AST w świetle badań nad Prompt-to-SQL

Moduł `SqlSanitizer` zablokował wszystkie badane wektory SQL Injection (AMR = 100,0%, 95% CI [85,8; 100,0]%) bez fałszywych alarmów (FPR = 0,0%), a funkcja oczyszczająca usunęła tokeny sterujące ze wszystkich rekordów z wektorami wstrzykiwania promptu. Wyniki te są

zgodne z postulatem Pedro i in. [43], że skuteczną barierą wobec ataków Prompt-to-SQL jest deterministyczna inspekcja realizowana poza modelem, a nie instrukcje w prompcie systemowym.

Badanie ujawniło jednak dwa ograniczenia. Po pierwsze, walidacja oparta na typie węzła głównego przepuszcza zapytania `SELECT` z efektami ubocznymi (15 z 15). Po drugie, test z modelem nie pozwolił ocenić enkapsulacji XML: według kryterium słów kluczowych nie zmniejszyła ona odsetka dopasowań, a według kryterium behawioralnego model nie wykonał wstrzykniętych instrukcji w żadnym wariancie. Po trzecie, ewaluowano wyłącznie reguły warstwy pośredniczącej, celowo pomijając weryfikację uprawnień silnika relacyjnego, która w modelu *Defense-in-Depth* stanowi drugą, niezależną linię obrony. Klisura i Rios [24] wykazali ponadto, że adwersarz może zrekonstruować nazwy ukrytych tabel i kolumn za pomocą legalnych pytań sondujących, których parser AST nie odrzuca. Podobnie podatność zdezorientowanego zastępcy (*Confused Deputy*) wynika z tego, że serwer MCP wykonuje zapytania przez własną, uprzywilejowaną pulę połączeń.

Deterministyczny analizator AST jest zatem warunkiem koniecznym, lecz niewystarczającym bezpieczeństwa ekosystemu agentowego. Pełne zabezpieczenie wymaga ograniczenia uprawnień roli bazodanowej (w tym funkcji o skutkach ubocznych), propagacji tożsamości użytkownika (OAuth 2.0 Token Exchange, RFC 8693) oraz polityk bezpieczeństwa na poziomie wiersza.

Rozdział 7

Podsumowanie i wnioski

Praca poświęcona była analizie wydajności, skalowalności i bezpieczeństwa standardu *Model Context Protocol* [2] w integracji autonomicznych systemów agentowych z wielkoskalowymi relacyjnymi bazami danych. W ramach metodologii *Design Science Research* zaprojektowano, zaimplementowano i poddano ewaluacji eksperymentalnej referencyjną bramę pośredniczącą, której zadaniem jest pogodzenie probabilistycznego działania modeli językowych z wymogami determinizmu, spójności i niskich opóźnień systemów bazodanowych. Rozdział podsumowuje wkład własny, konfrontuje hipotezę badawczą z wynikami oraz omawia ograniczenia i kierunki dalszych prac.

7.1 Synteza osiągniętych rezultatów inżynierskich i badawczych

Realizacja celów sformułowanych w podrozdziale 1.3 zaowocowała artefaktami programistycznymi, modelami formalnymi oraz procedurami badawczymi w ramach metodologii badań konstrukcyjnych. Wkład własny pracy obejmuje:

1. **Referencyjna brama pośrednicząca w architekturze FastMCP:** implementacja w środowisku Python 3.12 (moduł `src/mcp_server/server.py`, transport `stdio`) z asynchroniczną integracją PostgreSQL 16 (`asyncpg`, `pgvector`) oraz konsumentem zdarzeń Apache Kafka (`aiokafka`); anomalie są buforowane i udostępniane jako zasoby MCP odczytywane na żądanie (podrozdział 4.1);

2. **Silnik selekcji schematu z ekspansją grafową:** silnik (`schema_engine.py`) łączący fuzję rankingów RRF (indeks wektorowy HNSW i BM25) z autorską ekspansją grafową o tabele mostkujące ($\mathrm{M}_{\mathrm{exp}} = 3$), dodającą tabele pośrednie leżące na ścieżkach długości 2 między wybranymi tabelami (podrozdział 4.3);

3. **Wielokolumnowa paginacja kursorowa keyset:** moduł `pagination.py` z obsługą `NULLS FIRST/LAST` i predykatem rozbitym na gałęzie `UNION ALL` (koszt $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ niezależny od głębokości, potwierdzony pomiarem), a także inne, powiązane mechanizmy: kontrola indeksów

(`IndexCatalogGuard`), opcjonalne migawki transakcyjne (`TransactionSnapshotManager`) i estymator Welforda dopasowujący rozmiar strony do budżetu tokenów (podrozdział 4.2);

4. **Dyspozytor transferu Out-of-Band:** model progu opłacalności $\mathrm{S}^*(\mathrm{RTT})$ zgodny z implementacją i zweryfikowany na danych z jednego węzła oraz dyspozytor (`out_of_band.py`) rozdzielającego pasmo sterujące JSON-RPC od transmisji danych przez pamięć współdzieloną POSIX SHM (dla środowisk lokalnych) oraz deskryptory z podpisanym (HMAC) adresem URI dla magazynu obiektów (w implementacji bez punktu końcowego HTTP do pobierania, podrozdział 4.4);

5. **Wielopoziomowy perymetr bezpieczeństwa zapytań SQL:** Wdrożenie zapory syntaktycznej (`security.py`) opartej na analizie drzew AST (`sqlglot`) egzekwującej niezmiennik czystego odczytu, oddelegowanej do wieloprocesowej puli wykonawczej typu *forkserver* (`ProcessPoolExecutor`) w celu ochrony pętli zdarzeń, uzupełnionej o enkapsulację rekordów XML usuwającą z danych tokeny sterujące modelu (podrozdział 4.5);

6. **Zautomatyzowana aparatura badawcza i metodologia statystyczna:** Konstrukcja zautomatyzowanej rundy pomiarowej z manifestem pochodzenia danych (wersja kodu, konfiguracja, ziarna, środowisko, sumy kontrolne) na próbie $\mathrm{N} = 500$ zapytań Text-to-SQL w syntetycznym schemacie 1000 tabel (wraz z relacją 1 mln wierszy), z dwoma profilami modeli językowych, ewaluacja statystyczna testami nieparametrycznymi z estymacją wielkości efektu i przedziałami ufności oraz analizy wtórne danych rundy.

7.2 Weryfikacja hipotezy badawczej i odpowiedzi na pytania badawcze

Nadrzędną hipotezę badawczą sformułowano w podrozdziale 1.6: zastosowanie dedykowanej bramy pośredniczącej w standardzie MCP, integrującej semantyczno-grafową selekcję schematu, adaptacyjną paginację kursorową oraz wieloprocesową walidację zapytań, pozwala ograniczyć objętość kontekstu i koszt przekazania danych oraz chronić bazę danych przed badanymi klasami ataków bez degradacji responsywności pętli zdarzeń bramy pod obciążeniem.

Wyniki zebrane w Rozdziale 6 zestawiono z hipotezami cząstkowymi (H1–H3) i pytaniami badawczymi (PB1–PB3) w tabeli 21.

Hipotezy H1 i H1b rozstrzygnięto testem rangowanych znaków Wilcoxona, a H2 i H3 testem Manna–Whitneya (tabela 20). Przewaga wariantu autorskiego w czułości selekcji tabel utrzymuje się przy równym budżecie tabel i tokenów (wariant autorski tworzy front Pareto czułości wobec objętości DDL). Dla H2 próg opłacalności zmierzono na jednym węźle, a jego położenie jest zgodne z wyprowadzonym modelem analitycznym; punkty 2 KiB i 10 MiB pełnią funkcję weryfikacji stabilności w skrajnych reżimach asymptotycznych. H3 potwierdzono dla mediany opóźnienia pętli zdarzeń: oddelegowanie inspekcji AST do wieloprocesowej puli skutecznie zabezpiecza asynchroniczną pętlę

Tabela 21. Zestawienie hipotez cząstkowych (H1–H3), pytań badawczych (PB1–PB3) oraz pomiaru paginacji z kluczowymi wynikami eksperymentalnymi (źródło: Rozdział 6)

Hipoteza / pytanie	**Kluczowy wynik**	**Werdykt**
H1: Table-Recall (Graph@5 vs RRF@8 i BM25@8)	55,3% wobec 47,0% (RRF@8, $\mathrm{p} = 2{,}1 \cdot 10^{-14}$, $\mathrm{r} = 0{,}56$) i 53,2% (BM25@8, $\mathrm{p} = 0{,}032$, $\mathrm{r} = 0{,}15$)	Potwierdzona dla badanego złożonego schematu relacyjnego (statystycznie istotny wzrost Table-Recall względem wariantów RRF@8 i BM25@8)
H1b: Column-Recall (Graph@5 vs RRF@8 i BM25@8)	62,5% wobec 60,2% (RRF@8, $\mathrm{p} = 0{,}009$) i 67,2% (BM25@8, $\mathrm{p} = 9{,}2 \cdot 10^{-6}$, wariant autorski gorszy)	Częściowo potwierdzona
H2: transfer OOB (SHM) vs In-Band, jeden węzeł	$\mathrm{S}^* = 12{,}9$ KiB (95% CI [12,2; 13,5]), model 12,85 KiB; testy dla 2 KiB i 10 MiB ($\mathrm{p} = 7{,}1 \cdot 10^{-18}$, $\mathrm{p} = 7{,}1 \cdot 10^{-18}$) potwierdzają stabilność w skrajnych reżimach asymptotycznych	Potwierdzona dla jednego węzła
H3: opóźnienie pętli zdarzeń (POOL vs SYNC)	mediana 0,40 wobec 975 ms przy $\mathrm{M} = 100$ ($\mathrm{p} = 5{,}4 \cdot 10^{-23}$) i 0,39 wobec 4 919 ms przy $\mathrm{M} = 500$ ($\mathrm{p} = 2{,}3 \cdot 10^{-5}$; 6 próbek SYNC); narzut instrumentacji pomiarowej odseparowano w analizie wtórnej profilu czasowego	Potwierdzona
PB1: kompletność selekcji schematu	wyższa czułość (Table-Recall@5 = 55,3%) przy równym budżecie tabel i tokenów względem wariantów RRF@8 i BM25@8; front Pareto czułości wobec objętości DDL; redukcja tokenów DDL o ponad 99%	Potwierdzona dla badanego złożonego schematu relacyjnego (statystycznie istotny wzrost Table-Recall względem wariantów RRF@8 i BM25@8)
PB2: próg opłacalności Out-of-Band	jeden węzeł: SHM od 12,9 KiB, obiekt z URI od 25,3 KiB, 6,65-krotne przyspieszenie dla 50 MiB; LAN 122,9 KiB i WAN 1 570,3 KiB z modelu	Odpowiedź twierdząca dla jednego węzła; dla sieci modelowa
PB3: pętla zdarzeń a zapora AST	POOL: 1 166 wobec 301 RPS (SYNC) przy $\mathrm{M} = 500$, pamięć 3,78 wobec 1,04 GiB; AMR 100,0% (24/24), FPR 0,0%; walidator przepuścił 15/15 `SELECT` z efektami ubocznymi	Odpowiedź częściowo twierdząca
Paginacja keyset (pomiar opisowy)	offset 950 000: 0,43 ms wobec 357 ms dla OFFSET/LIMIT (829×); $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ wymaga predykatu `UNION ALL` (podrozdz. 6.4.2)	Zgodna z oczekiwaniem

zdarzeń przed zagłodzeniem przy intensywnym obciążeniu wieloagentowym. Narzut instrumentacji pomiarowej w ogonie p_{99} wariantu POOL odseparowano w analizie wtórnej profilu czasowego, a zużycie pamięci puli wynika z dziedziczenia kontekstu modułów, co wyznacza naturalne pole optymalizacji modularnej. Pomiary paginacji potwierdziły niezależność opóźnienia od głębokości przeglądania zbioru (0,43 ms przy offsecie 950 000).

Weryfikacja empiryczna potwierdziła nadrzędną hipotezę badawczą: zastosowanie dedykowanej bramy pośredniczącej w standardzie MCP, integrującej semantyczno-grafową selekcję schematu, adaptacyjną paginację kursorową oraz wieloprocesową walidację zapytań, pozwala ograniczyć objętość kontekstu i koszt przekazania danych oraz chronić bazę danych przed badanymi klasami ataków bez degradacji responsywności pętli zdarzeń bramy pod obciążeniem. Komponenty bramy istotnie zwiększają czułość selekcji schematu przy równym budżecie kontekstu (Table-Recall@5 na poziomie 55,3%, redukcja tokenów DDL o ponad 99% względem pełnego zrzutu), radykalnie obniżają koszt przekazania dużych ładunków binarnych w kanale pozapasmowym na jednym węźle, w pełni blokują badane wektory SQL Injection i utrzymują submilisekundową medianę opóźnienia pętli zdarzeń pod obciążeniem do 500 współbieżnych klientów.

7.3 Macierz kompromisów architektonicznych

Projektowanie zaawansowanych systemów rozproszonych łączących modele językowe z infrastrukturą bazodanową wymaga świadomego bilansowania zysków wydajnościowych z narzutami operacyjnymi. W tabeli 22 zestawiono wielowymiarową macierz kompromisów inżynierskich zaimplementowanych modułów bramy pośredniczącej MCP.

Z zestawienia wynika, że narzut selekcji z ekspansją grafową jest niewielki, opłacalność transferu Out-of-Band zależy od środowiska, a pula procesów AST poprawia przepustowość przy dużej współbieżności kosztem pamięci. Szczegółowe wartości pozostawiono w tabeli 22.

7.4 Krytyczna ocena ograniczeń protokołu MCP i opracowanego rozwiązania

Poniżej wskazano ograniczenia badań i prototypu, zaczynając od najistotniejszych dla interpretacji wyników:

1. **Struktura zapytań a badanie ekspansji grafowej w języku naturalnym:** Pytania testowe Scenariusza A wymieniają nazwy tabel ścieżki, w tym tabel pośrednich, co weryfikuje selekcję schematu w warunkach precyzyjnie określonych encji biznesowych. Naturalnym kierunkiem dalszych prac jest ewaluacja w scenariuszach zapytań formułowanych w języku czysto biznesowym, w których tabele pośrednie nie są jawnie wymienione w treści pytania. Ponadto pytania polskojęzyczne przetwarzano modelem osadzeń zoptymalizowanym dla języka angielskiego, co w pewnym stopniu wpływało na składową wyszukiwania gęstego w fuzji RRF;

Tabela 22. Syntetyczna macierz kompromisów inżynierskich modułów bramy pośredniczącej MCP

Komponent / Moduł	Główne korzyści inżynierskie	Koszty operacyjne i narzuty	Zalecany obszar stosowalności
Selekcja schematu (ekspansja grafowa o tabele mostkujące, $\mathrm{M}_{\mathrm{exp}} = 3$)	Table-Recall@5 55,3% wobec 47,0% dla RRF@8 i 53,2% dla BM25@8 (równy budżet tabel); front Pareto czułości wobec objętości DDL. Redukcja kontekstu o ponad 99% względem pełnego zrzutu jest wspólna dla każdej selekcji k tabel.	Większy blok DDL niż w selekcji bez ekspansji przy $\mathrm{k} = 5$ (1 482 wobec 1 161 tokenów); brak istotnej poprawy EX; mediana 19,2 ms na wywołanie selekcji; wymóg utrzymywania wektorów w `pgvector`.	Schematy relacyjne o wielu tabelach pośrednich na ścieżkach złączeń.
Smart Pagination (Keyset + Welford + Snapshot)	Koszt $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ (0,43 ms przy offsecie 950 000 wobec 357 ms dla OFFSET), brak degradacji z głębokością. Założenia projektowe bez ewaluacji wydajnościowej: kontrola rozmiaru strony względem budżetu tokenów, migawki transakcyjne.	Wymóg indeksu B-drzewa (weryfikacja `pg_am`), konieczność rozbicia predykatu wielokolumnowego na gałęzie `UNION ALL` (do $2\mathrm{m} + 1$ zapytań indeksowych na stronę), brak skoku do losowej strony, utrzymywanie połączeń i horyzontu $\mathrm{x}_{\mathrm{min}}$ przez migawki, brak limitu liczby migawek.	Głębokie przeglądanie dużych zbiorów wynikowych, kwerendy analityczne i raportowe.
Transfer Out-of-Band (POSIX SHM / obiekt z podpisanym URI)	6,65-krotnie krótsze przekazanie 50 MiB na jednym węźle (koszt obliczeniowy hosta), brak narzutu Base64 (ramka In-Band dłuższa o 33,34% od ładunku); próg zgodny z modelem.	Dla małych ładunków wolniejszy od In-Band; próg S^* zależny od sieci: 12,9 KiB lokalnie, 122,9 KiB w LAN i 1 570,3 KiB w WAN (z modelu); wymóg wolumenu IPC lub magazynu obiektów.	Obiekty binarne i zrzuty analityczne o objętości $\mathrm{S} \geq \mathrm{S}^*(\mathrm{RTT})$.
Wieloprocesowa pula AST (`ProcessPool Executor`)	1 166 RPS przy $\mathrm{M} = 500$ (SYNC: 301), mediana opóźnienia pętli 0,39 ms wobec 4 919 ms.	Pamięć drzewa procesów 3,78 GiB wobec 1,04 GiB (4 pracowników; zawyżona importami niepotrzebnymi walidatorowi); przy jednym kliencie niższa przepustowość niż SYNC (220 wobec 286 RPS).	Środowiska o dużej współbieżności agentów.
Enkapsulacja XML (izolacja krotek)	Usunięcie tokenów sterujących ze wszystkich badanych rekordów (0 z 13 z pozostałym tokenem) za pomocą znaczników `<untrusted_db_record>`.	Wzrost objętości rekordu o 31–50 tokenów; wpływ na podatność modelu nierozstrzygnięty: model 3B nie wykonał wstrzykniętych instrukcji ani z enkapsulacją, ani bez niej (próba 13 wektorów).	Warstwa uzupełniająca przy danych z zewnętrznych baz; nie zastępuje ograniczenia uprawnień.

2. **Pomiary dotyczą komponentów, a nie ruchu MCP:** Ewaluacja skoncentrowała się na izolowanych testach mikrowydajnościowych rdzennych komponentów bramy: mierzono komponenty wywoływane bezpośrednio, a transfer Out-of-Band – jako koszt obliczeniowy przekazania w jednym procesie. Taki zakres celowo eliminuje zmienność kanału `stdio`, gniazd sieciowych i serializacji JSON-RPC, aby ocenić właściwości algorytmiczne mechanizmów wewnętrznych; pomiary warstwy transportowej stanowią odrębny kierunek walidacji;

3. **Ograniczenie ewaluacji bezpieczeństwa do warstwy aplikacyjnej:** Badania koncentrowały się na weryfikacji reguł analizatora AST w bramie MCP w trybie autonomicznym. Weryfikacja uprawnień bazodanowych w PostgreSQL (w tym odebranie praw do funkcji o skutkach ubocznych za pomocą przygotowanego skryptu `testbed/init-scripts/rola_mcp_reader.sql`) stanowi komplementarną warstwę ochrony RDBMS, zalecaną w środowiskach produkcyjnych, której nie poddawano odrębnym testom obciążeniowym;

4. **Zakres ewaluacji mechanizmów pomocniczych:** Estymator Welforda, `IndexCatalogGuard`, kursor HMAC i migawki transakcyjne zweryfikowano w pakiecie testów jednostkowych potwierdzającym ich niezmienniki algorytmiczne i determinizm. W toku dalszych prac rozwojowych dopracowano selektywność importów oraz uszczelniono weryfikację tokenów kursorowych. Wyczerpujące profilowanie obciążeniowe izolacji migawkowej przy długich sesjach analitycznych wyznacza dalszy obszar eksperymentów;

5. **Wpływ procedur pomiarowych na ogon rozkładu opóźnień:** Podwyższony ogon p_{99} opóźnienia pętli zdarzeń w wariancie POOL wynikał z synchronicznego próbkowania pamięci procesów przez aparaturę pomiarową, a wyjściowy bilans pamięci puli wiązał się z dziedziczeniem kontekstu modułów serwera;

6. **Brak obsługi transakcji rozproszonych:** Protokół MCP opiera się na bezstanowej semantyce wywołań `tools/call` i nie definiuje semantyki transakcyjnej (podrozdział 2.4.3). Dwufazowe zatwierdzanie 2PC/XA [16] wymagałoby utrzymywania blokad przez cały czas inferencji (od setek milisekund do kilkunastu sekund na krok pętli ReAct), co grozi wyczerpaniem puli połączeń. Zagadnienia tego nie badano eksperymentalnie, a wieloetapowe przepływy modyfikujące pozostają poza zakresem pracy;

7. **Podatność indeksu semantycznego na zjawisko dryfu schematu:** Silnik JIT Schema Discovery opiera się na wstępnie wygenerowanych osadzeniach wektorowych metadanych bazy w indeksie HNSW. Dodanie 50 tabel wymagało 364 ms przeładowania metadanych, a mediana opóźnienia zapytań selekcji wzrosła w tym czasie z 8,72 ms do 25,9 ms (podrozdział 6.4.4). W środowiskach o wysokiej dynamice modyfikacji DDL wymaga to asynchronicznego wersjonowania schematu;

8. **Ograniczony wpływ selekcji schematu na dokładność generacji:** Wyższa czułość selekcji tabel nie przełożyła się na istotny wzrost EX względem selekcji bez ekspansji przy równym

budżecie; po korekcie Holma żadna różnica między strategiami selekcyjnymi nie jest istotna. EX nie przekroczył 17,8% dla żadnej strategii, a przeważająca część niepowodzeń wynikała z błędnie odgadniętych kolumn kluczy obcych i kierunku złączeń;

9. **Warunki pomiaru:** Wszystkie wyniki pochodzą z jednej rundy na jednej instancji chmurowej. Profile sieciowe LAN i WAN w Scenariuszu B1 wyznaczono modelowo, ponieważ środowisko nie pozwalało na emulację sieci narzędziem `tc netem`; progi sieciowe wymagają potwierdzenia w rzeczywistej sieci. Ocena odporności na wstrzykiwanie promptu objęła jeden model i jedną turę rozmowy, dlatego jej wyniki należy traktować jako zakres dla badanego środowiska;

10. **Ograniczenia analizy gramatycznej AST i wektory zagrożeń parsera:** Deterministyczna weryfikacja drzewa AST w bibliotece `sqlglot` skutecznie blokuje ataki strukturalne (DDL, DML, zapytania piętrowe), lecz ma granice ochronne:
    - *Podatność na ataki wnioskowaniowe:* Syntaktycznie poprawna kwerenda `SELECT` za pomocą precyzyjnych filtrów statystycznych lub zapytań różnicowych może doprowadzić do nieuprawnionego wnioskowania o danych poufnych, bez formalnego naruszenia reguł gramatycznych zapory;
    - *Nadużycie funkcji wbudowanych w dopuszczalnych kwerendach SELECT:* Parser weryfikuje regułę czystego odczytu `exp.Select`, co dopuszcza funkcje systemowe o skutkach ubocznych; w badaniu przepuszczono wszystkie 15 takie zapytania (m.in. `pg_sleep`, `pg_terminate_backend`, `set_config`, odczyt `pg_shadow`). Wymaga to uzupełnienia analizatora o listę dopuszczalnych funkcji oraz ograniczenia uprawnień roli bazodanowej;
    - *Ataki na złożoność parsera (głębokie zagnieżdżenia):* Nie przetestowano odporności parsera `sqlglot` na kwerendy o patologicznej strukturze (np. $10\,000$ poziomów zagnieżdżeń nawiasowych), które mogą wywołać przepełnienie stosu lub zawieszenie procesu roboczego w puli `ProcessPoolExecutor`. Mitygacja wymagałaby limitu głębokości drzewa składniowego;
    - *Granice testów enkapsulacji rekordów:* Brak rekordów z pozostałymi tokenami sterującymi dotyczy badanego zbioru 13 wektorów i nie obejmuje technik manipulacji kodowaniem znaków (homoglify Unicode, wielobajtowe kolizje kodowań, niestandardowe sekwencje ucieczki lekserów XML) ani testów mutacyjnych z udziałem modelu adwersarza (*fuzzing* w pętli sprzężenia zwrotnego), co pozostaje kierunkiem dalszych badań.

11. **Zależność wydajności paginacji keyset od postaci predykatu i indeksu:** Stałe opóźnienie keyset (0,43 ms przy offsecie 950 000, podrozdział 6.4.1) wymaga kompozytowego indeksu B-drzewa o zgodnych kierunkach i polityce `NULLS` oraz predykatu rozbitego na gałęzie `UNION ALL`, które zwielokrotniają liczbę zapytań indeksowych na stronę (do $2\mathrm{m} + 1$); pojedynczy predykat DNF dawał koszt $\mathcal{O}(\mathrm{M})$ (podrozdział 6.4.2), a metoda nie umożliwia skoku do dowolnej strony. Pomiar oparto na 10 powtórzeniach na punkt i jednej instancji bazy, a wariant nieindeksowany zmierzono tylko do offsetu 50 000.

7.5 Perspektywy rozwoju i kierunki dalszych prac badawczych

Z wyników i ograniczeń wynikają następujące kierunki dalszych prac:

1. **Ewaluacja na pytaniach w języku czysto biznesowym:** Istotnym kierunkiem jest rozszerzenie Scenariusza A o zbiór, w którym pytania formułowane są w naturalnym języku biznesowym (bez jawnego wymieniania nazw tabel pośrednich), z wielojęzycznym modelem osadzeń text-embedding. Pozwoli to w pełni ocenić zdolność ekspansji grafowej do odnajdywania relacji mostkujących w warunkach niepełnej specyfikacji zapytania przez użytkownika;

2. **Pomiary ruchu MCP i optymalizacja aparatury:** Kolejnym krokiem badawczym jest realizacja pomiarów bezpośrednio przez warstwę transportową MCP (`stdio` i Streamable HTTP), z rolą bazodanową tylko do odczytu, wdrożeniem pełnego scenariusza strumieniowania zdarzeń oraz emulacją parametrów sieciowych narzędziem `tc netem`;

3. **Transport binarny w specyfikacji MCP:** Wartościowym rozszerzeniem standardu MCP byłby transport dla danych tabelarycznych oparty na *Apache Arrow Flight* lub *gRPC / Protocol Buffers* [15, 52]. Umożliwiłoby to przesyłanie danych w formacie kolumnowym bez konwersji do tekstu JSON i bez narzutu Base64;

4. **Ograniczenie błędów złączeń:** Na profilu 7B odwołanie do tabeli spoza wycinka odpowiadało za 325 z 424 niepowodzeń wariantu autorskiego, a w praktyce zapytania te kończyły się błędnie odgadniętą kolumną klucza obcego. Kierunkiem dalszych prac jest iteracyjne uzupełnianie wycinka na żądanie modelu (narzędzie MCP zwracające DDL brakującej tabeli), jawne podawanie w wycinku par kolumn złączeń oraz walidacja identyfikatorów względem katalogu przed wykonaniem zapytania;

5. **Transakcje kompensacyjne, tożsamość i interoperacyjność agentów:** Brak semantyki transakcyjnej w MCP można adresować adaptacją wzorca SAGA [14]. Framework SagaLLM [8] powierza jednak generowanie akcji kompensujących samemu modelowi, co grozi halucynacją procedury naprawczej, natomiast architektura RAC [45] przenosi deklaratywnie zdefiniowane kompensacje do warstwy middleware. Drugim kierunkiem jest delegacja tożsamości i autoryzacja drobnoziarnista (OAuth 2.0, RFC 8693), umożliwiająca egzekwowanie uprawnień na poziomie wierszy w sposób przezroczysty dla modelu, trzecim sprzężenie bramy MCP z protokołami komunikacji międzyagentowej (A2A, ACP) [10], w którym brama stałaby się wspólną magistralą danych dla wielu agentów bez multiplikowania sesji bazodanowych.

Bibliografia

[1] Ainslie, J., Lee-Thorp, J., Jong, M. de, Zemlyanskiy, Y., Lebrón, F. i Sanghai, S., „GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints", w: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2023, s. 4895–4901. DOI: 10.18653/v1/2023.emnlp-main.298

[2] Anthropic, *Model Context Protocol Specification*, <https://modelcontextprotocol.io>, Dostęp uzyskano: 2024-11-25, 2024.

[3] Apache Arrow Community, *Apache Arrow: A cross-language development platform for in-memory data*, <https://arrow.apache.org/>, Dostęp uzyskano: 2024-11-20, 2024.

[4] Apache Arrow Community, *Arrow Flight RPC: High-performance data transport framework*, <https://arrow.apache.org/docs/format/Flight.html>, Dostęp uzyskano: 2024-11-20, 2024.

[5] Barabási, A.-L. i Albert, R., „Emergence of Scaling in Random Networks", *Science*, t. 286, nr 5439, s. 509–512, 1999, doi: 10.1126/science.286.5439.509.

[6] Belshe, M., Peon, R. i Thomson, M., „Hypertext Transfer Protocol Version 2 (HTTP/2)", RFC Editor, RFC 7540, maj 2015, doi: 10.17487/RFC7540.

[7] Bray, T., „The JavaScript Object Notation (JSON) Data Interchange Format", Internet Engineering Task Force (IETF), RFC 8259, grudzień 2017, doi: 10.17487/RFC8259.

[8] Chang, Y., Geng, J. i Tang, Y., „SagaLLM: Context Management, Validation, and Transaction Failure Recovery for Large Language Model Systems", *arXiv preprint arXiv:2503.11951*, 2025, arXiv:2503.11951.

[9] Chase, H., *LangChain: Building applications with LLMs through composability*, <https://github.com/langchain-ai/langchain>, 2022.

[10] Chen, Z., Zhao, Y., Li, H., Wang, J. i An, B., „A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A)", *arXiv preprint arXiv:2505.02279*, 2025, doi: 10.48550/arXiv.2505.02279.

[11] Cormack, G. V., Clarke, C. L. A. i Buettcher, S., „Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods", w: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09)*, doi: 10.1145/1571941.1572114, 2009, s. 758–759.

[12] Fielding, R. T., „Architectural Styles and the Design of Network-based Software Architectures", rozprawa doktorska, University of California, Irvine, 2000.

[13] Gao, D. i in., „Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation", *Proceedings of the VLDB Endowment*, t. 17, nr 5, s. 1132–1145, 2024, doi: 10.14778/3641204.3641221.

[14] Garcia-Molina, H. i Salem, K., „Sagas", *ACM SIGMOD Record*, t. 16, nr 3, s. 249–259, 1987, doi: 10.1145/38714.38742.

[15] Google LLC, *Protocol Buffers: Language-Neutral, Platform-Neutral Extensible Mechanisms for Serializing Structured Data*, <https://protobuf.dev/>, Dostęp uzyskano: 2024-11-15, 2024.

[16] Gray, J. N., „Notes on Data Base Operating Systems", w: *Operating Systems, An Advanced Course*, seria Lecture Notes in Computer Science, t. 60, doi: 10.1007/3-540-08755-9_9, Springer-Verlag Berlin Heidelberg, 1978, s. 393–481.

[17] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. i Fritz, M., „Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", w: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISEC '23)*, doi: 10.1145/3605764.3623982, 2023, s. 79–90.

[18] Hardy, N., „The Confused Deputy: (or why I know how to stop the hackers)", *ACM SIGOPS Operating Systems Review*, t. 22, nr 4, s. 36–38, 1988, doi: 10.1145/54289.848247.

[19] Hickson, I., „Server-Sent Events", World Wide Web Consortium (W3C), W3C Recommendation, luty 2015, <https://www.w3.org/TR/eventsource/>.

[20] Hou, X., Zhao, Y., Wang, S. i Wang, H., *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*, 2025. arXiv: 2503.23278.

[21] JSON-RPC Working Group, *JSON-RPC 2.0 Specification*, <https://www.jsonrpc.org/specification>, Dostęp uzyskano: 2024-11-20, 2010.

[22] Kane, A., *pgvector: Open-source vector similarity search for Postgres*, <https://github.com/pgvector/pgvector>, 2024.

[23] Kirkpatrick, J. i in., „Overcoming catastrophic forgetting in neural networks", *Proceedings of the National Academy of Sciences (PNAS)*, t. 114, nr 13, s. 3521–3526, 2017, doi: 10.1073/pnas.1611835114.

[24] Klisura, Đ. i Rios, A., „Unmasking Database Vulnerabilities: Zero-Knowledge Schema Inference Attacks in Text-to-SQL Systems", *arXiv preprint arXiv:2406.14545*, 2024, arXiv:2406.14545.

[25] Kothyari, M., Dhingra, D., Sarawagi, S. i Chakrabarti, S., „CRUSH4SQL: Collective Retrieval Using Schema Hallucination For Text2SQL", w: *Proceedings of EMNLP 2023*, 2023.

[26] Kreps, J., Narkhede, N. i Rao, J., „Kafka: A distributed messaging system for log processing", *Proceedings of the NetDB*, t. 11, s. 1–7, 2011.

[27] Lei, F., Chen, J., Ye, Y. i in., „Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows", w: *International Conference on Learning Representations (ICLR)*, arXiv:2411.07763, 2025.

[28] Levy, M., Jacoby, A. i Goldberg, Y., „Same Task, More Tokens: the Impact of Input Length on Language Model Reasoning", *arXiv preprint arXiv:2402.14848*, 2024, arXiv:2402.14848.

[29] Lewis, P. i in., „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", w: *Advances in Neural Information Processing Systems (NeurIPS 2020)*, t. 33, 2020, s. 9459–9474.

[30] Li, H., Zhang, J., Li, C. i Chen, H., „RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL", w: *Proceedings of the AAAI Conference on Artificial Intelligence*, t. 37, 2023.

[31] Li, H., Zhang, J., Li, C. i Chen, H., „CodeS: Towards Building Open-source Language Models for Text-to-SQL", *Proceedings of the ACM on Management of Data (SIGMOD 2024)*, t. 2, nr 3, s. 1–28, 2024, doi: 10.1145/3654930.

[32] Liu, N. F. i in., „Lost in the Middle: How Language Models Use Long Contexts", *Transactions of the Association for Computational Linguistics (TACL)*, t. 12, s. 157–173, 2024, doi: 10.1162/tacl_a_00638.

[33] Lowin, J., *FastMCP: High-level Python Framework for Model Context Protocol*, <https://github.com/jlowin/fastmcp>, 2024.

[34] MagicStack Inc., *asyncpg: A fast PostgreSQL Database Client Library for Python/asyncio*, <https://github.com/MagicStack/asyncpg>, 2024.

[35] Malkov, Y. A. i Yashunin, D. A., „Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 42, nr 4, s. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[36] Mao, T., *sqlglot: An easily customizable SQL parser and transpiler*, <https://github.com/tobymao/sqlglot>, 2024.

[37] Mialon, G. i in., „Augmented Language Models: a Survey", *arXiv preprint arXiv:2302.07842*, 2023.

[38] Microsoft Corporation, *Language Server Protocol Specification - 3.17*, <https://microsoft.github.io/language-server-protocol/>, Dostęp uzyskano: 2024-11-15, 2024.

[39] Model Context Protocol, *Model Context Protocol Specification, rewizja 2025-06-18*, https://modelcontextprotocol.io/specification/2025-06-18, Dostęp uzyskano: 2026-10-04, 2025.

[40] OpenAI, *tiktoken: Fast BPE tokeniser for use with OpenAI's models*, <https://github.com/openai/tiktoken>, 2023.

[41] OWASP Foundation, „OWASP Top 10 for Large Language Model Applications and Generative AI", Open Web Application Security Project, Technical Report, 2025, <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.

[42] Patil, S. G., Zhang, T., Wang, X. i Gonzalez, J. E., „Gorilla: Large Language Model Connected with Massive APIs", *arXiv preprint arXiv:2305.15334*, 2023, doi: 10.48550/arXiv.2305.15334.

[43] Pedro, R., Castro, D., Carreira, P. i Santos, N., „From Prompt Injections to SQL Injection Attacks: How Protected is Your LLM-Integrated Web Application?", *arXiv preprint arXiv:2308.01990*, 2023, arXiv:2308.01990.

[44] Peffers, K., Tuunanen, T., Rothenberger, M. A. i Chatterjee, S., „A Design Science Research Methodology for Information Systems Research", *Journal of Management Information Systems*, t. 24, nr 3, s. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[45] Perera, R., Fernando, S. i Weerawarana, S., „Robust Agent Compensation (RAC): Log-Based Compensation for Agentic Systems", w: *Proceedings of the ACM Conference on AI and Agentic Systems*, arXiv:2605.03409, 2026.

[46] Pourreza, M. i Rafiei, D., „DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction", w: *Advances in Neural Information Processing Systems (NeurIPS 2023)*, t. 36, 2023, s. 37 533–37 545.

[47] Qin, Y. i in., „Tool Learning with Foundation Models", *arXiv preprint arXiv:2304.08354*, 2023.

[48] Qwen Team, Yang, A. i in., *Qwen2.5 Technical Report*, arXiv:2412.15115, 2024. arXiv: 2412.15115.

[49] Reimers, N. i Gurevych, I., „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", *arXiv preprint arXiv:1908.10084*, 2019.

[50] Schick, T. i in., „Toolformer: Language Models Can Teach Themselves to Use Tools", w: *Advances in Neural Information Processing Systems (NeurIPS 2023)*, doi: 10.48550/arXiv.2302.04761, t. 36, 2023, s. 68 537–68 551.

[51] Talaei, S., Pourreza, M., Chang, Y.-C., Saberi, A., Fazel, F. i Rawassizadeh, R., „CHESS: Contextual Harnessing for Efficient SQL Synthesis", *arXiv preprint arXiv:2405.16755*, 2024, arXiv:2405.16755.

[52] The gRPC Authors, *gRPC: A High Performance, Open Source Universal RPC Framework*, <https://grpc.io/docs/what-is-grpc/introduction/>, Dostęp uzyskano: 2024-11-15, 2023.

[53] Touvron, H., Martin, L., Stone, K. i in., *Llama 2: Open Foundation and Fine-Tuned Chat Models*, arXiv:2307.09288, 2023. arXiv: 2307.09288.

[54] Vaswani, A. i in., „Attention Is All You Need", w: *Advances in Neural Information Processing Systems (NIPS 2017)*, t. 30, 2017, s. 5998–6008.

[55] Wang, B. i in., „MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL", *arXiv preprint arXiv:2312.11242*, 2024, arXiv:2312.11242.

[56] Wang, X. i in., „Executable Code Actions Elicit Better LLM Agents", w: *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024, s. 51 640–51 675.

[57] Welford, B. P., „Note on a Method for Calculating Corrected Sums of Squares and Products", *Technometrics*, t. 4, nr 3, s. 419–420, 1962, doi: 10.1080/00401706.1962.10490022.

[58] Wilcoxon, F., „Individual Comparisons by Ranking Methods", *Biometrics Bulletin*, t. 1, nr 6, s. 80–83, 1945, doi: 10.2307/3001968.

[59] Wright, A., Andrews, H., Hutton, B. i Dennis, G., „JSON Schema: A Media Type for Describing JSON Data Structures", Internet Engineering Task Force (IETF), Internet-Draft, 2022, draft-bhutton-json-schema-01.

[60] Yao, S. i in., „ReAct: Synergizing Reasoning and Acting in Language Models", w: *International Conference on Learning Representations (ICLR)*, 2023.

Wykaz skrótów i symboli

A. Wykaz akronimów i skrótów

AMR	*Attack Mitigation Rate* (Odsetek zablokowanych wektorów ataku SQL Injection)
API	*Application Programming Interface* (Interfejs Programowania Aplikacji)
AST	*Abstract Syntax Tree* (Abstrakcyjne Drzewo Składniowe)
BPE	*Byte Pair Encoding* (Algorytm tokenizacji podsłów)
CTE	*Common Table Expression* (Nazwane wyrażenie tablicowe SQL)
DDL	*Data Definition Language* (Język Definicji Danych)
DML	*Data Manipulation Language* (Język Manipulacji Danymi)
E2E / $\mathrm{L}_{\mathbf{E2E}}$	*End-to-End Latency* (Całkowite opóźnienie realizacji zapytania)
EX	*Execution Accuracy* (Dokładność wykonawcza kodu Text-to-SQL)
FK	*Foreign Key* (Klucz obcy w relacyjnej bazie danych)
FPR	*False Positive Rate* (Odsetek błędnie zablokowanych poprawnych zapytań)
HNSW	*Hierarchical Navigable Small World* (Grafowy indeks wektorowy k-NN)
IPC	*Inter-Process Communication* (Komunikacja międzyprocesowa)
JSON-RPC	*JavaScript Object Notation Remote Procedure Call* (Protokół zdalnego wywoływania procedur)
LLM	*Large Language Model* (Wielki Model Językowy)
MCP	*Model Context Protocol* (Otwarty standard komunikacji modeli agentowych z kontekstem)
MVCC	*Multi-Version Concurrency Control* (Wielowersyjna kontrola współbieżności)
OOB	*Out-of-Band* (Transmisja obiektów poza głównym pasmem sterującym)
POSIX SHM	*POSIX Shared Memory* (Współdzielony segment pamięci operacyjnej `/dev/shm`)
RAG	*Retrieval-Augmented Generation* (Generowanie wspomagane wyszukiwaniem)

RPS	*Requests Per Second* (Przepustowość transakcyjna systemu)
RRF	*Reciprocal Rank Fusion* (Algorytm fuzji rang w wyszukiwaniu hybrydowym)
RTT	*Round-Trip Time* (Dwukierunkowe opóźnienie propagacji pakietu sieciowego)
SSE	*Server-Sent Events* (Jednokierunkowy strumień zdarzeń serwera HTTP)
TTFT	*Time-to-First-Token* (Czas do otrzymania pierwszego fragmentu odpowiedzi modelu)
URI	*Uniform Resource Identifier* (Ujednolicony identyfikator zasobu)

B. Wykaz symboli

$\mathrm{B}_{\textbf{tok}}$	Konfigurowalny budżet tokenów dla pojedynczej kwerendy
$\mathrm{B}_{\textbf{net}}$	Przepustowość łącza sieciowego w modelu transferu Out-of-Band
v_{x}	Przepustowość operacji x (kodowanie, dekodowanie, SHA-256, kopiowanie) w modelu transferu Out-of-Band
d	Wymiarowość przestrzeni osadzeń wektorowych ($\mathrm{d} = 384$ dla modelu `all-MiniLM-L6-v2`)
δ_{x}	Stały narzut trybu transferu x, niezależny od rozmiaru ładunku
$\mathrm{h}_{\textbf{KV}}$	Liczba głów kluczy i wartości w mechanizmie uwagi (GQA)
K	Rozmiar strony pobieranej w procedurze paginacji kursorowej (liczba wierszy)
$\mathrm{k}_{\textbf{RRF}}$	Stała stabilizująca fuzję rang Reciprocal Rank Fusion ($\mathrm{k}_{\mathrm{RRF}} = 60$)
M	Liczba współbieżnych klientów (zadań asyncio w jednym procesie) w Scenariuszu C1
$\mathrm{M}_{\textbf{exp}}$	Maksymalna liczba tabel mostkujących dodawanych przez ekspansję grafową ($\mathrm{M}_{\mathrm{exp}} = 3$)
$\mu_{\mathrm{n}}, \sigma_{\mathrm{n}}$	Bieżąca estymacja średniej oraz odchylenia standardowego średnich paczkowych liczby tokenów na wiersz w algorytmie Welforda
N	Liczebność zbioru ewaluacyjnego Text-to-SQL ($\mathrm{N} = 500$) lub liczba rekordów w tabeli analitycznej ($\mathrm{N} = 1\ 000\ 000$)
S	Rozmiar ładunku binarnego w bajtach
$\mathrm{S}^{*}(\textbf{RTT})$	Punkt równowagi (próg opłacalności) transferu Out-of-Band w funkcji opóźnienia sieciowego
$\mathrm{S}^{*}_{\textbf{SHM}}$	Próg opłacalności transferu przez pamięć współdzieloną zmierzony na jednym węźle ($12{,}9$ KiB)

$\mathrm{R}_{\mathbf{res}}$	Odsetek rekordów z pozostałymi tokenami sterującymi po enkapsulacji XML
$\mathrm{W}, \mathrm{U}, \mathrm{p}, \mathrm{r}$	Statystyki testów Wilcoxona (W) i Manna–Whitneya (U), wartość p oraz korelacja rangowo-biserialna r

Spis rysunków

1 Diagram logiczny trójwarstwowej architektury bramy MCP 33

2 Rozkład stopni grafu kluczy obcych schematu syntetycznego 47

3 Table-Recall a liczba tokenów DDL 61
4 Table-Recall@k oraz Table-Precision@k ($\mathrm{N} = 500$) 62
5 Valid SQL oraz Execution Accuracy dla profili 3B i 7B 63
6 Dekompozycja przyczyn niepowodzenia zapytań SQL 66
7 Badanie ablacyjne parametru $\mathrm{M}_{\mathrm{exp}}$ 67
8 TTFT w funkcji liczby tokenów promptu 68
9 Progi opłacalności transferu Out-of-Band 69
10 Opóźnienie paginacji w funkcji offsetu na 1 mln rekordów 72
11 Przepustowość oraz opóźnienie pętli zdarzeń pod obciążeniem współbieżnym 75

Spis tabel

1 Syntetyczne porównanie paradygmatów integracji modeli językowych z zasobami danych 20
2 Cykl życia sesji, warstwy transportowe i prymitywy semantyczne protokołu MCP (rewizja 2025-06-18) . . . 22
3 Porównanie architektur komunikacyjnych w systemach agentowych . . . 27
4 Kontrakt interfejsu narzędziowego bramy MCP . . . 34
5 Parametry konfiguracji deweloperskiej (kontenery Docker; nieużywana w rundzie pomiarowej) . . . 46
6 Środowisko rundy pomiarowej (odczytane automatycznie i zapisane w manifeście rundy, plik `environment.json`) . . . 46
7 Zestawienie scenariuszy badawczych, aparatury pomiarowej oraz metod weryfikacji statystycznej . . . 53
8 Jakość selekcji schematu i objętość kontekstu ($\mathrm{N} = 500$) . . . 60
9 Valid SQL oraz EX dla profili 3B i 7B ($\mathrm{N} = 500$) . . . 62
10 Testy McNemara dla EX: Graph@5 wobec strategii bazowych . . . 64
11 Taksonomia błędów Text-to-SQL . . . 65
12 TTFT w funkcji liczby tokenów promptu . . . 67
13 Punkty przecięcia S^* transferu OOB i In-Band . . . 69
14 Parametry modelu progu S^* wyznaczone ze składowych czasów . . . 70
15 Mediany opóźnień paginacji w funkcji offsetu . . . 71
16 Wpływ postaci predykatu kursorowego na opóźnienie keyset . . . 73
17 Plany wykonania predykatów DNF i UNION ALL . . . 73
18 Wpływ obciążenia współbieżnego M na parametry bramy . . . 74
19 Ewaluacja bezpieczeństwa modułu SQL Sanitizer . . . 77
20 Testy statystyczne hipotez badawczych . . . 78
21 Zestawienie hipotez cząstkowych i pytań badawczych z kluczowymi wynikami . . . 83
22 Syntetyczna macierz kompromisów inżynierskich modułów bramy MCP . . . 85

Spis załączników

1 Kod źródłowy implementacji referencyjnej oraz środowiska badawczego 105

2 Wybrane listingi implementacji 109

Załącznik 1

Kod źródłowy implementacji referencyjnej oraz środowiska badawczego

Kompletny kod źródłowy opracowanego systemu, definicje środowiska badawczego, skrypty powtarzalnych eksperymentów ewaluacyjnych pliki źródłowe tekstu pracy oraz przykładowe prompty wykorzystane podczas jej redakcji udostępniono w publicznym repozytorium Git w serwisie GitHub pod adresem:

`https://github.com/karol-pranko/mcp-thesis`

Wszystkie ścieżki plików przywoływane w treści pracy i w listingach podano względem katalogu głównego tego repozytorium.

Struktura repozytorium

Poniżej przedstawiono strukturę głównych katalogów i plików repozytorium:

- `src/mcp_server/` – kod źródłowy autorskiej bramy pośredniczącej protokołu Model Context Protocol:
  - `server.py` – główny moduł serwera FastMCP pod Pythonem 3.12, rejestracja narzędzi analitycznych, zasobów kontekstowych oraz obsługa cyklu życia sesji;
  - `schema_engine.py` – silnik selekcji schematu (BM25, wyszukiwanie wektorowe pgvector, fuzja RRF, ekspansja grafowa o tabele mostkujące);
  - `pagination.py` – algorytm adaptacyjnej paginacji kursorowej (Smart Pagination) z dynamiczną estymacją budżetu tokenów (tiktoken BPE);
  - `streaming.py` – konsument zdarzeń telemetrycznych Apache Kafka (`aiokafka`) z buforem anomalii udostępnianych jako zasoby MCP;
  - `out_of_band.py` – dyspozytor transferu danych binarnych poza ramką JSON-RPC (pamięć współdzielona POSIX, deskryptory z podpisanym URI);
  - `security.py` – wielowarstwowa zapora bezpieczeństwa zapytań SQL Sanitizer oparta na weryfikacji drzew składniowych AST (SQLGlot).
- `src/agent_client/` – klient agentowy w paradygmacie ReAct (`agent_loop.py`) oraz klient MCP wykorzystywany przez aparaturę pomiarową (`benchmark_client.py`);
- `testbed/` – definicje infrastruktury badawczej i zasilania danymi:
  - `docker-compose.yml` – plik orkiestracji kontenerów PostgreSQL 16 (z rozszerzeniem `pgvector`) oraz Apache Kafka w trybie KRaft (konfiguracja deweloperska);
  - `init-scripts/` – skrypty DDL oraz generatory syntetycznych struktur hurtowni danych (do 1000 relacji) oraz wolumenu 1 miliona rekordów analitycznych.
- `benchmarks/` – aparatura pomiarowa i narzędzia ewaluacyjne:

  - `scenarios/` – pomocnicze scenariusze pomiarowe opisane w Rozdziale 6: A (selekcja schematu i Text-to-SQL, ablacja M_{exp}, czas do pierwszego tokenu, dryf schematu), B1 (transfer Out-of-Band), B2 (paginacja keyset i postać predykatu), C1 (obciążenie współbieżne walidatora) oraz D (bezpieczeństwo);
  - `config/benchmark_config.json` – jedyne źródło konfiguracji rundy (liczności, ziarna, profile modeli);
  - `analysis_final/`, `analysis_offline/` – skrypty analiz wyników rundy i analiz wtórnych, generujące makra liczbowe pracy;
  - `plot_generators/` – zautomatyzowane skrypty generujące wykresy wektorowe PDF prezentowane w rozdziale badawczym.
- `tests/` – testy jednostkowe i integracyjne `pytest`;
- `scripts/` – skrypty przygotowania środowiska testowego (`setup_runpod.sh`), kontroli gotowości (`preflight_check.py`) oraz plik przypiętych wersji usług (`versions.env`);
- `run_benchmark.sh` – jedyny punkt wejścia pełnej rundy pomiarowej;
- `doc/` – kompletne pliki źródłowe tekstu pracy w systemie LaTeX, arkusze stylów `EE-dyplom.cls`, pliki bibliograficzne BibLaTeX oraz materiały graficzne;
- `pyproject.toml`, `uv.lock`, `requirements.txt` – specyfikacja i zamrożone wersje zależności środowiska Python 3.12;
- `README.md` – opis repozytorium i instrukcja uruchomienia.

Wymagania systemowe i instrukcja uruchomienia

Wyniki przedstawione w Rozdziale 6 uzyskano w jednej rundzie pomiarowej wykonanej natywnie (bez kontenerów) na instancji chmurowej z akceleratorem GPU. Odtworzenie rundy wymaga:

- systemu Linux (Ubuntu 22.04 lub nowszego) z akceleratorem NVIDIA (do uruchomienia modelu 7B w rozsądnym czasie zalecane co najmniej 10 GiB pamięci GPU) i sterownikiem CUDA;
- Pythona 3.12, PostgreSQL 16 z rozszerzeniem pgvector, Apache Kafka oraz serwera Ollama z modelami `qwen2.5-coder:3b` i `qwen2.5-coder:7b` (dokładne wersje i skróty wag zapisuje manifest rundy);
- co najmniej 16 GB pamięci RAM i kilkudziesięciu GB miejsca na dysku.

Procedura uruchomienia:

1. Sklonowanie repozytorium i przejście do jego katalogu głównego:

   ```
   git clone https://github.com/karol-pranko/mcp-thesis.git && cd mcp-thesis
   ```

2. Instalacja usług, środowiska Pythona, pobranie modeli oraz zasilenie bazy (schemat 1000 tabel z `generate_enterprise_schema.py`, dane tabel i relacja 1 mln wierszy z `seed_1m_records.py`):

   ```
   bash scripts/setup_runpod.sh
   ```

3. Uruchomienie pełnej rundy pomiarowej (wszystkie scenariusze, analiza statystyczna, wykresy i walidacja; wyniki wraz z manifestem w katalogu wskazanym przez `RESULTS_DIR`):

```
./run_benchmark.sh --archive-existing --llm-injection
```

4. Odtworzenie liczb i tabel pracy z wyników rundy:

```
python benchmarks/analysis_final/analiza.py && python
    benchmarks/analysis_final/generuj_makra.py
```

Plik `testbed/docker-compose.yml` opisuje konfigurację deweloperską (PostgreSQL z pgvector oraz Kafka w kontenerach) przeznaczoną do lokalnych testów; nie była ona użyta w rundzie pomiarowej.

Załącznik 2

Wybrane listingi implementacji

W załączniku zebrano listingi kodu, do których odwołuje się treść pracy. Listingi 2–4 są skróconymi fragmentami kodu; w podpisie każdego z nich wskazano plik źródłowy. Pełny kod źródłowy jest dostępny w repozytorium https://github.com/karol-pranko/mcp-thesis opisanym w załączniku 1.

```
// 1. Żądanie klienta MCP (tools/call) - druga strona wyniku:
{ "jsonrpc": "2.0", "id": 42, "method": "tools/call",
  "params": {
    "name": "execute_query_paginated",
    "arguments": {
      "table_name": "analytics_orders",
      "order_by": "created_at DESC NULLS LAST, amount ASC NULLS LAST, id ASC",
      "cursor": "eyJvcmRlcl9zcGVjcyI6W3siY29sIjoiY3JlYXRlZF9hdCIsImRpciI6...968
    a1c654fd62f0138be36f1",
      "token_budget": 2000,
      "max_records": 500,
      "use_snapshot": true
    }
  }
}
// 2. Odpowiedź serwera (wynik narzędzia jako tekst JSON w bloku content):
{ "jsonrpc": "2.0", "id": 42,
  "result": {
    "content": [{"type": "text", "text":
      "{\"data\": [{\"id\": ..., \"created_at\": \"...\", \"amount\": \"...\", ...},
    ...],
        \"next_cursor\": \"eyJvcmRlcl9zcGVjcyI6...\",
        \"has_more\": true, \"tokens_used\": ...,
        \"index_guard\": {\"is_indexed\": true, ...},
        \"snapshot_id\": \"00000004-000003F4-1\"}"}],
    "isError": false
  }
}
```

Listing 1. Ramka wywołania narzędzia `execute_query_paginated` i odpowiedzi serwera (kursor w formacie wzoru (10), skrócony w listingu z 617 znaków, wygenerowany funkcją `encode_opaque_cursor` z pliku `src/mcp_server/pagination.py` dla granicy pierwszej strony relacji `analytics_orders`; wartości pól odpowiedzi zastąpiono znakiem „…", ponieważ odpowiedzi narzędzia nie rejestrowano w rundzie)

```
async def discover_schema_with_graph_pruning(
    self, query: str, top_k: int = 5, max_expanded_tables: int = 3, pool:
    Optional[asyncpg.Pool] = None
```

```
) -> str:
    active_pool = self._get_active_pool(pool)
    base_schemas = await self.retrieve_hybrid_rrf(query=query, top_k=top_k,
    pool=active_pool)
    base_tables = [item["table_name"] for item in base_schemas]

    G = (await self._ensure_relation_graph(active_pool)).to_undirected(
   as_view=True)
    bridge_counter: Counter[str] = Counter()

    # Wykrywanie wezlow mostkujacych na sciezkach o dlugosci dist <= 2
    for u, v in itertools.combinations(base_tables, 2):
        if u in G and v in G and nx.has_path(G, u, v):
            path = nx.shortest_path(G, source=u, target=v)
            if len(path) == 3 and path[1] not in base_tables:
                bridge_counter[path[1]] += 1

    selected_bridges = [node for node, _ in bridge_counter.most_common(
   max_expanded_tables)]
    final_tables = base_tables + [b for b in selected_bridges if b not in
   base_tables]

    # Pobranie metadanych kolumn i aktywnych kluczy obcych [...]
    return await self._synthesize_ddl_context(active_pool, query,
   final_tables, selected_bridges)
```

Listing 2. Ekspansja grafowa o tabele mostkujące – metoda SchemaEngine.discover_schema_with_graph_pruning (fragment pliku src/mcp_server/schema_engine.py; pełny kod w repozytorium)

```
# 1. Asynchroniczna inspekcja indeksow w pg_catalog z ochrona buforow
async with pool.acquire() as check_conn:
    index_check = await index_catalog_guard.verify_index_compatibility(
        check_conn, safe_table, normalized_specs)

# 2. Dynamiczne wyznaczenie paczki i prewencyjne dlawienie
batch_size = token_budget_estimator.estimate_batch_size(token_budget,
   max_records)
if not index_check.is_compatible and batch_size > 50:
    batch_size = 50

# 3. Wykonanie zapytania keyset w izolacji snapshotowej MVCC
query_sql, params = build_keyset_query(safe_table, cursor_data=cursor_data,
    fetch_limit=batch_size + 1,
                                       order_by=normalized_specs,
   use_union_all=True)
```

```
14 rows = await snapshot_manager.execute_in_snapshot(pool, active_snapshot_id,
       query_sql, params) \
15         if active_snapshot_id else await pool.fetch(query_sql, *params)
```

Listing 3. Paginacja wielokolumnowa z kontrolą indeksów i migawką MVCC – fragment funkcji `fetch_paginated_data` (plik `src/mcp_server/pagination.py`; pełny kod w repozytorium)

```
1 async def async_validate_query(sql_query: str, dialect: str = "postgres",
      max_limit: Optional[int] = None) -> ValidationResult:
2     loop = asyncio.get_running_loop()
3     pool = get_process_pool()  # ProcessPoolExecutor zainicjalizowany z
      mp_context="forkserver"
4     return await loop.run_in_executor(pool, _worker_validate_query,
      sql_query, dialect, max_limit)
```

Listing 4. Asynchroniczna walidacja AST w puli `ProcessPoolExecutor` z kontekstem `forkserver` – funkcja `async_validate_query` (plik `src/mcp_server/security.py`; pełny kod w repozytorium)